

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Cell 1 — Khai báo 4 lớp đang dùng

CELL 2.1 — Cấu hình GPU Memory Growth

```
import tensorflow as tf

gpus = tf.config.list_physical_devices('GPU')
if gpus:
    try:
        for gpu in gpus:
            tf.config.experimental.set_memory_growth(gpu, True)
        print("Đã bật memory growth cho GPU")
    except RuntimeError as e:
        print(e)
else:
    print("Không tìm thấy GPU")
print("GPU Devices:", gpus)
```

Đã bật memory growth cho GPU

GPU Devices: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

```
port time

def keep_alive():
    while True:
        print("Trạng thái: Đang hoạt động...", end="\r")
        time.sleep(60)
```

Bạn có thể chạy hàm này trong một cell riêng biệt
hoặc lồng vào vòng lặp training của mình.

```
import os

DATASET_DIR = "/content/drive/MyDrive/caffe_dataset"

CLASS_NAMES = [
```

```

    "pet_cafe",
    "study_cafe",
    "garden_cafe",
    "aesthetic_cafe"
]

# TODO: Khi có dữ liệu food_cafe, thêm lại:
# CLASS_NAMES = [
#     "pet_cafe",
#     "study_cafe",
#     "garden_cafe",
#     "aesthetic_cafe",
#     "food_cafe"
# ]

IMAGE_EXTS = (".jpg", ".jpeg", ".png", ".webp", ".bmp")

```

Cell 2 — Đếm lại số ảnh chỉ trong 4 lớp dùng để train

```

class_counts = {}

for class_name in CLASS_NAMES:
    class_dir = os.path.join(DATASET_DIR, class_name)
    files = [
        f for f in os.listdir(class_dir)
        if f.lower().endswith(IMAGE_EXTS)
    ]
    class_counts[class_name] = len(files)

print("Số lượng ảnh sẽ dùng để train:")
for k, v in class_counts.items():
    print(f"{k}: {v}")

print("Tổng số ảnh dùng để train:", sum(class_counts.values()))

```

FileNotFoundError: [Errno 2] No such file or directory: '/content/drive/MyDrive/cafe_dataset'

```

-----
FileNotFoundError                                Traceback (most recent call last)
/tmp/ipykernel_16543/2302971443.py in <cell line: 0>()
      4     class_dir = os.path.join(DATASET_DIR, class_name)
      5     files = [
----> 6         f for f in os.listdir(class_dir)

```

```

7         if f.lower().endswith(IMAGE_EXTS)
8     ]

```

FileNotFoundError: [Errno 2] No such file or directory: '/content/drive/MyDrive/cafe_dataset,'

Cell 3 — Hiển thị ngẫu nhiên ảnh mẫu mỗi lớp

```

import random
import matplotlib.pyplot as plt
from PIL import Image

def show_random_images(dataset_dir, class_names, num_samples=5):
    plt.figure(figsize=(4 * num_samples, 4 * len(class_names)))

    plot_index = 1
    for class_name in class_names:
        class_dir = os.path.join(dataset_dir, class_name)
        files = [
            f for f in os.listdir(class_dir)
            if f.lower().endswith(IMAGE_EXTS)
        ]

        sampled_files = random.sample(files, min(num_samples, len(files)))

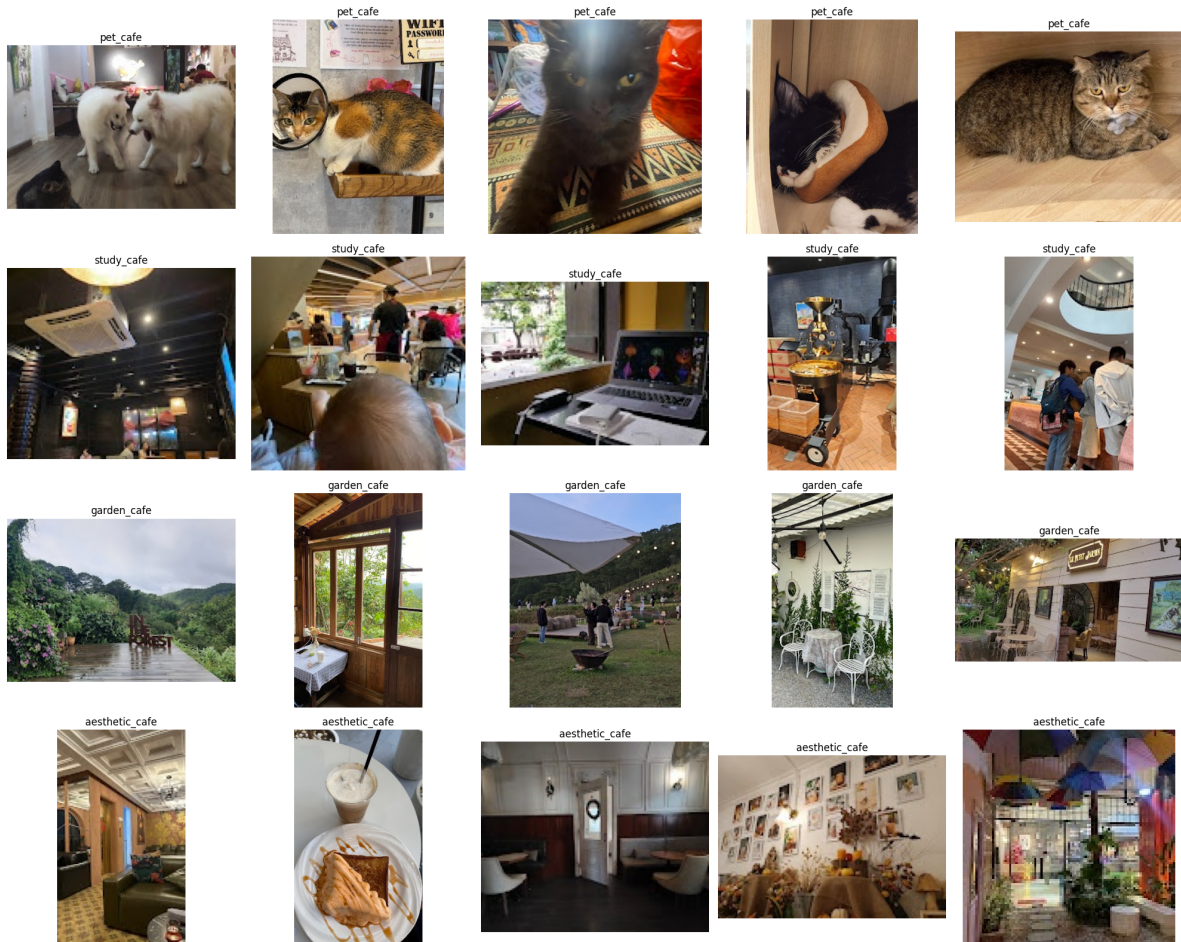
        for file_name in sampled_files:
            img_path = os.path.join(class_dir, file_name)

            try:
                img = Image.open(img_path).convert("RGB")
                plt.subplot(len(class_names), num_samples, plot_index)
                plt.imshow(img)
                plt.title(class_name)
                plt.axis("off")
                plot_index += 1
            except Exception as e:
                print(f"Lỗi đọc ảnh: {img_path} | {e}")

    plt.tight_layout()
    plt.show()

show_random_images(DATASET_DIR, CLASS_NAMES, num_samples=5)

```



BƯỚC 5B: Tạo lại split từ dataset JPG V2 và chuẩn bị pipeline nhẹ hơn

Cell 1 — Khai báo dataset mới

```
import os
import pandas as pd

DATASET_DIR = "/content/drive/MyDrive/cafe_dataset_jpg_v2"

CLASS_NAMES = [
    "pet_cafe",
    "study_cafe",
```



```

    "garden_cafe",
    "aesthetic_cafe",
    "food_cafe"
]

# TODO: Khi có food_cafe, thêm lại vào đây

IMAGE_EXTS = (".jpg",)

```

Cell 2 — Tạo DataFrame từ dataset JPG V2

Cell kiểm tra nhanh thư mục dataset JPG V2

```

import os

DATASET_DIR = "/content/drive/MyDrive/cafe_dataset_jpg_v2"

print("Dataset exists:", os.path.exists(DATASET_DIR))
print("Các thư mục tìm thấy:")

for item in os.listdir(DATASET_DIR):
    full_path = os.path.join(DATASET_DIR, item)
    if os.path.isdir(full_path):
        print("-", item)

```

```

Dataset exists: True
Các thư mục tìm thấy:
- study_cafe
- aesthetic_cafe
- garden_cafe
- pet_cafe
- food_cafe

```

Sau đó chạy cell này để đếm trực tiếp từng lớp

```

CLASS_NAMES = [
    "pet_cafe",
    "study_cafe",
    "garden_cafe",
    "aesthetic_cafe",
    "food_cafe"
]

```

```

]

for class_name in CLASS_NAMES:
    class_dir = os.path.join(DATASET_DIR, class_name)
    if os.path.exists(class_dir):
        count = len([f for f in os.listdir(class_dir) if f.lower().endswith(".jpg")])
        print(f"{class_name}: {count}")
    else:
        print(f"{class_name}: KHÔNG TỒN TẠI")

```

```

pet_cafe: 7433
study_cafe: 6168
garden_cafe: 4064
aesthetic_cafe: 4868
food_cafe: 3913

```

```

data = []

for class_name in CLASS_NAMES:
    class_dir = os.path.join(DATASET_DIR, class_name)
    files = [
        f for f in os.listdir(class_dir)
        if f.lower().endswith(IMAGE_EXTS)
    ]

    for file_name in files:
        file_path = os.path.join(class_dir, file_name)
        data.append({
            "file_path": file_path,
            "label_name": class_name
        })

df = pd.DataFrame(data)

print("Tổng số mẫu:", len(df))
print(df.head())

```

Tổng số mẫu: 26446

	file_path	label_name
0	/content/drive/MyDrive/cafe_dataset_jpg_v2/pet...	pet_cafe
1	/content/drive/MyDrive/cafe_dataset_jpg_v2/pet...	pet_cafe

```

2 /content/drive/MyDrive/cafe_dataset_jpg_v2/pet... pet_cafe
3 /content/drive/MyDrive/cafe_dataset_jpg_v2/pet... pet_cafe
4 /content/drive/MyDrive/cafe_dataset_jpg_v2/pet... pet_cafe

```

Cell 3 — Mã hóa nhãn

```

label_to_index = {label: idx for idx, label in enumerate(CLASS_NAMES)}
index_to_label = {idx: label for label, idx in label_to_index.items()}

df["label"] = df["label_name"].map(label_to_index)

print("Label mapping:")
print(label_to_index)

print("\nPhân bố nhãn:")
print(df["label_name"].value_counts())

```

Label mapping:

```
{'pet_cafe': 0, 'study_cafe': 1, 'garden_cafe': 2, 'aesthetic_cafe': 3, 'food_cafe': 4}
```

Phân bố nhãn:

```

label_name
pet_cafe      7433
study_cafe    6168
aesthetic_cafe 4868
garden_cafe   4064
food_cafe     3913
Name: count, dtype: int64

```

Cell 4 — Chia train / temp

```

from sklearn.model_selection import train_test_split

train_df, temp_df = train_test_split(
    df,
    test_size=0.30,
    stratify=df["label"],
    random_state=42
)

print("Train:", len(train_df))
print("Temp :", len(temp_df))

```

Train: 18512
Temp : 7934

Cell 5 — Chia temp thành val / test

```
val_df, test_df = train_test_split(
    temp_df,
    test_size=0.50,
    stratify=temp_df["label"],
    random_state=42
)

print("Train:", len(train_df))
print("Val  :", len(val_df))
print("Test :", len(test_df))
```

Train: 18512
Val : 3967
Test : 3967

Cell 6 — Kiểm tra phân bố lớp

```
def show_distribution(name, dataframe):
    print(f"\n===== {name} =====")
    print(dataframe["label_name"].value_counts())
    print("\nTỷ lệ:")
    print((dataframe["label_name"].value_counts(normalize=True) * 100).round(2))

show_distribution("TRAIN", train_df)
show_distribution("VALIDATION", val_df)
show_distribution("TEST", test_df)
```

```
===== TRAIN =====
label_name
pet_cafe      5203
study_cafe    4317
aesthetic_cafe 3408
garden_cafe   2845
food_cafe     2739
Name: count, dtype: int64
```

Tỷ lệ:

label_name	
pet_cafe	28.11
study_cafe	23.32
aesthetic_cafe	18.41
garden_cafe	15.37
food_cafe	14.80

Name: proportion, dtype: float64

===== VALIDATION =====

label_name	
pet_cafe	1115
study_cafe	925
aesthetic_cafe	730
garden_cafe	610
food_cafe	587

Name: count, dtype: int64

Tỷ lệ:

label_name	
pet_cafe	28.11
study_cafe	23.32
aesthetic_cafe	18.40
garden_cafe	15.38
food_cafe	14.80

Name: proportion, dtype: float64

===== TEST =====

label_name	
pet_cafe	1115
study_cafe	926
aesthetic_cafe	730
garden_cafe	609
food_cafe	587

Name: count, dtype: int64

Tỷ lệ:

label_name	
pet_cafe	28.11
study_cafe	23.34
aesthetic_cafe	18.40
garden_cafe	15.35

```
food_cafe          14.80
Name: proportion, dtype: float64
```

Cell 7 — Reset index

```
train_df = train_df.reset_index(drop=True)
val_df = val_df.reset_index(drop=True)
test_df = test_df.reset_index(drop=True)
```

Cell 8 — Lưu split mới

```
SAVE_SPLIT_DIR = "/content/drive/MyDrive/cafe_dataset_jpg_v2_splits"
os.makedirs(SAVE_SPLIT_DIR, exist_ok=True)

train_df.to_csv(os.path.join(SAVE_SPLIT_DIR, "train_split.csv"), index=False)
val_df.to_csv(os.path.join(SAVE_SPLIT_DIR, "val_split.csv"), index=False)
test_df.to_csv(os.path.join(SAVE_SPLIT_DIR, "test_split.csv"), index=False)

print("Đã lưu split mới tại:", SAVE_SPLIT_DIR)
```

Đã lưu split mới tại: /content/drive/MyDrive/cafe_dataset_jpg_v2_splits

Cell 9 — Kiểm tra overlap

```
train_paths = set(train_df["file_path"])
val_paths = set(val_df["file_path"])
test_paths = set(test_df["file_path"])

print("Train  Val :", len(train_paths.intersection(val_paths)))
print("Train  Test:", len(train_paths.intersection(test_paths)))
print("Val    Test :", len(val_paths.intersection(test_paths)))
```

```
Train  Val : 0
Train  Test: 0
Val    Test : 0
```

BƯỚC 6B: Tạo pipeline train nhẹ hơn, ổn định hơn trên Colab

6B.1 Cell 1 — Đọc lại split mới

```

import os
import pandas as pd
import tensorflow as tf

SAVE_SPLIT_DIR = "/content/drive/MyDrive/cafe_dataset_jpg_v2_splits"

train_df = pd.read_csv(os.path.join(SAVE_SPLIT_DIR, "train_split.csv"))
val_df = pd.read_csv(os.path.join(SAVE_SPLIT_DIR, "val_split.csv"))
test_df = pd.read_csv(os.path.join(SAVE_SPLIT_DIR, "test_split.csv"))

print("Train:", train_df.shape)
print("Val  :", val_df.shape)
print("Test :", test_df.shape)

```

Train: (18512, 3)

Val : (3967, 3)

Test : (3967, 3)

6B.2 Cell 2 — Khai báo tham số nhẹ hơn

```

IMG_SIZE = 160
BATCH_SIZE = 32
AUTOTUNE = tf.data.AUTOTUNE

CLASS_NAMES = [
    "pet_cafe",
    "study_cafe",
    "garden_cafe",
    "aesthetic_cafe",
    "food_cafe"
]

# TODO: Khi có food_cafe, thêm lại vào CLASS_NAMES

NUM_CLASSES = len(CLASS_NAMES)

print("IMG_SIZE =", IMG_SIZE)
print("BATCH_SIZE =", BATCH_SIZE)
print("NUM_CLASSES =", NUM_CLASSES)

```

IMG_SIZE = 160


```
BATCH_SIZE = 32
NUM_CLASSES = 5
```

6B.3 Cell 3 — Hàm đọc ảnh bằng TensorFlow native

```
def load_and_resize_image(file_path, label):
    image = tf.io.read_file(file_path)
    image = tf.image.decode_jpeg(image, channels=3)
    image = tf.image.resize(image, [IMG_SIZE, IMG_SIZE])
    image = tf.cast(image, tf.float32)
    return image, label
```

6B.4 Cell 4 — Data augmentation nhẹ

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.05),
    tf.keras.layers.RandomZoom(0.08),
], name="data_augmentation")
```

6B.5 Cell 5 — Tạo dataset function

```
def make_dataset(dataframe, training=False):
    file_paths = dataframe["file_path"].values
    labels = dataframe["label"].values

    ds = tf.data.Dataset.from_tensor_slices((file_paths, labels))
    ds = ds.map(load_and_resize_image, num_parallel_calls=AUTOTUNE)

    if training:
        ds = ds.shuffle(min(len(dataframe), 2000), reshuffle_each_iteration=True)

    ds = ds.batch(BATCH_SIZE)
    ds = ds.prefetch(AUTOTUNE)
    return ds
```

6B.6 Cell 6 — Tạo train / val / test dataset

```

train_ds = make_dataset(train_df, training=True)
val_ds = make_dataset(val_df, training=False)
test_ds = make_dataset(test_df, training=False)

print(train_ds)
print(val_ds)
print(test_ds)

```

```

<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 160, 160, 3), dtype=tf.float32, name=
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 160, 160, 3), dtype=tf.float32, name=
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 160, 160, 3), dtype=tf.float32, name=

```

6B.7 Cell 7 — Kiểm tra 1 batch

```

for images, labels in train_ds.take(1):
    print("Images shape:", images.shape)
    print("Labels shape:", labels.shape)
    print("Labels sample:", labels[:10].numpy())

```

```

Images shape: (16, 160, 160, 3)
Labels shape: (16,)
Labels sample: [4 0 0 4 3 1 2 0 3 4]

```

6B.8 Cell 8 — Hiển thị thử batch ảnh

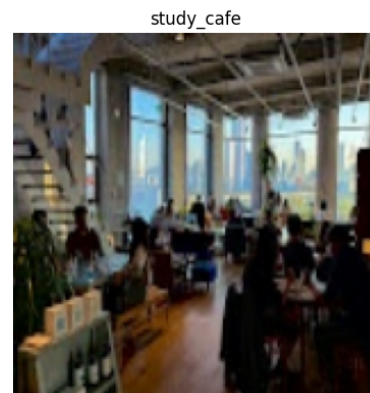
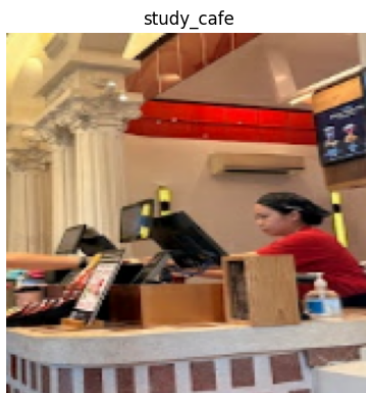
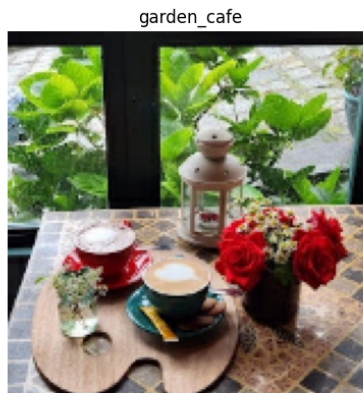
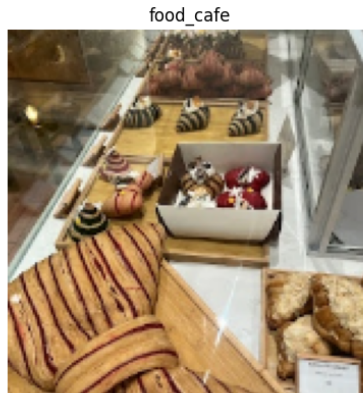
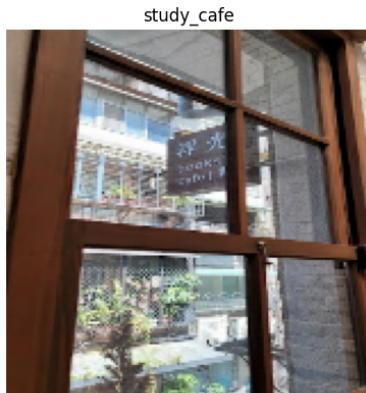
```

import matplotlib.pyplot as plt

def show_batch(dataset, class_names, num_images=9):
    plt.figure(figsize=(12, 12))
    for images, labels in dataset.take(1):
        for i in range(min(num_images, len(images))):
            ax = plt.subplot(3, 3, i + 1)
            plt.imshow(tf.cast(images[i], tf.uint8))
            plt.title(class_names[int(labels[i])])
            plt.axis("off")
    plt.tight_layout()
    plt.show()

show_batch(train_ds, CLASS_NAMES, num_images=9)

```



6B.9 Cell 9 — Hiện thị thử augmentation

```
import random
import matplotlib.pyplot as plt
from PIL import Image

for images, labels in train_ds.take(1):
    augmented_images = data_augmentation(images)
```

```
plt.figure(figsize=(12, 12))
for i in range(min(9, len(augmented_images))):
    ax = plt.subplot(3, 3, i + 1)
    img = tf.clip_by_value(augmented_images[i], 0, 255)
    plt.imshow(tf.cast(img, tf.uint8))
    plt.title(CLASS_NAMES[int(labels[i])])
    plt.axis("off")
plt.tight_layout()
plt.show()
```

study_cafe



aesthetic_cafe



pet_cafe



aesthetic_cafe



garden_cafe



pet_cafe



pet_cafe



pet_cafe



study_cafe



BƯỚC 7: Huấn luyện baseline MobileNetV2 bản nhẹ hơn

Mục tiêu Ta sẽ train baseline đầu tiên với cấu hình an toàn hơn:

dataset JPG V2

IMG_SIZE = 160

BATCH_SIZE = 16

MobileNetV2 pretrained

đóng băng backbone chỉ train head trước

7.1 Trước khi train: thêm một tối ưu để giảm nguy cơ crash

Cell 1 — Kiểm tra GPU

```
import tensorflow as tf
print("GPU:", tf.config.list_physical_devices('GPU'))
```

GPU: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

Cell 2 — Bật memory growth cho GPU

```
gpus = tf.config.list_physical_devices('GPU')
if not gpus:
    print("Không tìm thấy GPU")
# tf.config.experimental.set_memory_growth(gpu, True) đã được cấu hình ở cell 7c4115b6
```

vì TensorFlow đã khởi tạo các thiết bị GPU của bạn rồi. Khi GPU đã được khởi tạo, bạn không thể thay đổi cài đặt memory growth nữa.

Điều này thường xảy ra khi bạn chạy một số đoạn mã TensorFlow (ví dụ: tạo hoặc train mô hình) trước khi gọi

`tf.config.experimental.set_memory_growth`. Trong trường hợp của bạn, một số tác vụ TensorFlow khác đã chạy và chiếm dụng GPU

7.2 Cell 3 — Import phần model

```
from tensorflow.keras import layers
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input
```

7.3 Cell 4 — Xây model baseline nhẹ


```

NUM_CLASSES = len(CLASS_NAMES)

base_model = MobileNetV2(
    input_shape=(IMG_SIZE, IMG_SIZE, 3),
    include_top=False,
    weights="imagenet"
)

# Mở khóa toàn bộ base_model trước khi chọn các lớp cần đồng bằng
base_model.trainable = True

# Đồng bằng tất cả các lớp NGOÀI TRỪ 20 lớp cuối cùng
for layer in base_model.layers[:-20]:
    layer.trainable = False

inputs = tf.keras.Input(shape=(IMG_SIZE, IMG_SIZE, 3))

x = data_augmentation(inputs)      # augmentation chỉ ở train graph
x = preprocess_input(x)            # preprocess chuẩn cho MobileNetV2
x = base_model(x, training=False)  # Nên dùng training=True khi fine-tune để cập nhật BN
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.2)(x)
outputs = layers.Dense(NUM_CLASSES, activation="softmax")(x)

model = tf.keras.Model(inputs, outputs)

print("Số lớp trong base_model:", len(base_model.layers))
print("Một số lớp cuối và trạng thái trainable:")
for layer in base_model.layers[-25:]:
    print(f"{layer.name}: {layer.trainable}")

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5), # Giảm learning rate khi fine-tune
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

```

```

Số lớp trong base_model: 154
Một số lớp cuối và trạng thái trainable:
block_14_depthwise_BN: False

```

```

block_14_depthwise_relu: False
block_14_project: False
block_14_project_BN: False
block_14_add: False
block_15_expand: True
block_15_expand_BN: True
block_15_expand_relu: True
block_15_depthwise: True
block_15_depthwise_BN: True
block_15_depthwise_relu: True
block_15_project: True
block_15_project_BN: True
block_15_add: True
block_16_expand: True
block_16_expand_BN: True
block_16_expand_relu: True
block_16_depthwise: True
block_16_depthwise_BN: True
block_16_depthwise_relu: True
block_16_project: True
block_16_project_BN: True
Conv_1: True
Conv_1_bn: True
out_relu: True

```

Model: "functional_3"

Layer (type)	Output Shape	Param #
input_layer_6 (InputLayer)	(None, 160, 160, 3)	0
data_augmentation (Sequential)	(None, 160, 160, 3)	0
true_divide_2 (TrueDivide)	(None, 160, 160, 3)	0
subtract_2 (Subtract)	(None, 160, 160, 3)	0
mobilenetv2_1.00_160 (Functional)	(None, 5, 5, 1280)	2,257,984
global_average_pooling2d_2	(None, 1280)	0

(GlobalAveragePooling2D)		
dropout_2 (Dropout)	(None, 1280)	0
dense_2 (Dense)	(None, 5)	6,405

Total params: 2,264,389 (8.64 MB)

Trainable params: 1,212,485 (4.63 MB)

Non-trainable params: 1,051,904 (4.01 MB)

```
import multiprocessing as mp
mp.set_start_method("fork", force=True)
```

7.4 Cell 5 — Compile model

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-6),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()
```

Model: "functional_3"

Layer (type)	Output Shape	Param #
input_layer_6 (InputLayer)	(None, 160, 160, 3)	0
data_augmentation (Sequential)	(None, 160, 160, 3)	0
true_divide_2 (TrueDivide)	(None, 160, 160, 3)	0
subtract_2 (Subtract)	(None, 160, 160, 3)	0
mobilenetv2_1.00_160	(None, 5, 5, 1280)	2,257,984

(Functional)		
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None, 1280)	0
dropout_2 (Dropout)	(None, 1280)	0
dense_2 (Dense)	(None, 5)	6,405

Total params: 2,264,389 (8.64 MB)

Trainable params: 1,212,485 (4.63 MB)

Non-trainable params: 1,051,904 (4.01 MB)

7.5 Cell 6 — Tạo callback

```
callbacks = [
    tf.keras.callbacks.ModelCheckpoint(
        filepath="/content/best_mobilenetv2_stage1.keras",
        monitor="val_accuracy",
        save_best_only=True,
        verbose=1
    ),
    tf.keras.callbacks.EarlyStopping(
        monitor="val_accuracy",
        patience=3,
        restore_best_weights=True,
        verbose=1
    ),
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.2,
        patience=2,
        verbose=1
    )
]
```

```
import tensorflow as tf

print("Kiểm tra xem GPU có được TensorFlow nhận diện không:")
print(tf.config.list_physical_devices('GPU'))
```

Kiểm tra xem GPU có được TensorFlow nhận diện không:

```
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

Để kiểm tra trạng thái sử dụng GPU và RAM GPU trong thời gian thực khi huấn luyện, bạn có thể mở một terminal mới (hoặc chạy lệnh trong Colab nếu có thể chạy lệnh shell) và dùng lệnh `nvidia-smi`. Lệnh này sẽ cung cấp thông tin chi tiết về việc sử dụng GPU, bao gồm mức độ sử dụng (utilization) và bộ nhớ RAM GPU đang được cấp phát (memory usage).

Dưới đây là giải thích thêm về các nguyên nhân có thể gây ra tình trạng bạn đang gặp phải và cách khắc phục:

Nguyên nhân và Giải pháp:

- **Pipeline dữ liệu chậm:**

- **Nguyên nhân:** Nếu `tf.data.Dataset` của bạn không được tối ưu, GPU sẽ phải chờ dữ liệu từ CPU.
- **Giải pháp:** Bạn đã sử dụng AUTOTUNE, điều này rất tốt. Đảm bảo rằng các bước tiền xử lý ảnh (`load_and_resize_image`) không quá phức tạp hoặc là nút thắt cổ chai. Với các hàm `tf.image`, chúng thường rất hiệu quả.

- **Kích thước Batch nhỏ:**

- **Nguyên nhân:** GPU đạt hiệu suất cao nhất khi xử lý nhiều dữ liệu cùng lúc. Với batch size 8 hoặc 16, GPU có thể không được tận dụng tối đa.
- **Giải pháp:** Nếu RAM GPU cho phép, hãy thử tăng `BATCH_SIZE` lên 32, 64 hoặc thậm chí 128. Điều này có thể giúp GPU hoạt động hiệu quả hơn và tăng tốc độ huấn luyện tổng thể. Tuy nhiên, hãy cẩn thận để không bị lỗi out-of-memory (OOM).

- **Kiểm tra `training=False` trong `base_model`:**

- **Nguyên nhân:** Trong giai đoạn đầu tiên (Stage 1), bạn đã đặt `base_model.trainable = False` và gọi `base_model(x, training=False)`. Điều này khiến các lớp Batch Normalization trong backbone hoạt động ở chế độ inference (sử dụng các tham số đã học thay vì tính toán lại trên batch hiện tại). Điều này là đúng cho chiến lược transfer learning ban đầu, nhưng có thể ảnh hưởng đến cách GPU được sử dụng so với khi toàn bộ mô hình được fine-tune (`training=True`).

- **Giải pháp:** Khi bạn fine-tune mô hình (như ở BƯỚC 13), bạn đã đặt `base_model.trainable = True`, và do đó, các lớp Batch Normalization sẽ hoạt động ở chế độ training, có thể thay đổi mức độ tận dụng GPU.

Tóm lại: Khả năng cao nhất là sự kết hợp giữa kích thước batch nhỏ và pipeline dữ liệu chưa tối ưu hoàn toàn dẫn đến việc GPU không được sử dụng hết công suất. Hãy thử tăng `BATCH_SIZE` nếu có thể và quan sát `nvidia-smi` trong quá trình huấn luyện để xem mức độ sử dụng GPU (GPU-util) có tăng lên không.

7.6 Cell 7 — Train thử với số epoch vừa phải

```
EPOCHS = 40

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
    callbacks=callbacks
)
```

```
Epoch 1/40
1155/1157          0s 77ms/step - accuracy: 0.2980 - loss: 1.7683
```

7.7 Sau khi train xong, lưu label mapping

```
import json

index_to_label = {idx: label for idx, label in enumerate(CLASS_NAMES)}

with open("/content/index_to_label.json", "w", encoding="utf-8") as f:
    json.dump(index_to_label, f, ensure_ascii=False, indent=2)

print(index_to_label)
```

BƯỚC 8: Đánh giá mô hình chi tiết

Mục tiêu Ta sẽ không chỉ nhìn accuracy, mà đánh giá đầy đủ bằng:

accuracy trên test

classification report

confusion matrix biểu đồ loss/accuracy Điều này rất quan trọng vì mô hình phân loại nhiều lớp có thể:

accuracy ổn nhưng nhầm mạnh giữa vài lớp nào đó

8.1 Cell 1 — Load model tốt nhất

```
import tensorflow as tf

best_model = tf.keras.models.load_model("/content/best_mobilenetv2_stage1.keras")
print("Đã load model tốt nhất.")
```

8.2 Cell 2 — Đánh giá nhanh trên validation và test

```
val_loss, val_acc = best_model.evaluate(val_ds, verbose=1)
test_loss, test_acc = best_model.evaluate(test_ds, verbose=1)

print(f"\nValidation Accuracy: {val_acc:.4f}")
print(f"Validation Loss      : {val_loss:.4f}")

print(f"\nTest Accuracy      : {test_acc:.4f}")
print(f"Test Loss          : {test_loss:.4f}")
```

8.3 Cell 3 — Dự đoán trên test set

```
import numpy as np

y_true = []
y_pred = []

for images, labels in test_ds:
    preds = best_model.predict(images, verbose=0)
    pred_labels = np.argmax(preds, axis=1)

    y_true.extend(labels.numpy())
    y_pred.extend(pred_labels)

y_true = np.array(y_true)
y_pred = np.array(y_pred)

print("Số lượng mẫu test:", len(y_true))
```

8.4 Cell 4 — Classification report

```

from sklearn.metrics import classification_report

print(classification_report(
    y_true,
    y_pred,
    target_names=CLASS_NAMES,
    digits=5
))

```

8.6 Cell 5 — Confusion matrix

```

from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=CLASS_NAMES,
    yticklabels=CLASS_NAMES
)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix - MobileNetV2 Stage 1")
plt.tight_layout()
plt.show()

```

8.8 Cell 6 — Vẽ biểu đồ quá trình train

```

import matplotlib.pyplot as plt

history_dict = history.history

plt.figure(figsize=(12, 5))

# Accuracy
plt.subplot(1, 2, 1)

```

```

plt.plot(history_dict["accuracy"], label="Train Accuracy")
plt.plot(history_dict["val_accuracy"], label="Val Accuracy")
plt.title("Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

# Loss
plt.subplot(1, 2, 2)
plt.plot(history_dict["loss"], label="Train Loss")
plt.plot(history_dict["val_loss"], label="Val Loss")
plt.title("Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.tight_layout()
plt.show()

```

8.9 Cell 7 — Tính macro F1 riêng

```

from sklearn.metrics import f1_score

macro_f1 = f1_score(y_true, y_pred, average="macro")
weighted_f1 = f1_score(y_true, y_pred, average="weighted")

print(f"Macro F1    : {macro_f1:.4f}")
print(f"Weighted F1: {weighted_f1:.4f}")

```

8.11 Nếu muốn lưu confusion matrix thành ảnh

```

plt.figure(figsize=(8, 6))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=CLASS_NAMES,
    yticklabels=CLASS_NAMES
)
plt.xlabel("Predicted")

```

```
plt.ylabel("True")
plt.title("Confusion Matrix - MobileNetV2 Stage 1")
plt.tight_layout()
plt.savefig("/content/confusion_matrix_stage1.png", dpi=200)
plt.show()
```

BƯỚC 9: Train EfficientNetB0 baseline

9.1 Cell 1 — Xóa model cũ khỏi RAM và dọn session

Cell này rất quan trọng để tránh giữ model MobileNetV2 cũ trong bộ nhớ.

```
import tensorflow as tf
import gc

tf.keras.backend.clear_session()
gc.collect()

print("Đã clear session.")
```

Đã clear session.

9.2 Cell 2 — Kiểm tra GPU và bật memory growth

```
gpus = tf.config.list_physical_devices('GPU')
print("GPU:", gpus)

if not gpus:
    print("Không tìm thấy GPU")
# tf.config.experimental.set_memory_growth(gpu, True) đã được cấu hình ở cell 7c4115b6
```

GPU: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

9.3 Cell 3 — Đọc lại split


```

import os
import pandas as pd

SAVE_SPLIT_DIR = "/content/drive/MyDrive/cafe_dataset_jpg_v2_splits"

train_df = pd.read_csv(os.path.join(SAVE_SPLIT_DIR, "train_split.csv"))
val_df = pd.read_csv(os.path.join(SAVE_SPLIT_DIR, "val_split.csv"))
test_df = pd.read_csv(os.path.join(SAVE_SPLIT_DIR, "test_split.csv"))

print("Train:", train_df.shape)
print("Val  :", val_df.shape)
print("Test :", test_df.shape)

```

```

Train: (18512, 3)
Val  : (3967, 3)
Test : (3967, 3)

```

9.4 Cell 4 — Khai báo lại tham số an toàn

```

IMG_SIZE = 160
BATCH_SIZE = 32
AUTOTUNE = tf.data.AUTOTUNE

CLASS_NAMES = [
    "pet_cafe",
    "study_cafe",
    "garden_cafe",
    "aesthetic_cafe",
    "food_cafe"
]

# TODO: Khi có food_cafe, thêm lại vào đây

NUM_CLASSES = len(CLASS_NAMES)

print("IMG_SIZE =", IMG_SIZE)
print("BATCH_SIZE =", BATCH_SIZE)
print("NUM_CLASSES =", NUM_CLASSES)

```

```

IMG_SIZE = 160
BATCH_SIZE = 32
NUM_CLASSES = 5

```

9.5 Cell 5 — Hàm đọc ảnh và resize

```
import tensorflow as tf

def load_and_resize_image(file_path, label):
    image = tf.io.read_file(file_path)
    image = tf.image.decode_jpeg(image, channels=3)
    image = tf.image.resize(image, [IMG_SIZE, IMG_SIZE])
    image = tf.cast(image, tf.float32)
    return image, label
```

9.6 Cell 6 — Data augmentation nhẹ

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.05),
    tf.keras.layers.RandomZoom(0.08),
], name="data_augmentation")
```

9.7 Cell 7 — Tạo dataset function

```
def make_dataset(dataframe, training=False):
    file_paths = dataframe["file_path"].values
    labels = dataframe["label"].values

    ds = tf.data.Dataset.from_tensor_slices((file_paths, labels))
    ds = ds.map(load_and_resize_image, num_parallel_calls=AUTOTUNE)

    if training:
        ds = ds.shuffle(min(len(dataframe), 2000), reshuffle_each_iteration=True)

    ds = ds.batch(BATCH_SIZE)
    ds = ds.prefetch(AUTOTUNE)
    return ds
```

9.8 Cell 8 — Tạo dataset

```
train_ds = make_dataset(train_df, training=True)
val_ds = make_dataset(val_df, training=False)
test_ds = make_dataset(test_df, training=False)
```

```
print(train_ds)
print(val_ds)
print(test_ds)
```

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 160, 160, 3), dtype=tf.float32, name=
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 160, 160, 3), dtype=tf.float32, name=
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 160, 160, 3), dtype=tf.float32, name=
```

9.9 Cell 9 — Kiểm tra 1 batch

```
for images, labels in train_ds.take(1):
    print("Images shape:", images.shape)
    print("Labels shape:", labels.shape)
    print("Labels sample:", labels[:10].numpy())
```

```
Images shape: (8, 160, 160, 3)
Labels shape: (8,)
Labels sample: [1 1 1 1 2 2 0 4]
```

9.10 Cell 10 — Import EfficientNetB0

```
from tensorflow.keras import layers
from tensorflow.keras.applications import EfficientNetB0
```

9.11 Cell 11 — Build model EfficientNetB0 baseline

EfficientNet có preprocess riêng, nhưng trong Keras hiện đại thường đã tích hợp rescaling trong backbone.

Ta sẽ dùng trực tiếp để đơn giản và ổn định.

```
base_model = EfficientNetB0(
    input_shape=(IMG_SIZE, IMG_SIZE, 3),
    include_top=False,
    weights="imagenet"
)

base_model.trainable = False # baseline: chỉ train head

inputs = tf.keras.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
x = data_augmentation(inputs)
```

```

x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.2)(x)
outputs = layers.Dense(NUM_CLASSES, activation="softmax")(x)

effnet_model = tf.keras.Model(inputs, outputs)

```

9.12 Cell 12 — Compile model

```

effnet_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-6),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

effnet_model.summary()

```

Model: "functional_3"

Layer (type)	Output Shape	Param #
input_layer_4 (InputLayer)	(None, 160, 160, 3)	0
data_augmentation (Sequential)	(None, 160, 160, 3)	0
efficientnetb0 (Functional)	(None, 5, 5, 1280)	4,049,571
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 1280)	0
dropout_1 (Dropout)	(None, 1280)	0
dense_1 (Dense)	(None, 5)	6,405

Total params: 4,055,976 (15.47 MB)

Trainable params: 6,405 (25.02 KB)

Non-trainable params: 4,049,571 (15.45 MB)

9.13 Cell 13 — Callback

```
callbacks = [  
    tf.keras.callbacks.ModelCheckpoint(  
        filepath="/content/best_efficientnetb0_stage1.keras",  
        monitor="val_accuracy",  
        save_best_only=True,  
        verbose=1  
    ),  
    tf.keras.callbacks.EarlyStopping(  
        monitor="val_accuracy",  
        patience=3,  
        restore_best_weights=True,  
        verbose=1  
    ),  
    tf.keras.callbacks.ReduceLROnPlateau(  
        monitor="val_loss",  
        factor=0.2,  
        patience=2,  
        verbose=1  
    )  
]
```

9.14 Cell 14 — Train baseline EfficientNetB0

```
EPOCHS = 5  
  
history_effnet = effnet_model.fit(  
    train_ds,  
    validation_data=val_ds,  
    epochs=EPOCHS,  
    callbacks=callbacks  
)
```

Epoch 1/5

579/579 0s 5s/step - accuracy: 0.2055 - loss: 1.6903

Epoch 1: val_accuracy improved from None to 0.20469, saving model to /content/best_efficient

Epoch 1: finished saving model to /content/best_efficientnetb0_stage1.keras

```
579/579          3269s 6s/step - accuracy: 0.2069 - loss: 1.6883 - val_accuracy: 0.2047 -
06
Epoch 2/5
362/579          4:37 1s/step - accuracy: 0.2165 - loss: 1.6697
```

BƯỚC 10: Đánh giá EfficientNetB0 trên test và so sánh với MobileNetV2

Mục tiêu Ta sẽ:

load model EfficientNetB0 tốt nhất

evaluate trên validation/test

tạo classification report

tạo confusion matrix

tính macro F1 so sánh trực tiếp với MobileNetV2

10.1 Cell 1 — Load model tốt nhất

```
import tensorflow as tf

best_effnet_model = tf.keras.models.load_model("/content/best_efficientnetb0_stage1.keras")
print("Đã load EfficientNetB0 tốt nhất.")
```

ValueError: File not found: filepath=/content/best_efficientnetb0_stage1.keras. Please ensure

```
-----
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_6355/204805964.py in <cell line: 0>()
      1 import tensorflow as tf
      2
----> 3 best_effnet_model = tf.keras.models.load_model("/content/best_efficientnetb0_stage1.k
      4 print("Đã load EfficientNetB0 tốt nhất.")
/usr/local/lib/python3.12/dist-packages/keras/src/saving/saving_api.py in load_model(filepath)
    201     )
    202     elif str(filepath).endswith(".keras"):
--> 203         raise ValueError(
    204             f"File not found: filepath={filepath}. "
    205             "Please ensure the file is an accessible `.keras` "
ValueError: File not found: filepath=/content/best_efficientnetb0_stage1.keras. Please ensure
```

10.2 Cell 2 — Evaluate trên validation và test

```
val_loss_eff, val_acc_eff = best_effnet_model.evaluate(val_ds, verbose=1)
test_loss_eff, test_acc_eff = best_effnet_model.evaluate(test_ds, verbose=1)

print(f"\nEfficientNetB0 Validation Accuracy: {val_acc_eff:.4f}")
print(f"EfficientNetB0 Validation Loss      : {val_loss_eff:.4f}")

print(f"\nEfficientNetB0 Test Accuracy      : {test_acc_eff:.4f}")
print(f"EfficientNetB0 Test Loss           : {test_loss_eff:.4f}")
```

10.3 Cell 3 — Dự đoán trên test set

```
import numpy as np

y_true_eff = []
y_pred_eff = []

for images, labels in test_ds:
    preds = best_effnet_model.predict(images, verbose=0)
    pred_labels = np.argmax(preds, axis=1)

    y_true_eff.extend(labels.numpy())
    y_pred_eff.extend(pred_labels)

y_true_eff = np.array(y_true_eff)
y_pred_eff = np.array(y_pred_eff)

print("Số lượng mẫu test:", len(y_true_eff))
```

10.4 Cell 4 — Classification report

```
from sklearn.metrics import classification_report

print(classification_report(
    y_true_eff,
    y_pred_eff,
    target_names=CLASS_NAMES,
    digits=4
))
```

10.5 Cell 5 — Macro F1 và Weighted F1

```

from sklearn.metrics import f1_score

macro_f1_eff = f1_score(y_true_eff, y_pred_eff, average="macro")
weighted_f1_eff = f1_score(y_true_eff, y_pred_eff, average="weighted")

print(f"EfficientNetB0 Macro F1    : {macro_f1_eff:.4f}")
print(f"EfficientNetB0 Weighted F1: {weighted_f1_eff:.4f}")

```

10.6 Cell 6 — Confusion matrix

```

from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm_eff = confusion_matrix(y_true_eff, y_pred_eff)

plt.figure(figsize=(8, 6))
sns.heatmap(
    cm_eff,
    annot=True,
    fmt="d",
    cmap="Greens",
    xticklabels=CLASS_NAMES,
    yticklabels=CLASS_NAMES
)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix - EfficientNetB0 Stage 1")
plt.tight_layout()
plt.show()

```

10.7 Cell 7 — Vẽ biểu đồ train

```

history_eff_dict = history_effnet.history

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history_eff_dict["accuracy"], label="Train Accuracy")
plt.plot(history_eff_dict["val_accuracy"], label="Val Accuracy")
plt.title("EfficientNetB0 Accuracy")

```



```

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history_eff_dict["loss"], label="Train Loss")
plt.plot(history_eff_dict["val_loss"], label="Val Loss")
plt.title("EfficientNetB0 Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.tight_layout()
plt.show()

```

10.8 Cell 8 — So sánh trực tiếp với MobileNetV2

```

import pandas as pd

comparison_df = pd.DataFrame([
    {
        "Model": "MobileNetV2 Stage 1",
        "Val Accuracy": 0.8104,
        "Test Accuracy": 0.7962,
        "Macro F1": 0.7805
    },
    {
        "Model": "EfficientNetB0 Stage 1",
        "Val Accuracy": val_acc_eff,
        "Test Accuracy": test_acc_eff,
        "Macro F1": macro_f1_eff
    }
])

comparison_df

```

	Model	Val Accuracy	Test Accuracy	Macro F1
0	MobileNetV2 Stage 1	0.810400	0.796200	0.78050
1	EfficientNetB0 Stage 1	0.820414	0.816272	0.80053

BƯỚC 11: Train ResNet50 baseline

11.1 Cell 1 — Clear session và dọn RAM

```
import tensorflow as tf
import gc

tf.keras.backend.clear_session()
gc.collect()

print("Đã clear session.")
```

Đã clear session.

11.2 Cell 2 — Đọc lại split

```
import os
import pandas as pd

SAVE_SPLIT_DIR = "/content/drive/MyDrive/cafe_dataset_jpg_v2_splits"

train_df = pd.read_csv(os.path.join(SAVE_SPLIT_DIR, "train_split.csv"))
val_df = pd.read_csv(os.path.join(SAVE_SPLIT_DIR, "val_split.csv"))
test_df = pd.read_csv(os.path.join(SAVE_SPLIT_DIR, "test_split.csv"))

print("Train:", train_df.shape)
print("Val  :", val_df.shape)
print("Test :", test_df.shape)
```

Train: (18512, 3)
Val : (3967, 3)
Test : (3967, 3)

11.3 Cell 3 — Tham số an toàn

```
IMG_SIZE = 160
BATCH_SIZE = 32
AUTOTUNE = tf.data.AUTOTUNE

CLASS_NAMES = [
```

```

    "pet_cafe",
    "study_cafe",
    "garden_cafe",
    "aesthetic_cafe",
    "food_cafe"
]

# TODO: Khi có food_cafe, thêm lại vào đây

NUM_CLASSES = len(CLASS_NAMES)

print("IMG_SIZE =", IMG_SIZE)
print("BATCH_SIZE =", BATCH_SIZE)
print("NUM_CLASSES =", NUM_CLASSES)

```

```

IMG_SIZE = 160
BATCH_SIZE = 32
NUM_CLASSES = 5

```

11.4 Cell 4 — Hàm load ảnh

```

def load_and_resize_image(file_path, label):
    image = tf.io.read_file(file_path)
    image = tf.image.decode_jpeg(image, channels=3)
    image = tf.image.resize(image, [IMG_SIZE, IMG_SIZE])
    image = tf.cast(image, tf.float32)
    return image, label

```

11.5 Cell 5 — Data augmentation nhẹ

```

data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.05),
    tf.keras.layers.RandomZoom(0.08),
], name="data_augmentation")

```

11.6 Cell 6 — Tạo dataset function

```
def make_dataset(dataframe, training=False):
    file_paths = dataframe["file_path"].values
    labels = dataframe["label"].values

    ds = tf.data.Dataset.from_tensor_slices((file_paths, labels))
    ds = ds.map(load_and_resize_image, num_parallel_calls=AUTOTUNE)

    if training:
        ds = ds.shuffle(min(len(dataframe), 2000), reshuffle_each_iteration=True)

    ds = ds.batch(BATCH_SIZE)
    ds = ds.prefetch(AUTOTUNE)
    return ds
```

11.7 Cell 7 — Tạo train / val / test dataset

```
train_ds = make_dataset(train_df, training=True)
val_ds = make_dataset(val_df, training=False)
test_ds = make_dataset(test_df, training=False)

print(train_ds)
print(val_ds)
print(test_ds)
```

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 160, 160, 3), dtype=tf.float32, name=
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 160, 160, 3), dtype=tf.float32, name=
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 160, 160, 3), dtype=tf.float32, name=
```

11.8 Cell 8 — Kiểm tra 1 batch

```
for images, labels in train_ds.take(1):
    print("Images shape:", images.shape)
    print("Labels shape:", labels.shape)
    print("Labels sample:", labels[:10].numpy())
```

```
Images shape: (32, 160, 160, 3)
Labels shape: (32,)
Labels sample: [4 2 0 0 1 0 4 3 0 1]
```

11.9 Cell 9 — Import ResNet50

```

from tensorflow.keras import layers
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.applications.resnet50 import preprocess_input

```

11.10 Cell 10 — Build ResNet50 baseline

```

base_model = ResNet50(
    input_shape=(IMG_SIZE, IMG_SIZE, 3),
    include_top=False,
    weights="imagenet"
)

base_model.trainable = False # baseline: don't train head

inputs = tf.keras.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
x = data_augmentation(inputs)
x = preprocess_input(x)
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(NUM_CLASSES, activation="softmax")(x)

resnet_model = tf.keras.Model(inputs, outputs)

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_kernels_notop.h5
94765736/94765736 0s 0us/step

11.11 Cell 11 — Compile model

```

resnet_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-6),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

resnet_model.summary()

```

Model: "functional_1"

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 160, 160, 3)	0	-
data_augmentation (Sequential)	(None, 160, 160, 3)	0	input_layer_1[0]...
get_item (GetItem)	(None, 160, 160)	0	data_augmentatio...
get_item_1 (GetItem)	(None, 160, 160)	0	data_augmentatio...
get_item_2 (GetItem)	(None, 160, 160)	0	data_augmentatio...
stack (Stack)	(None, 160, 160, 3)	0	get_item[0][0], get_item_1[0][0], get_item_2[0][0]
add (Add)	(None, 160, 160, 3)	0	stack[0][0]
resnet50 (Functional)	(None, 5, 5, 2048)	23,587,712	add[0][0]
global_average_poo... (GlobalAveragePool...)	(None, 2048)	0	resnet50[0][0]
dropout (Dropout)	(None, 2048)	0	global_average_p...
dense (Dense)	(None, 5)	10,245	dropout[0][0]

Total params: 23,597,957 (90.02 MB)

Trainable params: 10,245 (40.02 KB)

Non-trainable params: 23,587,712 (89.98 MB)

11.12 Cell 12 — Callback

```

callbacks = [
    tf.keras.callbacks.ModelCheckpoint(
        filepath="/content/best_resnet50_stage1.keras",
        monitor="val_accuracy",
        save_best_only=True,
        verbose=1
    ),
    tf.keras.callbacks.EarlyStopping(
        monitor="val_accuracy",
        patience=3,
        restore_best_weights=True,
        verbose=1
    ),
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.2,
        patience=2,
        verbose=1
    )
]

```

11.13 Cell 13 — Train ResNet50 baseline

```

EPOCHS = 5

history_resnet = resnet_model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
    callbacks=callbacks
)

```

Epoch 1/5

579/579 0s 6s/step - accuracy: 0.1909 - loss: 2.8264

Epoch 1: val_accuracy improved from None to 0.19587, saving model to /content/best_resnet50_

Epoch 1: finished saving model to /content/best_resnet50_stage1.keras

579/579 4091s 7s/step - accuracy: 0.1930 - loss: 2.7511 - val_accuracy: 0.1959 -
06

Epoch 2/5

578/579 0s 142ms/step - accuracy: 0.2012 - loss: 2.5696

Epoch 2: val_accuracy improved from 0.19587 to 0.21729, saving model to /content/best_resnet

```

Epoch 2: finished saving model to /content/best_resnet50_stage1.keras
579/579          108s 173ms/step - accuracy: 0.2036 - loss: 2.5325 - val_accuracy: 0.2173
06
Epoch 3/5
578/579          0s 146ms/step - accuracy: 0.2194 - loss: 2.4008
Epoch 3: val_accuracy improved from 0.21729 to 0.23469, saving model to /content/best_resnet50_stage1.keras

Epoch 3: finished saving model to /content/best_resnet50_stage1.keras
579/579          113s 179ms/step - accuracy: 0.2200 - loss: 2.3761 - val_accuracy: 0.2347
06
Epoch 4/5
578/579          0s 141ms/step - accuracy: 0.2216 - loss: 2.3256
Epoch 4: val_accuracy improved from 0.23469 to 0.26065, saving model to /content/best_resnet50_stage1.keras

Epoch 4: finished saving model to /content/best_resnet50_stage1.keras
579/579          108s 173ms/step - accuracy: 0.2300 - loss: 2.2882 - val_accuracy: 0.2607
06
Epoch 5/5
578/579          0s 142ms/step - accuracy: 0.2516 - loss: 2.1987
Epoch 5: val_accuracy improved from 0.26065 to 0.28460, saving model to /content/best_resnet50_stage1.keras

Epoch 5: finished saving model to /content/best_resnet50_stage1.keras
579/579          113s 180ms/step - accuracy: 0.2544 - loss: 2.1836 - val_accuracy: 0.2846
06
Restoring model weights from the end of the best epoch: 5.

```

BƯỚC 12: Đánh giá ResNet50 trên test và lập bảng so sánh 3 mô hình

Mục tiêu Ta sẽ:

load model ResNet50 tốt nhất

evaluate trên validation và test

tạo classification report tạo confusion matrix

tính macro F1

lập bảng so sánh 3 model

12.1 Cell 1 — Load model tốt nhất


```
import tensorflow as tf

best_resnet_model = tf.keras.models.load_model("/content/best_resnet50_stage1.keras")
print("Đã load ResNet50 tốt nhất.")
```

Đã load ResNet50 tốt nhất.

12.2 Cell 2 — Evaluate trên validation và test

```
val_loss_res, val_acc_res = best_resnet_model.evaluate(val_ds, verbose=1)
test_loss_res, test_acc_res = best_resnet_model.evaluate(test_ds, verbose=1)

print(f"\nResNet50 Validation Accuracy: {val_acc_res:.4f}")
print(f"ResNet50 Validation Loss      : {val_loss_res:.4f}")

print(f"\nResNet50 Test Accuracy      : {test_acc_res:.4f}")
print(f"ResNet50 Test Loss             : {test_loss_res:.4f}")
```

```
124/124          721s 6s/step - accuracy: 0.2846 - loss: 1.9830
124/124          691s 6s/step - accuracy: 0.2700 - loss: 2.0033
```

```
ResNet50 Validation Accuracy: 0.2846
ResNet50 Validation Loss      : 1.9830
```

```
ResNet50 Test Accuracy      : 0.2700
ResNet50 Test Loss          : 2.0033
```

12.3 Cell 3 — Dự đoán trên test set

```
import numpy as np

y_true_res = []
y_pred_res = []

for images, labels in test_ds:
    preds = best_resnet_model.predict(images, verbose=0)
    pred_labels = np.argmax(preds, axis=1)

    y_true_res.extend(labels.numpy())
    y_pred_res.extend(pred_labels)
```

```

y_true_res = np.array(y_true_res)
y_pred_res = np.array(y_pred_res)

print("Số lượng mẫu test:", len(y_true_res))

```

Số lượng mẫu test: 3967

12.4 Cell 4 — Classification report

```

from sklearn.metrics import classification_report

print(classification_report(
    y_true_res,
    y_pred_res,
    target_names=CLASS_NAMES,
    digits=4
))

```

	precision	recall	f1-score	support
pet_cafe	0.2838	0.4215	0.3392	1115
study_cafe	0.2603	0.3078	0.2820	926
garden_cafe	0.1830	0.0706	0.1019	609
aesthetic_cafe	0.2515	0.1781	0.2085	730
food_cafe	0.3082	0.2436	0.2721	587
accuracy			0.2700	3967
macro avg	0.2573	0.2443	0.2408	3967
weighted avg	0.2605	0.2700	0.2555	3967

12.5 Cell 5 — Macro F1 và Weighted F1

```

from sklearn.metrics import f1_score

macro_f1_res = f1_score(y_true_res, y_pred_res, average="macro")
weighted_f1_res = f1_score(y_true_res, y_pred_res, average="weighted")

print(f"ResNet50 Macro F1 : {macro_f1_res:.4f}")
print(f"ResNet50 Weighted F1: {weighted_f1_res:.4f}")

```

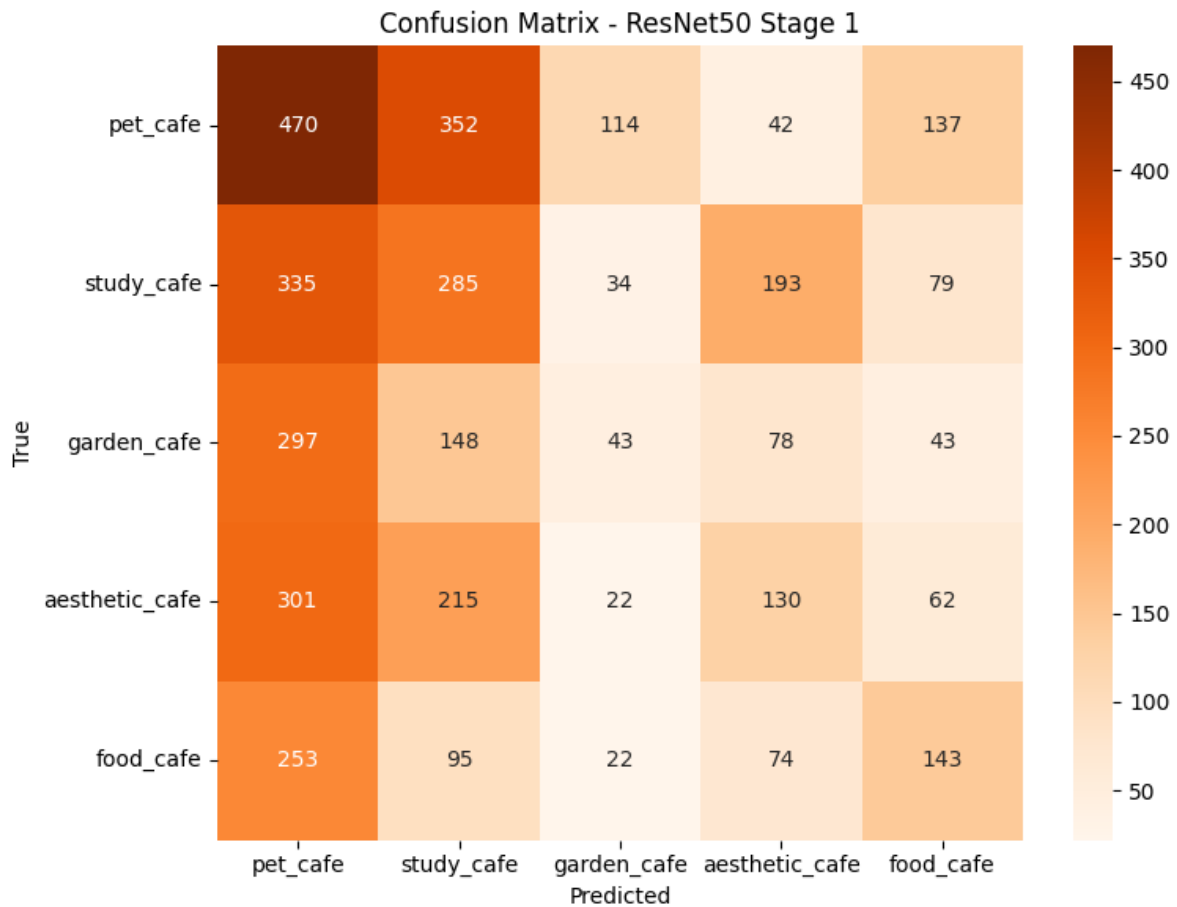
ResNet50 Macro F1 : 0.2408
ResNet50 Weighted F1: 0.2555

12.6 Cell 6 — Confusion matrix

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm_res = confusion_matrix(y_true_res, y_pred_res)

plt.figure(figsize=(8, 6))
sns.heatmap(
    cm_res,
    annot=True,
    fmt="d",
    cmap="Oranges",
    xticklabels=CLASS_NAMES,
    yticklabels=CLASS_NAMES
)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix - ResNet50 Stage 1")
plt.tight_layout()
plt.show()
```



12.7 Cell 7 — Vẽ biểu đồ train

```
history_res_dict = history_resnet.history

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history_res_dict["accuracy"], label="Train Accuracy")
plt.plot(history_res_dict["val_accuracy"], label="Val Accuracy")
plt.title("ResNet50 Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history_res_dict["loss"], label="Train Loss")
```

```
plt.plot(history_res_dict["val_loss"], label="Val Loss")
plt.title("ResNet50 Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.tight_layout()
plt.show()
```

NameError: name 'history_resnet' is not defined

```
-----
NameError                                Traceback (most recent call last)
/tmp/ipykernel_6355/2901201663.py in <cell line: 0>()
----> 1 history_res_dict = history_resnet.history
      2
      3 plt.figure(figsize=(12, 5))
      4
      5 plt.subplot(1, 2, 1)
NameError: name 'history_resnet' is not defined
```

12.8 Cell 8 — Lập bảng so sánh 3 mô hình

```
import pandas as pd

comparison_df = pd.DataFrame([
    {
        "Model": "MobileNetV2 Stage 1",
        "Val Accuracy": 0.8104,
        "Test Accuracy": 0.7962,
        "Macro F1": 0.7805
    },
    {
        "Model": "EfficientNetB0 Stage 1",
        "Val Accuracy": 0.8204,
        "Test Accuracy": 0.8163,
        "Macro F1": 0.8005
    },
    {
        "Model": "ResNet50 Stage 1",
        "Val Accuracy": val_acc_res,
        "Test Accuracy": test_acc_res,
        "Macro F1": macro_f1_res
    }
])
```

```

    }
])

comparison_df = comparison_df.sort_values(by=["Test Accuracy", "Macro F1"], ascending=False)
comparison_df

```

	Model	Val Accuracy	Test Accuracy	Macro F1
1	EfficientNetB0 Stage 1	0.820400	0.81630	0.800500
2	ResNet50 Stage 1	0.815976	0.80503	0.785058
0	MobileNetV2 Stage 1	0.810400	0.79620	0.780500

BƯỚC 13: Fine-tune EfficientNetB0

13.1 Cell 1 — Clear session

```

import tensorflow as tf
import gc

tf.keras.backend.clear_session()
gc.collect()

print("Đã clear session.")

```

Đã clear session.

13.2 Cell 2 — Đọc lại split

```

import os
import pandas as pd

SAVE_SPLIT_DIR = "/content/drive/MyDrive/cafe_dataset_jpg_v2_splits"

train_df = pd.read_csv(os.path.join(SAVE_SPLIT_DIR, "train_split.csv"))
val_df = pd.read_csv(os.path.join(SAVE_SPLIT_DIR, "val_split.csv"))
test_df = pd.read_csv(os.path.join(SAVE_SPLIT_DIR, "test_split.csv"))

print("Train:", train_df.shape)
print("Val  :", val_df.shape)
print("Test :", test_df.shape)

```

```
Train: (18512, 3)
Val   : (3967, 3)
Test  : (3967, 3)
```

13.3 Cell 3 — Khai báo lại tham số

```
IMG_SIZE = 160
BATCH_SIZE = 32
AUTOTUNE = tf.data.AUTOTUNE

CLASS_NAMES = [
    "pet_cafe",
    "study_cafe",
    "garden_cafe",
    "aesthetic_cafe",
    "food_cafe"
]

# TODO: Khi có food_cafe, thêm lại vào đây

NUM_CLASSES = len(CLASS_NAMES)

print("IMG_SIZE =", IMG_SIZE)
print("BATCH_SIZE =", BATCH_SIZE)
print("NUM_CLASSES =", NUM_CLASSES)
```

```
IMG_SIZE = 160
BATCH_SIZE = 32
NUM_CLASSES = 5
```

13.4 Cell 4 — Hàm load ảnh

```
def load_and_resize_image(file_path, label):
    image = tf.io.read_file(file_path)
    image = tf.image.decode_jpeg(image, channels=3)
    image = tf.image.resize(image, [IMG_SIZE, IMG_SIZE])
    image = tf.cast(image, tf.float32)
    return image, label
```

13.5 Cell 5 — Augmentation nhẹ

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.05),
    tf.keras.layers.RandomZoom(0.08),
], name="data_augmentation")
```

13.6 Cell 6 — Tạo dataset

```
def make_dataset(dataframe, training=False):
    file_paths = dataframe["file_path"].values
    labels = dataframe["label"].values

    ds = tf.data.Dataset.from_tensor_slices((file_paths, labels))
    ds = ds.map(load_and_resize_image, num_parallel_calls=AUTOTUNE)

    if training:
        ds = ds.shuffle(min(len(dataframe), 2000), reshuffle_each_iteration=True)

    ds = ds.batch(BATCH_SIZE)
    ds = ds.prefetch(AUTOTUNE)
    return ds

train_ds = make_dataset(train_df, training=True)
val_ds = make_dataset(val_df, training=False)
test_ds = make_dataset(test_df, training=False)
```

13.7 Cell 7 — Import EfficientNetB0

```
from tensorflow.keras import layers
from tensorflow.keras.applications import EfficientNetB0
```

13.8 Cell 8 — Xây lại model cùng kiến trúc stage 1

```
base_model = EfficientNetB0(
    input_shape=(IMG_SIZE, IMG_SIZE, 3),
    include_top=False,
    weights="imagenet"
)

base_model.trainable = False
```



```

inputs = tf.keras.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
x = data_augmentation(inputs)
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.2)(x)
outputs = layers.Dense(NUM_CLASSES, activation="softmax")(x)

effnet_finetune_model = tf.keras.Model(inputs, outputs)

```

Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb0_notop16705208/16705208 0s 0us/step

13.9 Cell 9 — Load trọng số tốt nhất từ stage 1

```

effnet_finetune_model.load_weights("/content/best_efficientnetb0_stage1.keras")
print("Đã load weights từ EfficientNetB0 stage 1.")

```

FileNotFoundError: [Errno 2] No such file or directory: '/content/best_efficientnetb0_stage1

```

-----
FileNotFoundError                                Traceback (most recent call last)
/tmp/ipykernel_1195/3296541576.py in <cell line: 0>()
----> 1 effnet_finetune_model.load_weights("/content/best_efficientnetb0_stage1.keras")
      2 print("Đã load weights từ EfficientNetB0 stage 1.")
/usr/local/lib/python3.12/dist-packages/keras/src/utils/traceback_utils.py in error_handler()
    120         # To get the full stack trace, call:
    121         # `keras.config.disable_traceback_filtering()`
--> 122         raise e.with_traceback(filtered_tb) from None
    123     finally:
    124         del filtered_tb
/usr/lib/python3.12/zipfile/__init__.py in __init__(self, file, mode, compression, allowZip64)
    1355         filemode = modeDict[filemode]
    1356         continue
-> 1357         raise
    1358         break
    1359     else:
FileNotFoundError: [Errno 2] No such file or directory: '/content/best_efficientnetb0_stage1

```

13.10 Cell 10 — Mở các lớp cuối của backbone

Ta mở khoảng 30 lớp cuối.

```

base_model.trainable = True

for layer in base_model.layers[:-30]:
    layer.trainable = False

print("Số lớp trong base_model:", len(base_model.layers))
print("Một số lớp cuối và trạng thái trainable:")
for layer in base_model.layers[-10:]:
    print(layer.name, layer.trainable)

```

```

Số lớp trong base_model: 238
Một số lớp cuối và trạng thái trainable:
block7a_se_squeeze True
block7a_se_reshape True
block7a_se_reduce True
block7a_se_expand True
block7a_se_excite True
block7a_project_conv True
block7a_project_bn True
top_conv True
top_bn True
top_activation True

```

13.11 Cell 11 — Compile với learning rate nhỏ

```

effnet_finetune_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

```

13.12 Cell 12 — Callback cho fine-tune

```

callbacks_finetune = [
    tf.keras.callbacks.ModelCheckpoint(
        filepath="/content/best_efficientnetb0_finetuned.keras",
        monitor="val_accuracy",
        save_best_only=True,
        verbose=1
    ),
    tf.keras.callbacks.EarlyStopping(

```

```

        monitor="val_accuracy",
        patience=3,
        restore_best_weights=True,
        verbose=1
    ),
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.2,
        patience=2,
        verbose=1
    )
]

```

13.13 Cell 13 — Train fine-tune

```

EPOCHS_FINE = 5

history_effnet_finetune = effnet_finetune_model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS_FINE,
    callbacks=callbacks_finetune
)

```

Epoch 1/5

1972/1972 0s 1s/step - accuracy: 0.6872 - loss: 0.8336

Epoch 1: val_accuracy improved from None to 0.81124, saving model to /content/best_efficientnetb0_finetuned.keras

Epoch 1: finished saving model to /content/best_efficientnetb0_finetuned.keras

1972/1972 3994s 2s/step - accuracy: 0.7154 - loss: 0.7517 - val_accuracy: 0.81124
05

Epoch 2/5

1970/1972 0s 45ms/step - accuracy: 0.7388 - loss: 0.6867

Epoch 2: val_accuracy improved from 0.81124 to 0.82396, saving model to /content/best_efficientnetb0_finetuned.keras

Epoch 2: finished saving model to /content/best_efficientnetb0_finetuned.keras

1972/1972 111s 53ms/step - accuracy: 0.7505 - loss: 0.6549 - val_accuracy: 0.8240
05

Epoch 3/5

1970/1972 0s 45ms/step - accuracy: 0.7596 - loss: 0.6385

Epoch 3: val_accuracy improved from 0.82396 to 0.83107, saving model to /content/best_efficientnetb0_finetuned.keras

```
Epoch 3: finished saving model to /content/best_efficientnetb0_finetuned.keras
1972/1972          111s 53ms/step - accuracy: 0.7655 - loss: 0.6173 - val_accuracy: 0.8311
05
Epoch 4/5
1970/1972          0s 46ms/step - accuracy: 0.7748 - loss: 0.5935
Epoch 4: val_accuracy improved from 0.83107 to 0.83225, saving model to /content/best_efficientnetb0_finetuned.keras

Epoch 4: finished saving model to /content/best_efficientnetb0_finetuned.keras
1972/1972          151s 57ms/step - accuracy: 0.7805 - loss: 0.5776 - val_accuracy: 0.8322
05
Epoch 5/5
1970/1972          0s 45ms/step - accuracy: 0.7862 - loss: 0.5687
Epoch 5: val_accuracy improved from 0.83225 to 0.83580, saving model to /content/best_efficientnetb0_finetuned.keras

Epoch 5: finished saving model to /content/best_efficientnetb0_finetuned.keras
1972/1972          110s 53ms/step - accuracy: 0.7905 - loss: 0.5594 - val_accuracy: 0.8358
05
Restoring model weights from the end of the best epoch: 5.
```

BƯỚC 14: Đánh giá EfficientNetB0 Fine-tuned trên test và chất model cuối cùng

Mục tiêu

Ta sẽ:

load model fine-tuned tốt nhất

evaluate trên validation/test

tạo classification report

tạo confusion matrix

tính macro F1

so sánh với:

MobileNetV2 Stage 1

EfficientNetB0 Stage 1

ResNet50 Stage 1

14.1 Cell 1 — Load model fine-tuned tốt nhất

```
import tensorflow as tf
```

```
best_effnet_finetuned = tf.keras.models.load_model("/content/best_efficientnetb0_finetuned.k  
print("Đã load EfficientNetB0 fine-tuned tốt nhất.")
```

Đã load EfficientNetB0 fine-tuned tốt nhất.

14.2 Cell 2 — Evaluate trên validation và test

```
val_loss_ft, val_acc_ft = best_effnet_finetuned.evaluate(val_ds, verbose=1)  
test_loss_ft, test_acc_ft = best_effnet_finetuned.evaluate(test_ds, verbose=1)  
  
print(f"\nEfficientNetB0 Fine-tuned Validation Accuracy: {val_acc_ft:.4f}")  
print(f"EfficientNetB0 Fine-tuned Validation Loss      : {val_loss_ft:.4f}")  
  
print(f"\nEfficientNetB0 Fine-tuned Test Accuracy       : {test_acc_ft:.4f}")  
print(f"EfficientNetB0 Fine-tuned Test Loss           : {test_loss_ft:.4f}")
```

423/423 21s 38ms/step - accuracy: 0.8358 - loss: 0.4621

423/423 689s 2s/step - accuracy: 0.8308 - loss: 0.4593

EfficientNetB0 Fine-tuned Validation Accuracy: 0.8358

EfficientNetB0 Fine-tuned Validation Loss : 0.4621

EfficientNetB0 Fine-tuned Test Accuracy : 0.8308

EfficientNetB0 Fine-tuned Test Loss : 0.4593

14.3 Cell 3 — Dự đoán trên test set

```
import numpy as np
```

```
y_true_ft = []
```

```
y_pred_ft = []
```

```
for images, labels in test_ds:  
    preds = best_effnet_finetuned.predict(images, verbose=0)  
    pred_labels = np.argmax(preds, axis=1)  
  
    y_true_ft.extend(labels.numpy())  
    y_pred_ft.extend(pred_labels)
```

```

y_true_ft = np.array(y_true_ft)
y_pred_ft = np.array(y_pred_ft)

print("Số lượng mẫu test:", len(y_true_ft))

```

Số lượng mẫu test: 3380

14.4 Cell 4 — Classification report

```

from sklearn.metrics import classification_report

print(classification_report(
    y_true_ft,
    y_pred_ft,
    target_names=CLASS_NAMES,
    digits=4
))

```

	precision	recall	f1-score	support
pet_cafe	0.9153	0.9309	0.9231	1115
study_cafe	0.8502	0.7905	0.8193	926
garden_cafe	0.7648	0.8276	0.7950	609
aesthetic_cafe	0.7355	0.7315	0.7335	730
accuracy			0.8308	3380
macro avg	0.8165	0.8201	0.8177	3380
weighted avg	0.8315	0.8308	0.8306	3380

14.5 Cell 5 — Macro F1 và Weighted F1

```

from sklearn.metrics import f1_score

macro_f1_ft = f1_score(y_true_ft, y_pred_ft, average="macro")
weighted_f1_ft = f1_score(y_true_ft, y_pred_ft, average="weighted")

print(f"EfficientNetB0 Fine-tuned Macro F1 : {macro_f1_ft:.4f}")
print(f"EfficientNetB0 Fine-tuned Weighted F1: {weighted_f1_ft:.4f}")

```

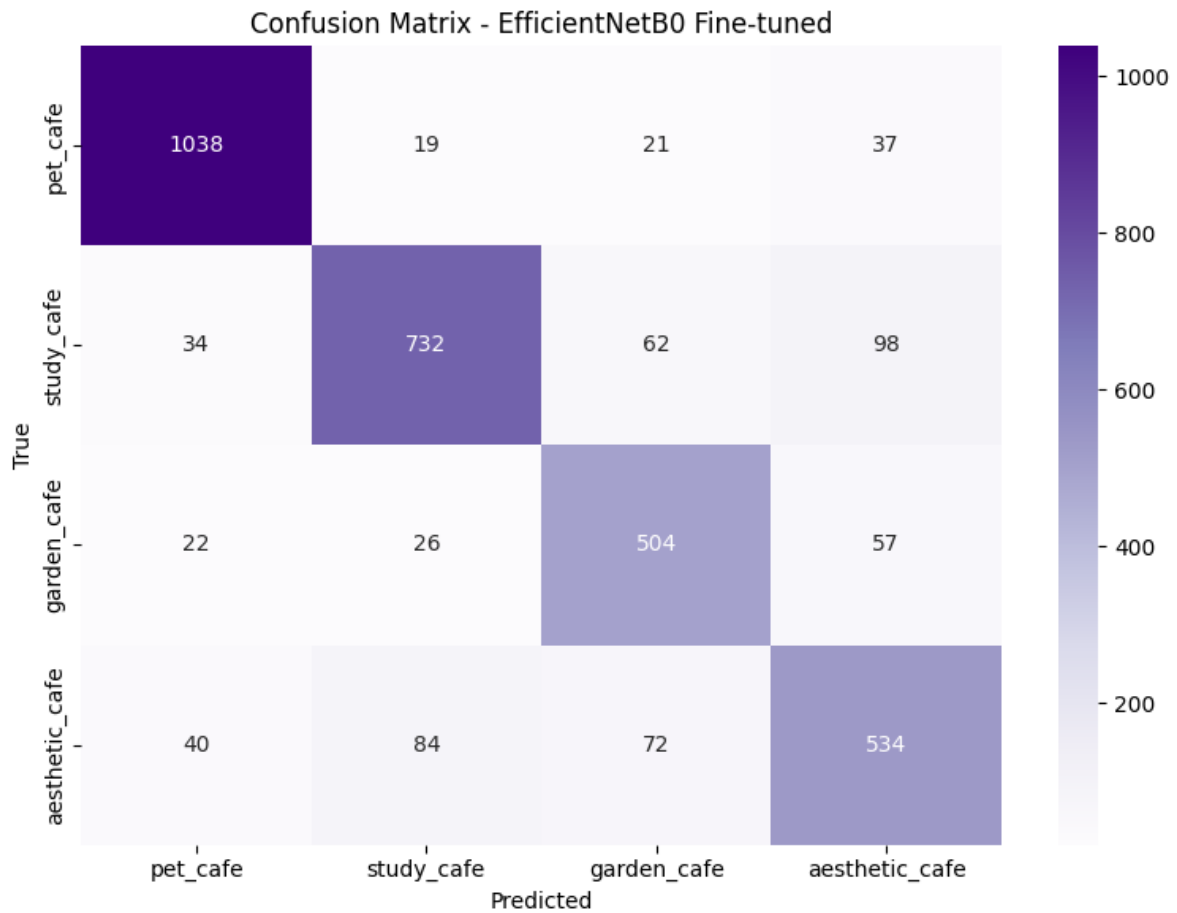
EfficientNetB0 Fine-tuned Macro F1 : 0.8177
EfficientNetB0 Fine-tuned Weighted F1: 0.8306

14.6 Cell 6 — Confusion matrix

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm_ft = confusion_matrix(y_true_ft, y_pred_ft)

plt.figure(figsize=(8, 6))
sns.heatmap(
    cm_ft,
    annot=True,
    fmt="d",
    cmap="Purples",
    xticklabels=CLASS_NAMES,
    yticklabels=CLASS_NAMES
)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix - EfficientNetB0 Fine-tuned")
plt.tight_layout()
plt.show()
```



14.7 Cell 7 — Vẽ biểu đồ fine-tune

```
history_ft_dict = history_effnet_finetune.history

plt.figure(figsize=(12, 5))

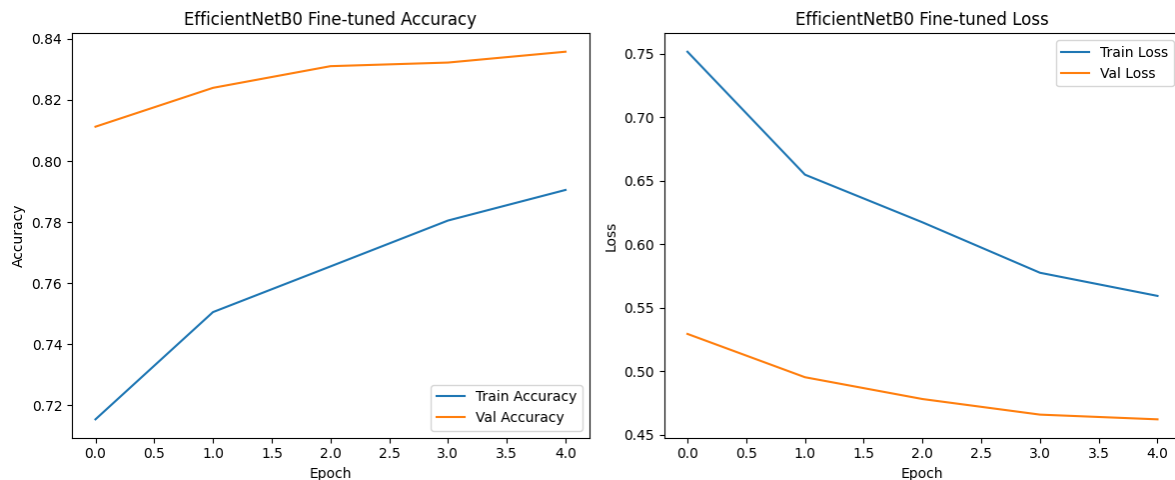
plt.subplot(1, 2, 1)
plt.plot(history_ft_dict["accuracy"], label="Train Accuracy")
plt.plot(history_ft_dict["val_accuracy"], label="Val Accuracy")
plt.title("EfficientNetB0 Fine-tuned Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history_ft_dict["loss"], label="Train Loss")
```



```
plt.plot(history_ft_dict["val_loss"], label="Val Loss")
plt.title("EfficientNetB0 Fine-tuned Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.tight_layout()
plt.show()
```



14.8 Cell 8 — Bảng so sánh cuối cùng

```
import pandas as pd

final_comparison_df = pd.DataFrame([
    {
        "Model": "MobileNetV2 Stage 1",
        "Val Accuracy": 0.8104,
        "Test Accuracy": 0.7962,
        "Macro F1": 0.7805
    },
    {
        "Model": "EfficientNetB0 Stage 1",
        "Val Accuracy": 0.8204,
        "Test Accuracy": 0.8163,
        "Macro F1": 0.8005
    },
    {
        "Model": "ResNet50 Stage 1",
```

```

        "Val Accuracy": 0.8160,
        "Test Accuracy": 0.8050,
        "Macro F1": 0.7851
    },
    {
        "Model": "EfficientNetB0 Fine-tuned",
        "Val Accuracy": val_acc_ft,
        "Test Accuracy": test_acc_ft,
        "Macro F1": macro_f1_ft
    }
])

final_comparison_df = final_comparison_df.sort_values(
    by=["Test Accuracy", "Macro F1"],
    ascending=False
)

final_comparison_df

```

	Model	Val Accuracy	Test Accuracy	Macro F1
3	EfficientNetB0 Fine-tuned	0.835799	0.830769	0.817699
1	EfficientNetB0 Stage 1	0.820400	0.816300	0.800500
2	ResNet50 Stage 1	0.816000	0.805000	0.785100
0	MobileNetV2 Stage 1	0.810400	0.796200	0.780500

FULL CODE COLAB MỚI — EfficientNetB0 5 lớp chuẩn

CELL 1 — Mount Drive

```

from google.colab import drive
drive.mount('/content/drive')

```

Mounted at /content/drive

CELL 2 - Import thư viện

```

import os
import gc
import json
import copy
import random
import shutil
import time
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, WeightedRandomSampler

import torchvision
from torchvision import transforms, models

from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, f1_score
from sklearn.utils.class_weight import compute_class_weight

warnings.filterwarnings("ignore")


def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

set_seed(42)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

use_amp = torch.cuda.is_available()
print("AMP enabled:", use_amp)

```

Using device: cuda
AMP enabled: True

CELL 4 — Clear session

```
gc.collect()
if torch.cuda.is_available():
    torch.cuda.empty_cache()
print("Cache cleared.")
```

Cache cleared.

CELL 5 — Khai báo dataset

```
PROJECT_ROOT = "."
DATASET_DIR = os.path.join(PROJECT_ROOT, "dataset_cafe")

CLASS_NAMES = [
    "pet_cafe",
    "study_cafe",
    "garden_cafe",
    "aesthetic_cafe",
    "food_cafe"
]

IMAGE_EXTS = (".jpg", ".jpeg", ".png", ".bmp", ".webp")

print("PROJECT_ROOT:", os.path.abspath(PROJECT_ROOT))
print("DATASET_DIR :", os.path.abspath(DATASET_DIR))
```

PROJECT_ROOT: d:\Máy học 5 classes
DATASET_DIR : d:\Máy học 5 classes\dataset_cafe

CELL 6 — Kiểm tra số lượng ảnh

```
print("Dataset exists:", os.path.exists(DATASET_DIR))

for class_name in CLASS_NAMES:
    class_dir = os.path.join(DATASET_DIR, class_name)
    if not os.path.exists(class_dir):
```

```

        print(f"[WARNING] Missing folder: {class_dir}")
        continue

    files = [f for f in os.listdir(class_dir) if f.lower().endswith(IMAGE_EXTS)]
    print(f"{class_name}: {len(files)}")

```

```

Dataset exists: True
pet_cafe: 7435
study_cafe: 6155
garden_cafe: 4064
aesthetic_cafe: 4868
food_cafe: 3913

```

CELL 7 — Tạo DataFrame

```

data = []

for class_name in CLASS_NAMES:
    class_dir = os.path.join(DATASET_DIR, class_name)
    if not os.path.exists(class_dir):
        continue

    files = [f for f in os.listdir(class_dir) if f.lower().endswith(IMAGE_EXTS)]

    for file_name in files:
        data.append({
            "file_path": os.path.join(class_dir, file_name),
            "label_name": class_name
        })

df = pd.DataFrame(data)

label_to_index = {label: idx for idx, label in enumerate(CLASS_NAMES)}
index_to_label = {idx: label for label, idx in label_to_index.items()}

df["label"] = df["label_name"].map(label_to_index)

print("Tổng số mẫu:", len(df))
print(df["label_name"].value_counts())
print(label_to_index)
df.head()

```

Tổng số mẫu: 26435

label_name

pet_cafe 7435

study_cafe 6155

aesthetic_cafe 4868

garden_cafe 4064

food_cafe 3913

Name: count, dtype: int64

{'pet_cafe': 0, 'study_cafe': 1, 'garden_cafe': 2, 'aesthetic_cafe': 3, 'food_cafe': 4}

	file_path	label_name	label
0	.\dataset_cafe\pet_cafe\ABOP9ps-1jrbJGqoaXubF0...	pet_cafe	0
1	.\dataset_cafe\pet_cafe\ABOP9ps0WfMNQrYBVuFMns...	pet_cafe	0
2	.\dataset_cafe\pet_cafe\ABOP9ps2ZLwdSli42n8qx3...	pet_cafe	0
3	.\dataset_cafe\pet_cafe\ABOP9ps9KKisZizc71oa49...	pet_cafe	0
4	.\dataset_cafe\pet_cafe\ABOP9psAA5R52fa2YULwea...	pet_cafe	0

CELL 8 — Chia train/val/test

```
train_df, temp_df = train_test_split(
    df,
    test_size=0.30,
    stratify=df["label"],
    random_state=42
)

val_df, test_df = train_test_split(
    temp_df,
    test_size=0.50,
    stratify=temp_df["label"],
    random_state=42
)

train_df = train_df.reset_index(drop=True)
val_df = val_df.reset_index(drop=True)
test_df = test_df.reset_index(drop=True)

print("Train:", train_df.shape)
print("Val  :", val_df.shape)
print("Test :", test_df.shape)
```

```
Train: (18504, 3)
Val   : (3965, 3)
Test  : (3966, 3)
```

CELL 9 — Tính class_weight

```
classes = np.unique(train_df["label"])

class_weights = compute_class_weight(
    class_weight="balanced",
    classes=classes,
    y=train_df["label"]
)

class_weight_dict = {int(c): float(w) for c, w in zip(classes, class_weights)}
class_weights_tensor = torch.tensor(class_weights, dtype=torch.float32).to(device)

print("Class weight:", class_weight_dict)
print("Class weights tensor:", class_weights_tensor)
```

```
Class weight: {0: 0.7111452728670253, 1: 0.8590529247910863, 2: 1.3008084358523726, 3: 1.085...
Class weights tensor: tensor([0.7111, 0.8591, 1.3008, 1.0859, 1.3512], device='cuda:0')
```

CELL 10 — Tham số mới

```
IMG_SIZE = 224
BATCH_SIZE = 32
NUM_CLASSES = len(CLASS_NAMES)
NUM_WORKERS = 0 # local Windows/Jupyter nên để 0 cho ổn định; Linux có thể tăng 2-4

print("IMG_SIZE =", IMG_SIZE)
print("BATCH_SIZE =", BATCH_SIZE)
print("NUM_CLASSES =", NUM_CLASSES)
print("NUM_WORKERS =", NUM_WORKERS)
```

```
IMG_SIZE = 224
BATCH_SIZE = 32
NUM_CLASSES = 5
NUM_WORKERS = 0
```

CELL 12 — Augmentation

```
train_transforms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=8),
    transforms.RandomAffine(degrees=0, translate=(0.08, 0.08)),
    transforms.ColorJitter(contrast=0.10),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

val_test_transforms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

print("Transforms ready.")
```

Transforms ready.

CELL 12 — Tạo DataLoader

```
train_dataset = CafeDataset(train_df, transform=train_transforms)
val_dataset = CafeDataset(val_df, transform=val_test_transforms)
test_dataset = CafeDataset(test_df, transform=val_test_transforms)

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=NUM_WORKERS,
    pin_memory=torch.cuda.is_available()
)
```



```

val_loader = DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS,
    pin_memory=torch.cuda.is_available()
)

test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS,
    pin_memory=torch.cuda.is_available()
)

print("Train batches:", len(train_loader))
print("Val batches  :", len(val_loader))
print("Test batches :", len(test_loader))

```

```

Train batches: 579
Val batches  : 124
Test batches : 124

```

CELL 13 — Kiểm tra batch

```

for images, labels in train_loader:
    print("Images shape:", images.shape)
    print("Labels shape:", labels.shape)
    print("Labels sample:", labels[:10].numpy())
    break

```

```

Images shape: torch.Size([32, 3, 224, 224])
Labels shape: torch.Size([32])
Labels sample: [1 4 2 1 2 0 1 1 0 4]

```

STAGE 1 — Train head đúng

CELL 15 — Build model stage 1

```

weights = models.EfficientNet_BO_Weights.IMAGENET1K_V1
model_stage1 = models.efficientnet_b0(weights=weights)

in_features = model_stage1.classifier[1].in_features
model_stage1.classifier = nn.Sequential(
    nn.Dropout(p=0.3),
    nn.Linear(in_features, NUM_CLASSES)
)

for param in model_stage1.features.parameters():
    param.requires_grad = False

model_stage1 = model_stage1.to(device)
print(model_stage1)

```

Downloading: "https://download.pytorch.org/models/efficientnet_b0_rwightman-7f5810bc.pth" to C:\Users\ASUS\.cache\torch\hub\checkpoints\efficientnet_b0_rwightman-7f5810bc.pth

100%| | 20.5M/20.5M [00:03<00:00, 5.97MB/s]

```

EfficientNet(
  (features): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (1): Sequential(
      (0): MBConv(
        (block): Sequential(
          (0): Conv2dNormActivation(
            (0): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), groups=32)
            (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
            (2): SiLU(inplace=True)
          )
          (1): SqueezeExcitation(
            (avgpool): AdaptiveAvgPool2d(output_size=1)
            (fc1): Conv2d(32, 8, kernel_size=(1, 1), stride=(1, 1))
            (fc2): Conv2d(8, 32, kernel_size=(1, 1), stride=(1, 1))
            (activation): SiLU(inplace=True)
          )
        )
      )
    )
  )

```

```

        (scale_activation): Sigmoid()
    )
    (2): Conv2dNormActivation(
      (0): Conv2d(32, 16, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (stochastic_depth): StochasticDepth(p=0.0, mode=row)
)
(2): Sequential(
  (0): MBConv(
    (block): Sequential(
      (0): Conv2dNormActivation(
        (0): Conv2d(16, 96, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): SiLU(inplace=True)
      )
      (1): Conv2dNormActivation(
        (0): Conv2d(96, 96, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), groups=96)
        (1): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): SiLU(inplace=True)
      )
      (2): SqueezeExcitation(
        (avgpool): AdaptiveAvgPool2d(output_size=1)
        (fc1): Conv2d(96, 4, kernel_size=(1, 1), stride=(1, 1))
        (fc2): Conv2d(4, 96, kernel_size=(1, 1), stride=(1, 1))
        (activation): SiLU(inplace=True)
        (scale_activation): Sigmoid()
      )
      (3): Conv2dNormActivation(
        (0): Conv2d(96, 24, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(24, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (stochastic_depth): StochasticDepth(p=0.0125, mode=row)
  )
  (1): MBConv(
    (block): Sequential(
      (0): Conv2dNormActivation(
        (0): Conv2d(24, 144, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(144, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): SiLU(inplace=True)

```

```

    )
    (1): Conv2dNormActivation(
      (0): Conv2d(144, 144, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), groups=1)
      (1): BatchNorm2d(144, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (2): SqueezeExcitation(
      (avgpool): AdaptiveAvgPool2d(output_size=1)
      (fc1): Conv2d(144, 6, kernel_size=(1, 1), stride=(1, 1))
      (fc2): Conv2d(6, 144, kernel_size=(1, 1), stride=(1, 1))
      (activation): SiLU(inplace=True)
      (scale_activation): Sigmoid()
    )
    (3): Conv2dNormActivation(
      (0): Conv2d(144, 24, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(24, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (stochastic_depth): StochasticDepth(p=0.025, mode=row)
)
)
(3): Sequential(
  (0): MBConv(
    (block): Sequential(
      (0): Conv2dNormActivation(
        (0): Conv2d(24, 144, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(144, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): SiLU(inplace=True)
      )
      (1): Conv2dNormActivation(
        (0): Conv2d(144, 144, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2), groups=1)
        (1): BatchNorm2d(144, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): SiLU(inplace=True)
      )
      (2): SqueezeExcitation(
        (avgpool): AdaptiveAvgPool2d(output_size=1)
        (fc1): Conv2d(144, 6, kernel_size=(1, 1), stride=(1, 1))
        (fc2): Conv2d(6, 144, kernel_size=(1, 1), stride=(1, 1))
        (activation): SiLU(inplace=True)
        (scale_activation): Sigmoid()
      )
    )
    (3): Conv2dNormActivation(
      (0): Conv2d(144, 40, kernel_size=(1, 1), stride=(1, 1), bias=False)

```

```

        (1): BatchNorm2d(40, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
    (stochastic_depth): StochasticDepth(p=0.037500000000000006, mode=row)
)
(1): MBConv(
  (block): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(40, 240, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(240, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(240, 240, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2), groups=1)
      (1): BatchNorm2d(240, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (2): SqueezeExcitation(
      (avgpool): AdaptiveAvgPool2d(output_size=1)
      (fc1): Conv2d(240, 10, kernel_size=(1, 1), stride=(1, 1))
      (fc2): Conv2d(10, 240, kernel_size=(1, 1), stride=(1, 1))
      (activation): SiLU(inplace=True)
      (scale_activation): Sigmoid()
    )
    (3): Conv2dNormActivation(
      (0): Conv2d(240, 40, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(40, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (stochastic_depth): StochasticDepth(p=0.05, mode=row)
)
(4): Sequential(
  (0): MBConv(
    (block): Sequential(
      (0): Conv2dNormActivation(
        (0): Conv2d(40, 240, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(240, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): SiLU(inplace=True)
      )
      (1): Conv2dNormActivation(
        (0): Conv2d(240, 240, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), groups=1)
        (1): BatchNorm2d(240, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

```

```

        (2): SiLU(inplace=True)
    )
    (2): SqueezeExcitation(
      (avgpool): AdaptiveAvgPool2d(output_size=1)
      (fc1): Conv2d(240, 10, kernel_size=(1, 1), stride=(1, 1))
      (fc2): Conv2d(10, 240, kernel_size=(1, 1), stride=(1, 1))
      (activation): SiLU(inplace=True)
      (scale_activation): Sigmoid()
    )
    (3): Conv2dNormActivation(
      (0): Conv2d(240, 80, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(80, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (stochastic_depth): StochasticDepth(p=0.0625, mode=row)
)
(1): MBConv(
  (block): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(80, 480, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(480, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(480, 480, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), groups=480)
      (1): BatchNorm2d(480, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (2): SqueezeExcitation(
      (avgpool): AdaptiveAvgPool2d(output_size=1)
      (fc1): Conv2d(480, 20, kernel_size=(1, 1), stride=(1, 1))
      (fc2): Conv2d(20, 480, kernel_size=(1, 1), stride=(1, 1))
      (activation): SiLU(inplace=True)
      (scale_activation): Sigmoid()
    )
    (3): Conv2dNormActivation(
      (0): Conv2d(480, 80, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(80, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (stochastic_depth): StochasticDepth(p=0.07500000000000001, mode=row)
)
(2): MBConv(

```

```

(block): Sequential(
  (0): Conv2dNormActivation(
    (0): Conv2d(80, 480, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (1): BatchNorm2d(480, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): SiLU(inplace=True)
  )
  (1): Conv2dNormActivation(
    (0): Conv2d(480, 480, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), groups=1)
    (1): BatchNorm2d(480, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): SiLU(inplace=True)
  )
  (2): SqueezeExcitation(
    (avgpool): AdaptiveAvgPool2d(output_size=1)
    (fc1): Conv2d(480, 20, kernel_size=(1, 1), stride=(1, 1))
    (fc2): Conv2d(20, 480, kernel_size=(1, 1), stride=(1, 1))
    (activation): SiLU(inplace=True)
    (scale_activation): Sigmoid()
  )
  (3): Conv2dNormActivation(
    (0): Conv2d(480, 80, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (1): BatchNorm2d(80, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  )
)
(stochastic_depth): StochasticDepth(p=0.08750000000000001, mode=row)
)
(5): Sequential(
  (0): MBConv(
    (block): Sequential(
      (0): Conv2dNormActivation(
        (0): Conv2d(80, 480, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(480, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): SiLU(inplace=True)
      )
      (1): Conv2dNormActivation(
        (0): Conv2d(480, 480, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2), groups=1)
        (1): BatchNorm2d(480, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): SiLU(inplace=True)
      )
      (2): SqueezeExcitation(
        (avgpool): AdaptiveAvgPool2d(output_size=1)
        (fc1): Conv2d(480, 20, kernel_size=(1, 1), stride=(1, 1))
        (fc2): Conv2d(20, 480, kernel_size=(1, 1), stride=(1, 1))
      )
    )
  )
)

```

```

        (activation): SiLU(inplace=True)
        (scale_activation): Sigmoid()
    )
    (3): Conv2dNormActivation(
      (0): Conv2d(480, 112, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(112, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (stochastic_depth): StochasticDepth(p=0.1, mode=row)
)
(1): MBConv(
  (block): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(112, 672, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(672, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(672, 672, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2), groups=672)
      (1): BatchNorm2d(672, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (2): SqueezeExcitation(
      (avgpool): AdaptiveAvgPool2d(output_size=1)
      (fc1): Conv2d(672, 28, kernel_size=(1, 1), stride=(1, 1))
      (fc2): Conv2d(28, 672, kernel_size=(1, 1), stride=(1, 1))
      (activation): SiLU(inplace=True)
      (scale_activation): Sigmoid()
    )
    (3): Conv2dNormActivation(
      (0): Conv2d(672, 112, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(112, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (stochastic_depth): StochasticDepth(p=0.1125, mode=row)
)
(2): MBConv(
  (block): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(112, 672, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(672, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )

```



```

(1): Conv2dNormActivation(
  (0): Conv2d(672, 672, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2), groups=
  (1): BatchNorm2d(672, eps=1e-05, momentum=0.1, affine=True, track_running_stats=
  (2): SiLU(inplace=True)
)
(2): SqueezeExcitation(
  (avgpool): AdaptiveAvgPool2d(output_size=1)
  (fc1): Conv2d(672, 28, kernel_size=(1, 1), stride=(1, 1))
  (fc2): Conv2d(28, 672, kernel_size=(1, 1), stride=(1, 1))
  (activation): SiLU(inplace=True)
  (scale_activation): Sigmoid()
)
(3): Conv2dNormActivation(
  (0): Conv2d(672, 112, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (1): BatchNorm2d(112, eps=1e-05, momentum=0.1, affine=True, track_running_stats=
)
)
(stochastic_depth): StochasticDepth(p=0.125, mode=row)
)
(6): Sequential(
  (0): MBConv(
    (block): Sequential(
      (0): Conv2dNormActivation(
        (0): Conv2d(112, 672, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(672, eps=1e-05, momentum=0.1, affine=True, track_running_stats=
        (2): SiLU(inplace=True)
      )
      (1): Conv2dNormActivation(
        (0): Conv2d(672, 672, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2), groups=
        (1): BatchNorm2d(672, eps=1e-05, momentum=0.1, affine=True, track_running_stats=
        (2): SiLU(inplace=True)
      )
      (2): SqueezeExcitation(
        (avgpool): AdaptiveAvgPool2d(output_size=1)
        (fc1): Conv2d(672, 28, kernel_size=(1, 1), stride=(1, 1))
        (fc2): Conv2d(28, 672, kernel_size=(1, 1), stride=(1, 1))
        (activation): SiLU(inplace=True)
        (scale_activation): Sigmoid()
      )
      (3): Conv2dNormActivation(
        (0): Conv2d(672, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True, track_running_stats=

```

```

    )
    )
    (stochastic_depth): StochasticDepth(p=0.1375, mode=row)
)
(1): MBConv(
  (block): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(192, 1152, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(1152, 1152, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2), groups=1)
      (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (2): SqueezeExcitation(
      (avgpool): AdaptiveAvgPool2d(output_size=1)
      (fc1): Conv2d(1152, 48, kernel_size=(1, 1), stride=(1, 1))
      (fc2): Conv2d(48, 1152, kernel_size=(1, 1), stride=(1, 1))
      (activation): SiLU(inplace=True)
      (scale_activation): Sigmoid()
    )
    (3): Conv2dNormActivation(
      (0): Conv2d(1152, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (stochastic_depth): StochasticDepth(p=0.15000000000000002, mode=row)
)
(2): MBConv(
  (block): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(192, 1152, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(1152, 1152, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2), groups=1)
      (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (2): SqueezeExcitation(

```

```

        (avgpool): AdaptiveAvgPool2d(output_size=1)
        (fc1): Conv2d(1152, 48, kernel_size=(1, 1), stride=(1, 1))
        (fc2): Conv2d(48, 1152, kernel_size=(1, 1), stride=(1, 1))
        (activation): SiLU(inplace=True)
        (scale_activation): Sigmoid()
    )
    (3): Conv2dNormActivation(
      (0): Conv2d(1152, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (stochastic_depth): StochasticDepth(p=0.1625, mode=row)
)
(3): MBConv(
  (block): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(192, 1152, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(1152, 1152, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2), groups=1)
      (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (2): SqueezeExcitation(
      (avgpool): AdaptiveAvgPool2d(output_size=1)
      (fc1): Conv2d(1152, 48, kernel_size=(1, 1), stride=(1, 1))
      (fc2): Conv2d(48, 1152, kernel_size=(1, 1), stride=(1, 1))
      (activation): SiLU(inplace=True)
      (scale_activation): Sigmoid()
    )
    (3): Conv2dNormActivation(
      (0): Conv2d(1152, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (stochastic_depth): StochasticDepth(p=0.17500000000000002, mode=row)
)
(7): Sequential(
  (0): MBConv(
    (block): Sequential(

```

```

(0): Conv2dNormActivation(
  (0): Conv2d(192, 1152, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (2): SiLU(inplace=True)
)
(1): Conv2dNormActivation(
  (0): Conv2d(1152, 1152, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), groups=1)
  (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (2): SiLU(inplace=True)
)
(2): SqueezeExcitation(
  (avgpool): AdaptiveAvgPool2d(output_size=1)
  (fc1): Conv2d(1152, 48, kernel_size=(1, 1), stride=(1, 1))
  (fc2): Conv2d(48, 1152, kernel_size=(1, 1), stride=(1, 1))
  (activation): SiLU(inplace=True)
  (scale_activation): Sigmoid()
)
(3): Conv2dNormActivation(
  (0): Conv2d(1152, 320, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (1): BatchNorm2d(320, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
)
)
(stochastic_depth): StochasticDepth(p=0.1875, mode=row)
)
)
(8): Conv2dNormActivation(
  (0): Conv2d(320, 1280, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (1): BatchNorm2d(1280, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (2): SiLU(inplace=True)
)
)
(avgpool): AdaptiveAvgPool2d(output_size=1)
(classifier): Sequential(
  (0): Dropout(p=0.3, inplace=False)
  (1): Linear(in_features=1280, out_features=5, bias=True)
)
)

```

CELL 15 — Loss, optimizer, scheduler stage 1

```

criterion = nn.CrossEntropyLoss(weight=class_weights_tensor)

optimizer_stage1 = optim.Adam(
    filter(lambda p: p.requires_grad, model_stage1.parameters()),
    lr=1e-3
)

scheduler_stage1 = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer_stage1,
    mode="min",
    factor=0.2,
    patience=2
)

```

CELL 16 — Hàm train / eval

```

from torch.cuda.amp import autocast, GradScaler

scaler = GradScaler(enabled=use_amp)

def train_one_epoch(model, loader, criterion, optimizer, device, scaler=None):
    model.train()

    running_loss = 0.0
    running_corrects = 0
    total = 0

    for inputs, labels in loader:
        inputs = inputs.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)

        optimizer.zero_grad()

        with autocast(enabled=use_amp):
            outputs = model(inputs)
            loss = criterion(outputs, labels)

        if scaler is not None and use_amp:
            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()
        else:

```

```

        loss.backward()
        optimizer.step()

    preds = torch.argmax(outputs, dim=1)

    running_loss += loss.item() * inputs.size(0)
    running_corrects += torch.sum(preds == labels).item()
    total += labels.size(0)

epoch_loss = running_loss / total
epoch_acc = running_corrects / total
return epoch_loss, epoch_acc

def evaluate_one_epoch(model, loader, criterion, device):
    model.eval()

    running_loss = 0.0
    running_corrects = 0
    total = 0

    all_labels = []
    all_preds = []

    with torch.no_grad():
        for inputs, labels in loader:
            inputs = inputs.to(device, non_blocking=True)
            labels = labels.to(device, non_blocking=True)

            with autocast(enabled=use_amp):
                outputs = model(inputs)
                loss = criterion(outputs, labels)

            preds = torch.argmax(outputs, dim=1)

            running_loss += loss.item() * inputs.size(0)
            running_corrects += torch.sum(preds == labels).item()
            total += labels.size(0)

            all_labels.extend(labels.cpu().numpy())
            all_preds.extend(preds.cpu().numpy())

```

```

epoch_loss = running_loss / total
epoch_acc = running_corrects / total
return epoch_loss, epoch_acc, np.array(all_labels), np.array(all_preds)

```

CELL 17 — Hàm train model có early stopping

```

def train_model(
    model,
    train_loader,
    val_loader,
    criterion,
    optimizer,
    scheduler,
    device,
    epochs,
    save_path,
    early_stop_patience=3
):
    history = {
        "train_loss": [],
        "train_accuracy": [],
        "val_loss": [],
        "val_accuracy": []
    }

    best_val_acc = 0.0
    best_model_wts = copy.deepcopy(model.state_dict())
    patience_counter = 0

    for epoch in range(epochs):
        print(f"\nEpoch {epoch + 1}/{epochs}")

        train_loss, train_acc = train_one_epoch(
            model, train_loader, criterion, optimizer, device, scaler
        )

        val_loss, val_acc, _, _ = evaluate_one_epoch(
            model, val_loader, criterion, device
        )

        scheduler.step(val_loss)

```

```

history["train_loss"].append(train_loss)
history["train_accuracy"].append(train_acc)
history["val_loss"].append(val_loss)
history["val_accuracy"].append(val_acc)

current_lr = optimizer.param_groups[0]["lr"]
print(f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f}")
print(f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f}")
print(f"Learning Rate: {current_lr:.8f}")

if val_acc > best_val_acc:
    best_val_acc = val_acc
    best_model_wts = copy.deepcopy(model.state_dict())
    torch.save(model.state_dict(), save_path)
    print(f"Saved best model to: {save_path}")
    patience_counter = 0
else:
    patience_counter += 1
    print(f"No improvement. Patience: {patience_counter}/{early_stop_patience}")

if patience_counter >= early_stop_patience:
    print("Early stopping triggered.")
    break

model.load_state_dict(best_model_wts)
return model, history

```

CELL 18 — Tạo thư mục outputs local

```

OUTPUT_DIR = os.path.join(PROJECT_ROOT, "outputs")
os.makedirs(OUTPUT_DIR, exist_ok=True)

STAGE1_MODEL_PATH = os.path.join(OUTPUT_DIR, "best_effnetb0_stage1_5class.pth")
FINETUNE_MODEL_PATH = os.path.join(OUTPUT_DIR, "best_effnetb0_finetuned_5class.pth")

print("OUTPUT_DIR:", os.path.abspath(OUTPUT_DIR))

```

OUTPUT_DIR: d:\Máy học 5 classes\outputs

CELL 19 — Train stage 1


```
model_stage1, history_stage1 = train_model(  
    model=model_stage1,  
    train_loader=train_loader,  
    val_loader=val_loader,  
    criterion=criterion,  
    optimizer=optimizer_stage1,  
    scheduler=scheduler_stage1,  
    device=device,  
    epochs=12,  
    save_path=STAGE1_MODEL_PATH,  
    early_stop_patience=4  
)
```

Epoch 1/12

Train Loss: 0.7935 | Train Acc: 0.7268

Val Loss: 0.6213 | Val Acc: 0.7997

Learning Rate: 0.00100000

Saved best model to: .\outputs\best_effnetb0_stage1_5class.pth

Epoch 2/12

Train Loss: 0.6988 | Train Acc: 0.7564

Val Loss: 0.5993 | Val Acc: 0.8043

Learning Rate: 0.00100000

Saved best model to: .\outputs\best_effnetb0_stage1_5class.pth

Epoch 3/12

Train Loss: 0.6633 | Train Acc: 0.7694

Val Loss: 0.5708 | Val Acc: 0.8139

Learning Rate: 0.00100000

Saved best model to: .\outputs\best_effnetb0_stage1_5class.pth

Epoch 4/12

Train Loss: 0.6611 | Train Acc: 0.7652

Val Loss: 0.5682 | Val Acc: 0.8141

Learning Rate: 0.00100000

Saved best model to: .\outputs\best_effnetb0_stage1_5class.pth

Epoch 5/12

Train Loss: 0.6591 | Train Acc: 0.7675

Val Loss: 0.5680 | Val Acc: 0.8144

Learning Rate: 0.00100000

Saved best model to: .\outputs\best_effnetb0_stage1_5class.pth

Epoch 6/12

Train Loss: 0.6617 | Train Acc: 0.7667

Val Loss: 0.5879 | Val Acc: 0.8055

Learning Rate: 0.00100000

No improvement. Patience: 1/4

Epoch 7/12

Train Loss: 0.6588 | Train Acc: 0.7686

Val Loss: 0.5741 | Val Acc: 0.8088

Learning Rate: 0.00100000

No improvement. Patience: 2/4

Epoch 8/12

Train Loss: 0.6505 | Train Acc: 0.7703

Val Loss: 0.5735 | Val Acc: 0.8113

Learning Rate: 0.00020000

No improvement. Patience: 3/4

Epoch 9/12

Train Loss: 0.6430 | Train Acc: 0.7696

Val Loss: 0.5591 | Val Acc: 0.8154

Learning Rate: 0.00020000

Saved best model to: .\outputs\best_effnetb0_stage1_5class.pth

Epoch 10/12

Train Loss: 0.6426 | Train Acc: 0.7756

Val Loss: 0.5521 | Val Acc: 0.8184

Learning Rate: 0.00020000

Saved best model to: .\outputs\best_effnetb0_stage1_5class.pth

Epoch 11/12

Train Loss: 0.6329 | Train Acc: 0.7777

Val Loss: 0.5539 | Val Acc: 0.8166

Learning Rate: 0.00020000

No improvement. Patience: 1/4

Epoch 12/12

Train Loss: 0.6254 | Train Acc: 0.7788

Val Loss: 0.5484 | Val Acc: 0.8166

Learning Rate: 0.00020000

No improvement. Patience: 2/4

STAGE 1 EVALUATION

CELL 20 — Load best stage 1

```
best_stage1 = models.efficientnet_b0(weights=None)

in_features = best_stage1.classifier[1].in_features
best_stage1.classifier = nn.Sequential(
    nn.Dropout(p=0.3),
    nn.Linear(in_features, NUM_CLASSES)
)

best_stage1.load_state_dict(torch.load(STAGE1_MODEL_PATH, map_location=device))
best_stage1 = best_stage1.to(device)
best_stage1.eval()

print("Loaded best stage 1 model.")
```

Loaded best stage 1 model.

CELL 21 — Evaluate stage 1

```
val_loss_s1, val_acc_s1, _, _ = evaluate_one_epoch(best_stage1, val_loader, criterion, device)
test_loss_s1, test_acc_s1, _, _ = evaluate_one_epoch(best_stage1, test_loader, criterion, device)

print(f"Stage 1 Validation Accuracy: {val_acc_s1:.4f}")
print(f"Stage 1 Validation Loss      : {val_loss_s1:.4f}")
print(f"Stage 1 Test Accuracy         : {test_acc_s1:.4f}")
print(f"Stage 1 Test Loss              : {test_loss_s1:.4f}")
```

```
Stage 1 Validation Accuracy: 0.8184
Stage 1 Validation Loss      : 0.5521
Stage 1 Test Accuracy        : 0.8119
Stage 1 Test Loss            : 0.5424
```

STAGE 2 — Fine-tune

CELL 22 — Clear cache

```
gc.collect()
if torch.cuda.is_available():
    torch.cuda.empty_cache()
print("Cleared before fine-tuning.")
```

Cleared before fine-tuning.

CELL 23 — Build model fine-tune và load weight stage 1

```
model_finetune = models.efficientnet_b0(weights=weights)

in_features = model_finetune.classifier[1].in_features
model_finetune.classifier = nn.Sequential(
    nn.Dropout(p=0.3),
    nn.Linear(in_features, NUM_CLASSES)
)

model_finetune.load_state_dict(torch.load(STAGE1_MODEL_PATH, map_location=device))
model_finetune = model_finetune.to(device)

print("Loaded stage 1 weights for fine-tuning.")
```

Loaded stage 1 weights for fine-tuning.

CELL 24 — Mở một phần backbone

```
for param in model_finetune.features.parameters():
    param.requires_grad = False

for param in model_finetune.features[-2:].parameters():
    param.requires_grad = True

for param in model_finetune.classifier.parameters():
    param.requires_grad = True

trainable_count = sum(p.numel() for p in model_finetune.parameters() if p.requires_grad)
total_count = sum(p.numel() for p in model_finetune.parameters())

print("Trainable params:", trainable_count)
print("Total params      :", total_count)
```

Trainable params: 1135797

Total params : 4013953

CELL 25 — Compile fine-tune

```

criterion_ft = nn.CrossEntropyLoss(weight=class_weights_tensor)

optimizer_finetune = optim.Adam(
    filter(lambda p: p.requires_grad, model_finetune.parameters()),
    lr=1e-6
)

scheduler_finetune = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer_finetune,
    mode="min",
    factor=0.2,
    patience=1
)

```

CELL 26 — Train fine-tune

```

model_finetune, history_finetune = train_model(
    model=model_finetune,
    train_loader=train_loader,
    val_loader=val_loader,
    criterion=criterion_ft,
    optimizer=optimizer_finetune,
    scheduler=scheduler_finetune,
    device=device,
    epochs=80,
    save_path=FINETUNE_MODEL_PATH,
    early_stop_patience=4
)

```

Epoch 1/80

Train Loss: 0.6356 | Train Acc: 0.7748

Val Loss: 0.5538 | Val Acc: 0.8164

Learning Rate: 0.00000100

Saved best model to: .\outputs\best_effnetb0_finetuned_5class.pth

Epoch 2/80

Train Loss: 0.6238 | Train Acc: 0.7753

Val Loss: 0.5423 | Val Acc: 0.8187

Learning Rate: 0.00000100

Saved best model to: .\outputs\best_effnetb0_finetuned_5class.pth

Epoch 3/80
Train Loss: 0.6259 | Train Acc: 0.7805
Val Loss: 0.5469 | Val Acc: 0.8172
Learning Rate: 0.00000100
No improvement. Patience: 1/4

Epoch 4/80
Train Loss: 0.6269 | Train Acc: 0.7771
Val Loss: 0.5369 | Val Acc: 0.8217
Learning Rate: 0.00000100
Saved best model to: .\outputs\best_effnetb0_finetuned_5class.pth

Epoch 5/80
Train Loss: 0.6186 | Train Acc: 0.7818
Val Loss: 0.5459 | Val Acc: 0.8161
Learning Rate: 0.00000100
No improvement. Patience: 1/4

Epoch 6/80
Train Loss: 0.6205 | Train Acc: 0.7826
Val Loss: 0.5368 | Val Acc: 0.8179
Learning Rate: 0.00000100
No improvement. Patience: 2/4

Epoch 7/80
Train Loss: 0.6116 | Train Acc: 0.7839
Val Loss: 0.5359 | Val Acc: 0.8250
Learning Rate: 0.00000100
Saved best model to: .\outputs\best_effnetb0_finetuned_5class.pth

Epoch 8/80
Train Loss: 0.6135 | Train Acc: 0.7864
Val Loss: 0.5280 | Val Acc: 0.8250
Learning Rate: 0.00000100
No improvement. Patience: 1/4

Epoch 9/80
Train Loss: 0.6113 | Train Acc: 0.7831
Val Loss: 0.5349 | Val Acc: 0.8227
Learning Rate: 0.00000100
No improvement. Patience: 2/4

Epoch 10/80

Train Loss: 0.6078 | Train Acc: 0.7850

Val Loss: 0.5330 | Val Acc: 0.8212

Learning Rate: 0.00000020

No improvement. Patience: 3/4

Epoch 11/80

Train Loss: 0.6012 | Train Acc: 0.7874

Val Loss: 0.5257 | Val Acc: 0.8262

Learning Rate: 0.00000020

Saved best model to: .\outputs\best_effnetb0_finetuned_5class.pth

Epoch 12/80

Train Loss: 0.5990 | Train Acc: 0.7889

Val Loss: 0.5369 | Val Acc: 0.8202

Learning Rate: 0.00000020

No improvement. Patience: 1/4

Epoch 13/80

Train Loss: 0.6062 | Train Acc: 0.7860

Val Loss: 0.5292 | Val Acc: 0.8212

Learning Rate: 0.00000004

No improvement. Patience: 2/4

Epoch 14/80

Train Loss: 0.6036 | Train Acc: 0.7867

Val Loss: 0.5333 | Val Acc: 0.8230

Learning Rate: 0.00000004

No improvement. Patience: 3/4

Epoch 15/80

Train Loss: 0.5945 | Train Acc: 0.7895

Val Loss: 0.5268 | Val Acc: 0.8237

Learning Rate: 0.00000001

No improvement. Patience: 4/4

Early stopping triggered.

FINAL EVALUATION

CELL 27 — Load best fine-tuned model

```

best_model = models.efficientnet_b0(weights=None)

in_features = best_model.classifier[1].in_features
best_model.classifier = nn.Sequential(
    nn.Dropout(p=0.3),
    nn.Linear(in_features, NUM_CLASSES)
)

best_model.load_state_dict(torch.load(FINETUNE_MODEL_PATH, map_location=device))
best_model = best_model.to(device)
best_model.eval()

print("Loaded best fine-tuned model.")

```

Loaded best fine-tuned model.

CELL 28 — Evaluate final

```

val_loss, val_acc, _, _ = evaluate_one_epoch(best_model, val_loader, criterion_ft, device)
test_loss, test_acc, _, _ = evaluate_one_epoch(best_model, test_loader, criterion_ft, device)

print(f"Final Validation Accuracy: {val_acc:.4f}")
print(f"Final Validation Loss      : {val_loss:.4f}")
print(f"Final Test Accuracy         : {test_acc:.4f}")
print(f"Final Test Loss              : {test_loss:.4f}")

```

```

Final Validation Accuracy: 0.8262
Final Validation Loss      : 0.5257
Final Test Accuracy        : 0.8192
Final Test Loss            : 0.5185

```

CELL 29 — Predict test

```

y_true = []
y_pred = []

best_model.eval()
with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)

```



```

        outputs = best_model(images)
        preds = torch.argmax(outputs, dim=1)

        y_true.extend(labels.numpy())
        y_pred.extend(preds.cpu().numpy())

y_true = np.array(y_true)
y_pred = np.array(y_pred)

```

CELL 30 — Classification report

```

print(classification_report(
    y_true,
    y_pred,
    target_names=CLASS_NAMES,
    digits=4
))

```

	precision	recall	f1-score	support
pet_cafe	0.9533	0.8970	0.9243	1116
study_cafe	0.7805	0.8236	0.8015	924
garden_cafe	0.7364	0.8210	0.7764	609
aesthetic_cafe	0.7089	0.6438	0.6748	730
food_cafe	0.8564	0.8739	0.8651	587
accuracy			0.8182	3966
macro avg	0.8071	0.8119	0.8084	3966
weighted avg	0.8204	0.8182	0.8183	3966

CELL 31 — F1 score

```

macro_f1 = f1_score(y_true, y_pred, average="macro")
weighted_f1 = f1_score(y_true, y_pred, average="weighted")

print(f"Macro F1    : {macro_f1:.4f}")
print(f"Weighted F1: {weighted_f1:.4f}")

```

```

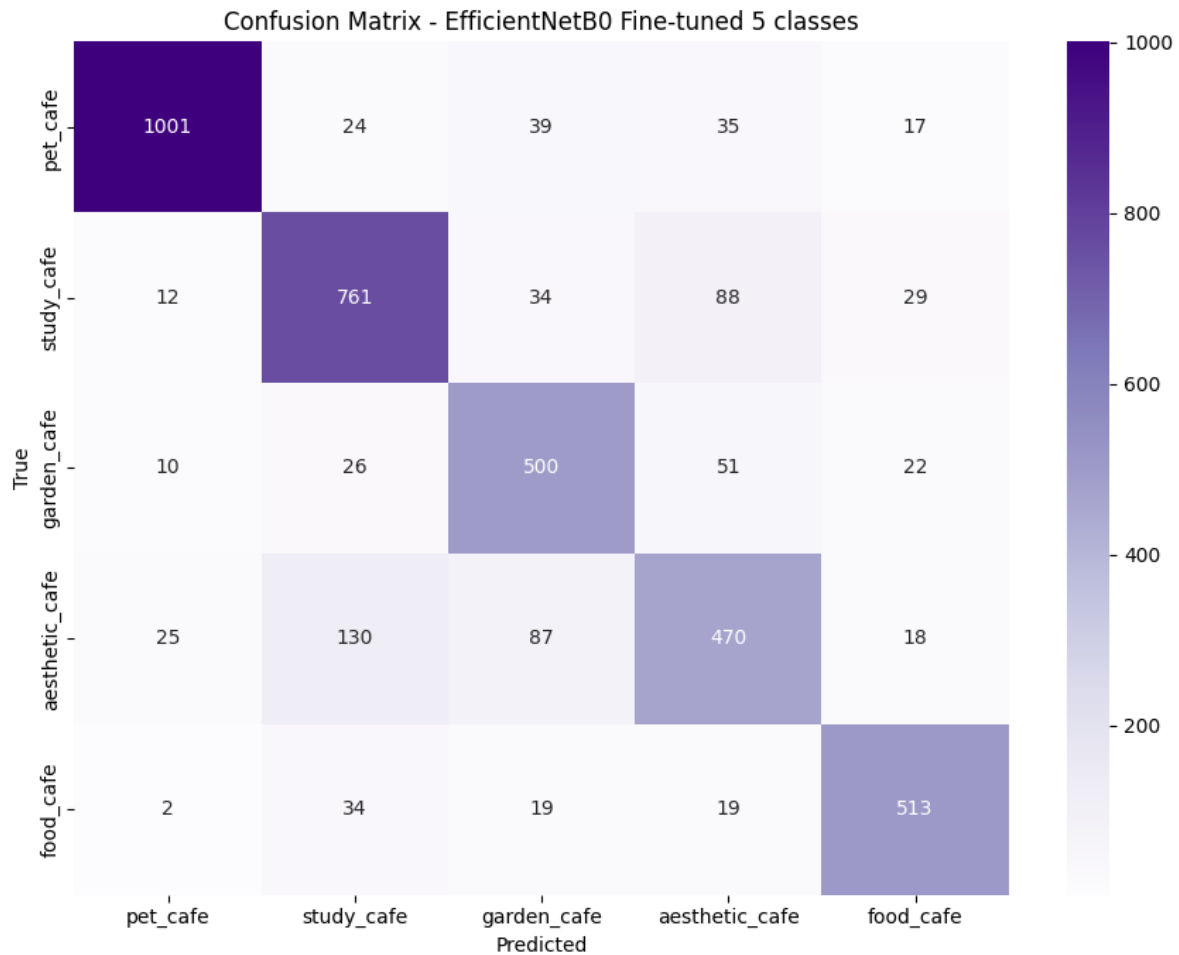
Macro F1    : 0.8084
Weighted F1: 0.8183

```

CELL 32 — Confusion matrix

```
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(9, 7))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Purples",
    xticklabels=CLASS_NAMES,
    yticklabels=CLASS_NAMES
)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix - EfficientNetB0 Fine-tuned 5 classes")
plt.tight_layout()
plt.show()
```



CELL 33 — Biểu đồ train stage 1

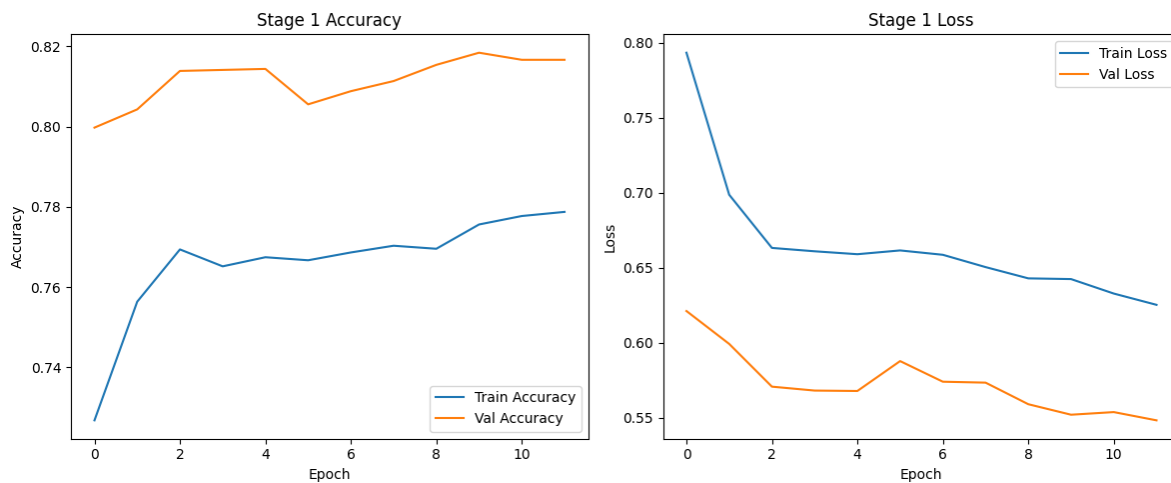
```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history_stage1["train_accuracy"], label="Train Accuracy")
plt.plot(history_stage1["val_accuracy"], label="Val Accuracy")
plt.title("Stage 1 Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history_stage1["train_loss"], label="Train Loss")
plt.plot(history_stage1["val_loss"], label="Val Loss")
```

```
plt.title("Stage 1 Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.tight_layout()
plt.show()
```



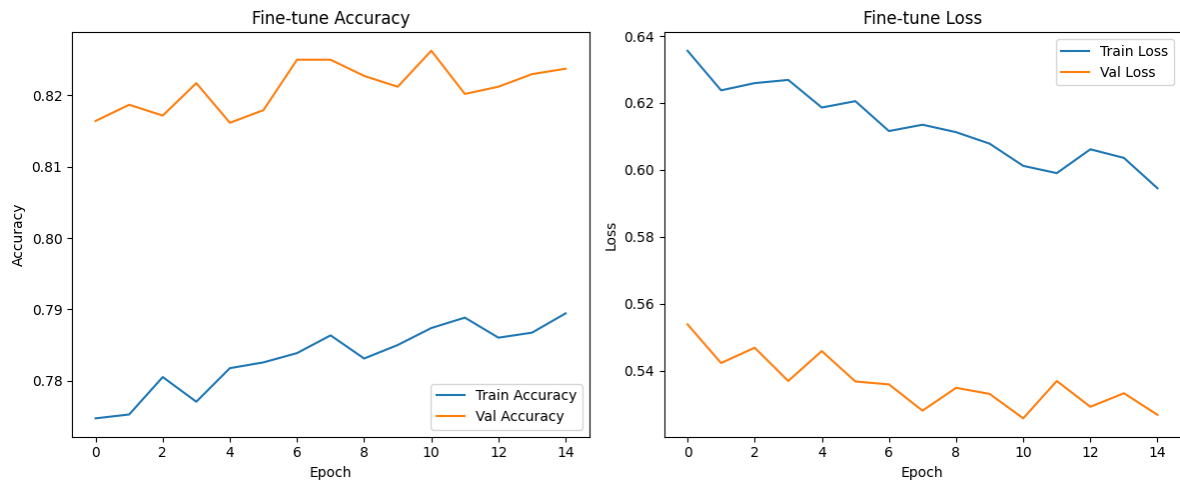
CELL 34 — Biểu đồ fine-tune

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history_finetune["train_accuracy"], label="Train Accuracy")
plt.plot(history_finetune["val_accuracy"], label="Val Accuracy")
plt.title("Fine-tune Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history_finetune["train_loss"], label="Train Loss")
plt.plot(history_finetune["val_loss"], label="Val Loss")
plt.title("Fine-tune Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
```

```
plt.tight_layout()
plt.show()
```



CELL 35 — Save final model + label mapping local

```
FINAL_DIR = os.path.join(OUTPUT_DIR, "cafe_final_model_5class_pytorch")
os.makedirs(FINAL_DIR, exist_ok=True)

torch.save(best_model.state_dict(), os.path.join(FINAL_DIR, "best_effnetb0_finetuned_5class.pth"))

with open(os.path.join(FINAL_DIR, "index_to_label.json"), "w", encoding="utf-8") as f:
    json.dump(index_to_label, f, ensure_ascii=False, indent=2)

with open(os.path.join(FINAL_DIR, "label_to_index.json"), "w", encoding="utf-8") as f:
    json.dump(label_to_index, f, ensure_ascii=False, indent=2)

print("Saved final model to:", os.path.abspath(FINAL_DIR))
```

Saved final model to: d:\Máy học 5 classes\outputs\cafe_final_model_5class_pytorch

=====

train 2 lop

kiem tra du lieu nhi pha

CELL 1 — Import thư viện

```
import os
```

CELL 2 — Khai báo đường dẫn dataset local

```
DATASET_DIR = "./cafe_dataset_2lop"    # dataset local trong thư mục gốc project

print("Dataset exists:", os.path.exists(DATASET_DIR))
print("Absolute path :", os.path.abspath(DATASET_DIR))
```

Dataset exists: True

Absolute path : d:\Máy học 5 classes\cafe_dataset_2lop

CELL 3 — Kiểm tra thư mục lớp

```
EXPECTED_CLASSES = ["cafe", "other"]

found_classes = [
    d for d in os.listdir(DATASET_DIR)
    if os.path.isdir(os.path.join(DATASET_DIR, d))
]

print("Các thư mục tìm thấy:")
for c in found_classes:
    print("-", c)

print("\nKiểm tra đủ 2 lớp chưa:")
for c in EXPECTED_CLASSES:
    print(f"{c}: {'OK' if c in found_classes else 'THIẾU'}")
```

Các thư mục tìm thấy:

- cafe
- other

Kiểm tra đủ 2 lớp chưa:

cafe: OK
other: OK

CELL 4 — Đếm số lượng ảnh mỗi lớp

```

IMAGE_EXTS = (".jpg", ".jpeg", ".png", ".webp", ".bmp")

class_counts = {}

for class_name in EXPECTED_CLASSES:
    class_dir = os.path.join(DATASET_DIR, class_name)

    if not os.path.exists(class_dir):
        class_counts[class_name] = 0
        continue

    files = [
        f for f in os.listdir(class_dir)
        if f.lower().endswith(IMAGE_EXTS)
    ]
    class_counts[class_name] = len(files)

print("Số lượng ảnh mỗi lớp:")
for k, v in class_counts.items():
    print(f"{k}: {v}")

print("Tổng số ảnh:", sum(class_counts.values()))

```

Số lượng ảnh mỗi lớp:
cafe: 3739
other: 3492
Tổng số ảnh: 7231

CELL 5 — Đánh giá sơ bộ cân bằng dữ liệu

```

for class_name, count in class_counts.items():
    if count < 100:
        print(f" Lớp {class_name} rất ít ảnh: {count}")
    elif count < 500:
        print(f" Lớp {class_name} tương đối ít ảnh: {count}")
    else:
        print(f" Lớp {class_name} ổn: {count}")

```

Lớp cafe ổn: 3739
Lớp other ổn: 3492

Cell 1 — Import và tạo DataFrame

```
import os
import pandas as pd

DATASET_DIR = r"d:\Máy học 5 classes\cafe_dataset_2lop" # sửa nếu cần
CLASS_NAMES = ["other", "cafe"] # để nhãn 0=other, 1=cafe
IMAGE_EXTS = (".jpg", ".jpeg", ".png", ".webp", ".bmp")

data = []

for class_name in CLASS_NAMES:
    class_dir = os.path.join(DATASET_DIR, class_name)

    files = [
        f for f in os.listdir(class_dir)
        if f.lower().endswith(IMAGE_EXTS)
    ]

    for file_name in files:
        data.append({
            "file_path": os.path.join(class_dir, file_name),
            "label_name": class_name
        })

df = pd.DataFrame(data)

print("Tổng số mẫu:", len(df))
print(df.head())
```

Tổng số mẫu: 7231

	file_path	label_name
0	d:\Máy học 5 classes\cafe_dataset_2lop\other\-	other
1	d:\Máy học 5 classes\cafe_dataset_2lop\other\-	other
2	d:\Máy học 5 classes\cafe_dataset_2lop\other\0...	other
3	d:\Máy học 5 classes\cafe_dataset_2lop\other\1...	other
4	d:\Máy học 5 classes\cafe_dataset_2lop\other\1...	other

Cell 2 — Mã hóa nhãn


```

label_to_index = {"other": 0, "cafe": 1}
index_to_label = {0: "other", 1: "cafe"}

df["label"] = df["label_name"].map(label_to_index)

print("Label mapping:")
print(label_to_index)

print("\nPhân bố nhãn:")
print(df["label_name"].value_counts())

```

Label mapping:
{'other': 0, 'cafe': 1}

Phân bố nhãn:
label_name
cafe 3739
other 3492
Name: count, dtype: int64

Cell 3 — Chia train / temp

```

from sklearn.model_selection import train_test_split

train_df, temp_df = train_test_split(
    df,
    test_size=0.30,
    stratify=df["label"],
    random_state=42
)

print("Train:", len(train_df))
print("Temp :", len(temp_df))

```

Train: 5061
Temp : 2170

Cell 4 — Chia temp thành val / test

```

val_df, test_df = train_test_split(
    temp_df,
    test_size=0.50,
    stratify=temp_df["label"],
    random_state=42
)

print("Train:", len(train_df))
print("Val  :", len(val_df))
print("Test :", len(test_df))

```

```

Train: 5061
Val   : 1085
Test  : 1085

```

Cell 5 — Kiểm tra phân bố lớp

```

def show_distribution(name, dataframe):
    print(f"\n===== {name} =====")
    print(dataframe["label_name"].value_counts())
    print("\nTỷ lệ:")
    print((dataframe["label_name"].value_counts(normalize=True) * 100).round(2))

show_distribution("TRAIN", train_df)
show_distribution("VALIDATION", val_df)
show_distribution("TEST", test_df)

```

```

===== TRAIN =====
label_name
cafe      2617
other     2444
Name: count, dtype: int64

```

```

Tỷ lệ:
label_name
cafe      51.71
other     48.29
Name: proportion, dtype: float64

```

```

===== VALIDATION =====

```

```
label_name
cafe      561
other     524
Name: count, dtype: int64
```

```
Tỷ lệ:
label_name
cafe      51.71
other     48.29
Name: proportion, dtype: float64
```

```
===== TEST =====
label_name
cafe      561
other     524
Name: count, dtype: int64
```

```
Tỷ lệ:
label_name
cafe      51.71
other     48.29
Name: proportion, dtype: float64
```

CELL 16 — Compile stage 1

Cell 6 — Reset index

```
train_df = train_df.reset_index(drop=True)
val_df = val_df.reset_index(drop=True)
test_df = test_df.reset_index(drop=True)
```

Cell 7 — Kiểm tra overlap giữa các tập

```
train_paths = set(train_df["file_path"])
val_paths = set(val_df["file_path"])
test_paths = set(test_df["file_path"])

print("Train  Val :", len(train_paths.intersection(val_paths)))
print("Train  Test:", len(train_paths.intersection(test_paths)))
print("Val    Test :", len(val_paths.intersection(test_paths)))
```

```
Train  Val : 0
```

```
Train  Test: 0
Val    Test   : 0
```

Cell 8 — Lưu split ra CSV

```
SAVE_SPLIT_DIR = r"d:\Máy học 5 classes\cafe_dataset_2lop_splits" # sửa nếu cần
os.makedirs(SAVE_SPLIT_DIR, exist_ok=True)

train_df.to_csv(os.path.join(SAVE_SPLIT_DIR, "train_split.csv"), index=False)
val_df.to_csv(os.path.join(SAVE_SPLIT_DIR, "val_split.csv"), index=False)
test_df.to_csv(os.path.join(SAVE_SPLIT_DIR, "test_split.csv"), index=False)

print("Đã lưu split vào:", SAVE_SPLIT_DIR)
```

Đã lưu split vào: d:\Máy học 5 classes\cafe_dataset_2lop_splits

Cell 1 — Đọc lại file split

```
import os
import pandas as pd
import torch

SAVE_SPLIT_DIR = r"d:\Máy học 5 classes\cafe_dataset_2lop_splits"

train_df = pd.read_csv(os.path.join(SAVE_SPLIT_DIR, "train_split.csv"))
val_df = pd.read_csv(os.path.join(SAVE_SPLIT_DIR, "val_split.csv"))
test_df = pd.read_csv(os.path.join(SAVE_SPLIT_DIR, "test_split.csv"))

print("Train:", train_df.shape)
print("Val   :", val_df.shape)
print("Test  :", test_df.shape)
```

```
Train: (5061, 3)
Val   : (1085, 3)
Test  : (1085, 3)
```

Cell 2 — Khai báo tham số

```

IMG_SIZE = 224
BATCH_SIZE = 32
NUM_WORKERS = 0 # để 0 cho ổn định trên máy local / Jupyter Windows

CLASS_NAMES = ["other", "cafe"]
NUM_CLASSES = len(CLASS_NAMES)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("IMG_SIZE =", IMG_SIZE)
print("BATCH_SIZE =", BATCH_SIZE)
print("NUM_CLASSES =", NUM_CLASSES)
print("DEVICE =", device)

```

```

IMG_SIZE = 224
BATCH_SIZE = 32
NUM_CLASSES = 2
DEVICE = cuda

```

Cell 3 — Dataset class đọc ảnh

```

from torch.utils.data import Dataset
from PIL import Image

class BinaryCafeDataset(Dataset):
    def __init__(self, dataframe, transform=None):
        self.dataframe = dataframe.reset_index(drop=True)
        self.transform = transform

    def __len__(self):
        return len(self.dataframe)

    def __getitem__(self, idx):
        file_path = self.dataframe.loc[idx, "file_path"]
        label = int(self.dataframe.loc[idx, "label"])

        image = Image.open(file_path).convert("RGB")

        if self.transform:
            image = self.transform(image)

        return image, label

```

Cell 4 — Data augmentation

```
from torchvision import transforms

train_transforms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(8),
    transforms.RandomAffine(degrees=0, translate=(0.08, 0.08)),
    transforms.ColorJitter(contrast=0.10),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

val_test_transforms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

print("Defined train_transforms and val_test_transforms.")
```

Defined train_transforms and val_test_transforms.

Cell 5 — Hàm tạo dataset / dataloader

```
from torch.utils.data import DataLoader

def make_dataloader(dataframe, transform, training=False):
    dataset = BinaryCafeDataset(dataframe, transform=transform)

    loader = DataLoader(
        dataset,
        batch_size=BATCH_SIZE,
        shuffle=training,
        num_workers=NUM_WORKERS,
```

```

        pin_memory=torch.cuda.is_available()
    )
    return loader

```

Cell 6 — Tạo train / val / test dataloader

```

train_loader = make_dataloader(train_df, transform=train_transforms, training=True)
val_loader = make_dataloader(val_df, transform=val_test_transforms, training=False)
test_loader = make_dataloader(test_df, transform=val_test_transforms, training=False)

print(train_loader)
print(val_loader)
print(test_loader)

```

```

<torch.utils.data.dataloader.DataLoader object at 0x00000225349448E0>
<torch.utils.data.dataloader.DataLoader object at 0x0000022534946110>
<torch.utils.data.dataloader.DataLoader object at 0x0000022534947130>

```

Cell 7 — Kiểm tra 1 batch

```

for images, labels in train_loader:
    print("Images shape:", images.shape)
    print("Labels shape:", labels.shape)
    print("Labels sample:", labels[:10].numpy())
    break

```

```

Images shape: torch.Size([32, 3, 224, 224])
Labels shape: torch.Size([32])
Labels sample: [1 1 0 1 0 1 1 0 1 0]

```

Cell 8 — Hiển thị thử batch ảnh

```

import matplotlib.pyplot as plt
import numpy as np

def unnormalize_image(img_tensor):
    img = img_tensor.numpy().transpose((1, 2, 0))
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])
    img = std * img + mean

```

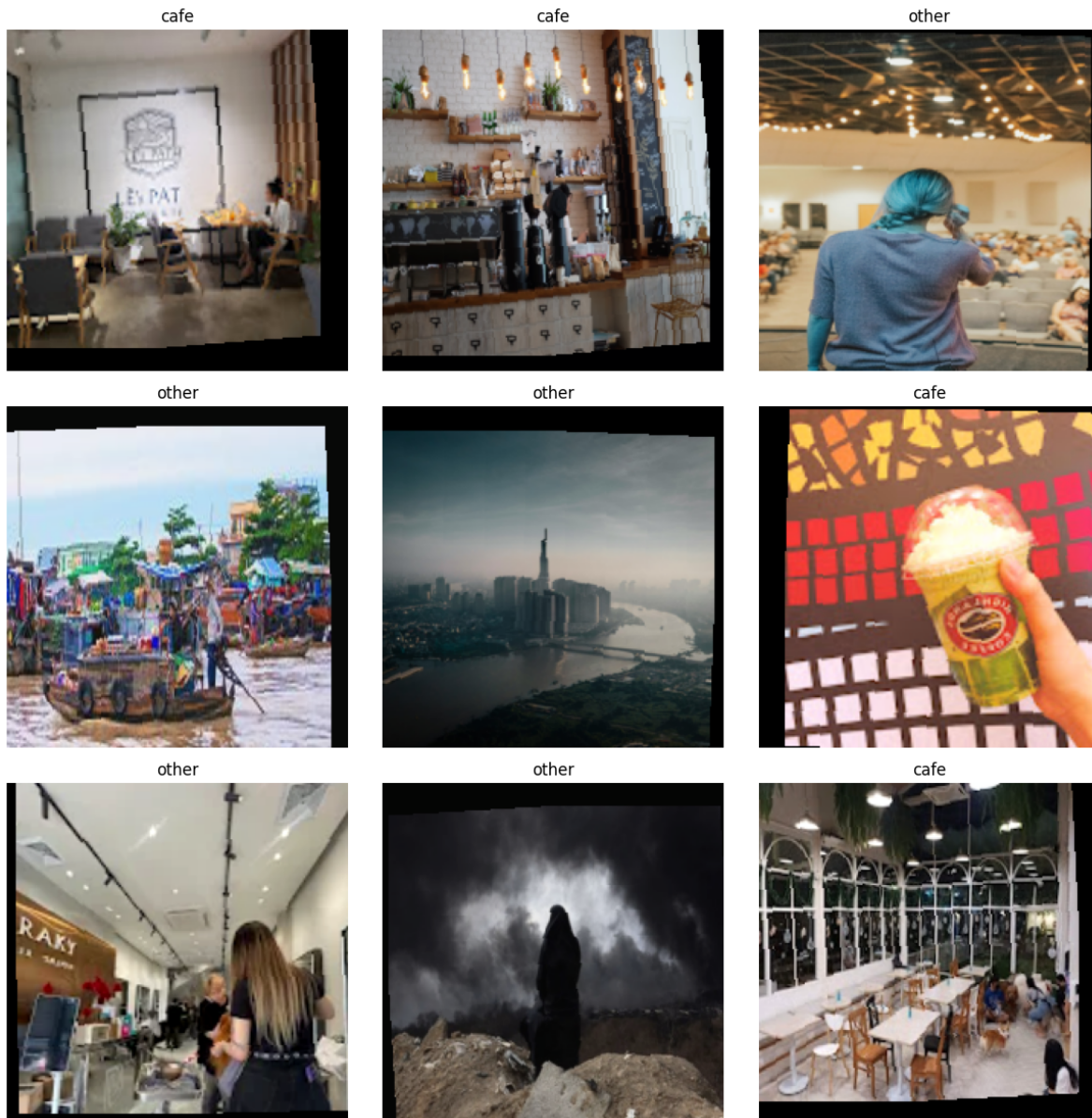
```

    img = np.clip(img, 0, 1)
    return img

def show_batch(dataloader, class_names, num_images=9):
    for images, labels in dataloader:
        plt.figure(figsize=(12, 12))
        for i in range(min(num_images, len(images))):
            ax = plt.subplot(3, 3, i + 1)
            img = unnormalize_image(images[i].cpu())
            plt.imshow(img)
            plt.title(class_names[int(labels[i])])
            plt.axis("off")
        plt.tight_layout()
        plt.show()
        break

show_batch(train_loader, CLASS_NAMES, num_images=9)

```

Cell 9 — Hiển thị thử augmentation

```
def show_augmentation_examples(dataframe, transform, class_names, num_images=9):
    sample_path = dataframe.loc[0, "file_path"]
    sample_label = int(dataframe.loc[0, "label"])

    original_image = Image.open(sample_path).convert("RGB")
```

```

plt.figure(figsize=(12, 12))
for i in range(num_images):
    aug_img = transform(original_image)
    aug_img = unnormalize_image(aug_img)

    ax = plt.subplot(3, 3, i + 1)
    plt.imshow(aug_img)
    plt.title(class_names[sample_label])
    plt.axis("off")

plt.tight_layout()
plt.show()

show_augmentation_examples(train_df, train_transforms, CLASS_NAMES, num_images=9)

```



Cell 1 — Import thư viện PyTorch cần thiết

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import models
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score
import numpy as np
```

```
import copy
```

Cell 2 — Kiểm tra device

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("DEVICE =", device)
```

DEVICE = cuda

Cell 3 — Build EfficientNetB0 Stage 1

```
from torchvision.models import efficientnet_b0, EfficientNet_B0_Weights

weights = EfficientNet_B0_Weights.DEFAULT
model_eff = efficientnet_b0(weights=weights)

# Đóng băng toàn bộ backbone
for param in model_eff.features.parameters():
    param.requires_grad = False

# Thay classifier head cho 2 lớp
in_features = model_eff.classifier[1].in_features
model_eff.classifier = nn.Sequential(
    nn.Dropout(p=0.3),
    nn.Linear(in_features, 2)
)

model_eff = model_eff.to(device)
print(model_eff)
```

```
EfficientNet(
  (features): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (1): Sequential(
      (0): MBConv(
        (block): Sequential(
```

```

(0): Conv2dNormActivation(
  (0): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), groups=32)
  (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (2): SiLU(inplace=True)
)
(1): SqueezeExcitation(
  (avgpool): AdaptiveAvgPool2d(output_size=1)
  (fc1): Conv2d(32, 8, kernel_size=(1, 1), stride=(1, 1))
  (fc2): Conv2d(8, 32, kernel_size=(1, 1), stride=(1, 1))
  (activation): SiLU(inplace=True)
  (scale_activation): Sigmoid()
)
(2): Conv2dNormActivation(
  (0): Conv2d(32, 16, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (1): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
)
)
(stochastic_depth): StochasticDepth(p=0.0, mode=row)
)
)
(2): Sequential(
  (0): MBConv(
    (block): Sequential(
      (0): Conv2dNormActivation(
        (0): Conv2d(16, 96, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): SiLU(inplace=True)
      )
      (1): Conv2dNormActivation(
        (0): Conv2d(96, 96, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), groups=96)
        (1): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): SiLU(inplace=True)
      )
      (2): SqueezeExcitation(
        (avgpool): AdaptiveAvgPool2d(output_size=1)
        (fc1): Conv2d(96, 4, kernel_size=(1, 1), stride=(1, 1))
        (fc2): Conv2d(4, 96, kernel_size=(1, 1), stride=(1, 1))
        (activation): SiLU(inplace=True)
        (scale_activation): Sigmoid()
      )
      (3): Conv2dNormActivation(
        (0): Conv2d(96, 24, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(24, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

```

```

    )
    )
    (stochastic_depth): StochasticDepth(p=0.0125, mode=row)
)
(1): MBConv(
  (block): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(24, 144, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(144, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(144, 144, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), groups=1)
      (1): BatchNorm2d(144, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (2): SqueezeExcitation(
      (avgpool): AdaptiveAvgPool2d(output_size=1)
      (fc1): Conv2d(144, 6, kernel_size=(1, 1), stride=(1, 1))
      (fc2): Conv2d(6, 144, kernel_size=(1, 1), stride=(1, 1))
      (activation): SiLU(inplace=True)
      (scale_activation): Sigmoid()
    )
    (3): Conv2dNormActivation(
      (0): Conv2d(144, 24, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(24, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (stochastic_depth): StochasticDepth(p=0.025, mode=row)
)
(3): Sequential(
  (0): MBConv(
    (block): Sequential(
      (0): Conv2dNormActivation(
        (0): Conv2d(24, 144, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(144, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): SiLU(inplace=True)
      )
      (1): Conv2dNormActivation(
        (0): Conv2d(144, 144, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2), groups=1)
        (1): BatchNorm2d(144, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): SiLU(inplace=True)
      )
    )
  )

```

```

    )
    (2): SqueezeExcitation(
      (avgpool): AdaptiveAvgPool2d(output_size=1)
      (fc1): Conv2d(144, 6, kernel_size=(1, 1), stride=(1, 1))
      (fc2): Conv2d(6, 144, kernel_size=(1, 1), stride=(1, 1))
      (activation): SiLU(inplace=True)
      (scale_activation): Sigmoid()
    )
    (3): Conv2dNormActivation(
      (0): Conv2d(144, 40, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(40, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (stochastic_depth): StochasticDepth(p=0.037500000000000006, mode=row)
)
(1): MBConv(
  (block): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(40, 240, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(240, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(240, 240, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2), groups=1)
      (1): BatchNorm2d(240, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (2): SqueezeExcitation(
      (avgpool): AdaptiveAvgPool2d(output_size=1)
      (fc1): Conv2d(240, 10, kernel_size=(1, 1), stride=(1, 1))
      (fc2): Conv2d(10, 240, kernel_size=(1, 1), stride=(1, 1))
      (activation): SiLU(inplace=True)
      (scale_activation): Sigmoid()
    )
    (3): Conv2dNormActivation(
      (0): Conv2d(240, 40, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(40, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (stochastic_depth): StochasticDepth(p=0.05, mode=row)
)
(4): Sequential(

```

```

(0): MBConv(
  (block): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(40, 240, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(240, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(240, 240, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), groups=1)
      (1): BatchNorm2d(240, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (2): SqueezeExcitation(
      (avgpool): AdaptiveAvgPool2d(output_size=1)
      (fc1): Conv2d(240, 10, kernel_size=(1, 1), stride=(1, 1))
      (fc2): Conv2d(10, 240, kernel_size=(1, 1), stride=(1, 1))
      (activation): SiLU(inplace=True)
      (scale_activation): Sigmoid()
    )
    (3): Conv2dNormActivation(
      (0): Conv2d(240, 80, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(80, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (stochastic_depth): StochasticDepth(p=0.0625, mode=row)
)
(1): MBConv(
  (block): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(80, 480, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(480, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(480, 480, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), groups=1)
      (1): BatchNorm2d(480, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (2): SqueezeExcitation(
      (avgpool): AdaptiveAvgPool2d(output_size=1)
      (fc1): Conv2d(480, 20, kernel_size=(1, 1), stride=(1, 1))
      (fc2): Conv2d(20, 480, kernel_size=(1, 1), stride=(1, 1))
      (activation): SiLU(inplace=True)
    )
  )
)

```



```

        (scale_activation): Sigmoid()
    )
    (3): Conv2dNormActivation(
      (0): Conv2d(480, 80, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(80, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (stochastic_depth): StochasticDepth(p=0.07500000000000001, mode=row)
)
(2): MBConv(
  (block): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(80, 480, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(480, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(480, 480, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), groups=80)
      (1): BatchNorm2d(480, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (2): SqueezeExcitation(
      (avgpool): AdaptiveAvgPool2d(output_size=1)
      (fc1): Conv2d(480, 20, kernel_size=(1, 1), stride=(1, 1))
      (fc2): Conv2d(20, 480, kernel_size=(1, 1), stride=(1, 1))
      (activation): SiLU(inplace=True)
      (scale_activation): Sigmoid()
    )
    (3): Conv2dNormActivation(
      (0): Conv2d(480, 80, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(80, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (stochastic_depth): StochasticDepth(p=0.08750000000000001, mode=row)
)
)
(5): Sequential(
  (0): MBConv(
    (block): Sequential(
      (0): Conv2dNormActivation(
        (0): Conv2d(80, 480, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(480, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): SiLU(inplace=True)

```

```

)
(1): Conv2dNormActivation(
  (0): Conv2d(480, 480, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2), groups=1)
  (1): BatchNorm2d(480, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (2): SiLU(inplace=True)
)
(2): SqueezeExcitation(
  (avgpool): AdaptiveAvgPool2d(output_size=1)
  (fc1): Conv2d(480, 20, kernel_size=(1, 1), stride=(1, 1))
  (fc2): Conv2d(20, 480, kernel_size=(1, 1), stride=(1, 1))
  (activation): SiLU(inplace=True)
  (scale_activation): Sigmoid()
)
(3): Conv2dNormActivation(
  (0): Conv2d(480, 112, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (1): BatchNorm2d(112, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
)
)
(stochastic_depth): StochasticDepth(p=0.1, mode=row)
)
(1): MBConv(
  (block): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(112, 672, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(672, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(672, 672, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2), groups=1)
      (1): BatchNorm2d(672, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (2): SqueezeExcitation(
      (avgpool): AdaptiveAvgPool2d(output_size=1)
      (fc1): Conv2d(672, 28, kernel_size=(1, 1), stride=(1, 1))
      (fc2): Conv2d(28, 672, kernel_size=(1, 1), stride=(1, 1))
      (activation): SiLU(inplace=True)
      (scale_activation): Sigmoid()
    )
    (3): Conv2dNormActivation(
      (0): Conv2d(672, 112, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(112, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
)

```

```

    )
    (stochastic_depth): StochasticDepth(p=0.1125, mode=row)
)
(2): MBConv(
  (block): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(112, 672, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(672, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(672, 672, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2), groups=672)
      (1): BatchNorm2d(672, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (2): SqueezeExcitation(
      (avgpool): AdaptiveAvgPool2d(output_size=1)
      (fc1): Conv2d(672, 28, kernel_size=(1, 1), stride=(1, 1))
      (fc2): Conv2d(28, 672, kernel_size=(1, 1), stride=(1, 1))
      (activation): SiLU(inplace=True)
      (scale_activation): Sigmoid()
    )
    (3): Conv2dNormActivation(
      (0): Conv2d(672, 112, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(112, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (stochastic_depth): StochasticDepth(p=0.125, mode=row)
)
(6): Sequential(
  (0): MBConv(
    (block): Sequential(
      (0): Conv2dNormActivation(
        (0): Conv2d(112, 672, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(672, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): SiLU(inplace=True)
      )
      (1): Conv2dNormActivation(
        (0): Conv2d(672, 672, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2), groups=672)
        (1): BatchNorm2d(672, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): SiLU(inplace=True)
      )
    )
  )
)

```

```

(2): SqueezeExcitation(
  (avgpool): AdaptiveAvgPool2d(output_size=1)
  (fc1): Conv2d(672, 28, kernel_size=(1, 1), stride=(1, 1))
  (fc2): Conv2d(28, 672, kernel_size=(1, 1), stride=(1, 1))
  (activation): SiLU(inplace=True)
  (scale_activation): Sigmoid()
)
(3): Conv2dNormActivation(
  (0): Conv2d(672, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (1): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
)
)
(stochastic_depth): StochasticDepth(p=0.1375, mode=row)
)
(1): MBConv(
  (block): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(192, 1152, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(1152, 1152, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2), groups=1)
      (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (2): SqueezeExcitation(
      (avgpool): AdaptiveAvgPool2d(output_size=1)
      (fc1): Conv2d(1152, 48, kernel_size=(1, 1), stride=(1, 1))
      (fc2): Conv2d(48, 1152, kernel_size=(1, 1), stride=(1, 1))
      (activation): SiLU(inplace=True)
      (scale_activation): Sigmoid()
    )
    (3): Conv2dNormActivation(
      (0): Conv2d(1152, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (stochastic_depth): StochasticDepth(p=0.15000000000000002, mode=row)
)
(2): MBConv(
  (block): Sequential(
    (0): Conv2dNormActivation(

```

```

        (0): Conv2d(192, 1152, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): SiLU(inplace=True)
    )
    (1): Conv2dNormActivation(
        (0): Conv2d(1152, 1152, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2), groups=1)
        (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): SiLU(inplace=True)
    )
    (2): SqueezeExcitation(
        (avgpool): AdaptiveAvgPool2d(output_size=1)
        (fc1): Conv2d(1152, 48, kernel_size=(1, 1), stride=(1, 1))
        (fc2): Conv2d(48, 1152, kernel_size=(1, 1), stride=(1, 1))
        (activation): SiLU(inplace=True)
        (scale_activation): Sigmoid()
    )
    (3): Conv2dNormActivation(
        (0): Conv2d(1152, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
)
(stochastic_depth): StochasticDepth(p=0.1625, mode=row)
)
(3): MBConv(
  (block): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(192, 1152, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(1152, 1152, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2), groups=1)
      (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (2): SqueezeExcitation(
      (avgpool): AdaptiveAvgPool2d(output_size=1)
      (fc1): Conv2d(1152, 48, kernel_size=(1, 1), stride=(1, 1))
      (fc2): Conv2d(48, 1152, kernel_size=(1, 1), stride=(1, 1))
      (activation): SiLU(inplace=True)
      (scale_activation): Sigmoid()
    )
    (3): Conv2dNormActivation(

```

```

        (0): Conv2d(1152, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
)
(stochastic_depth): StochasticDepth(p=0.17500000000000002, mode=row)
)
)
(7): Sequential(
  (0): MBConv(
    (block): Sequential(
      (0): Conv2dNormActivation(
        (0): Conv2d(192, 1152, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): SiLU(inplace=True)
      )
      (1): Conv2dNormActivation(
        (0): Conv2d(1152, 1152, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), groups=1)
        (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): SiLU(inplace=True)
      )
      (2): SqueezeExcitation(
        (avgpool): AdaptiveAvgPool2d(output_size=1)
        (fc1): Conv2d(1152, 48, kernel_size=(1, 1), stride=(1, 1))
        (fc2): Conv2d(48, 1152, kernel_size=(1, 1), stride=(1, 1))
        (activation): SiLU(inplace=True)
        (scale_activation): Sigmoid()
      )
      (3): Conv2dNormActivation(
        (0): Conv2d(1152, 320, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(320, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (stochastic_depth): StochasticDepth(p=0.1875, mode=row)
  )
)
(8): Conv2dNormActivation(
  (0): Conv2d(320, 1280, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (1): BatchNorm2d(1280, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (2): SiLU(inplace=True)
)
)
(avgpool): AdaptiveAvgPool2d(output_size=1)
(classifier): Sequential(

```

```

        (0): Dropout(p=0.3, inplace=False)
        (1): Linear(in_features=1280, out_features=2, bias=True)
    )
)

```

Cell 4 — Loss, optimizer

```

criterion = nn.CrossEntropyLoss()

optimizer = optim.Adam(
    model_eff.classifier.parameters(),
    lr=1e-3
)

```

Cell 5 — Hàm train 1 epoch

```

def train_one_epoch(model, dataloader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    all_preds = []
    all_labels = []

    for images, labels in dataloader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()

        outputs = model(images)
        loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)

        preds = torch.argmax(outputs, dim=1)
        all_preds.extend(preds.detach().cpu().numpy())
        all_labels.extend(labels.detach().cpu().numpy())

    epoch_loss = running_loss / len(dataloader.dataset)
    epoch_acc = accuracy_score(all_labels, all_preds)

```

```
return epoch_loss, epoch_acc
```

Cell 6 — Hàm evaluate

```
def evaluate(model, dataloader, criterion, device):
    model.eval()
    running_loss = 0.0
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for images, labels in dataloader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            loss = criterion(outputs, labels)

            running_loss += loss.item() * images.size(0)

            preds = torch.argmax(outputs, dim=1)
            all_preds.extend(preds.detach().cpu().numpy())
            all_labels.extend(labels.detach().cpu().numpy())

    epoch_loss = running_loss / len(dataloader.dataset)
    epoch_acc = accuracy_score(all_labels, all_preds)

    return epoch_loss, epoch_acc, np.array(all_labels), np.array(all_preds)
```

Cell 7 — Hàm train stage 1 có lưu best model

```
def train_model_stage1(model, train_loader, val_loader, criterion, optimizer, device, epochs):
    best_model_wts = copy.deepcopy(model.state_dict())
    best_val_acc = 0.0

    history = {
        "train_loss": [],
        "train_acc": [],
        "val_loss": [],
        "val_acc": []
    }
```



```

for epoch in range(epochs):
    print(f"\nEpoch {epoch+1}/{epochs}")

    train_loss, train_acc = train_one_epoch(model, train_loader, criterion, optimizer, device)
    val_loss, val_acc, _, _ = evaluate(model, val_loader, criterion, device)

    history["train_loss"].append(train_loss)
    history["train_acc"].append(train_acc)
    history["val_loss"].append(val_loss)
    history["val_acc"].append(val_acc)

    print(f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f}")
    print(f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f}")

    if val_acc > best_val_acc:
        best_val_acc = val_acc
        best_model_wts = copy.deepcopy(model.state_dict())
        torch.save(model.state_dict(), "best_efficientnetb0_stage1_binary.pth")
        print(" Saved best EfficientNetB0 Stage 1")

    model.load_state_dict(best_model_wts)
    return model, history, best_val_acc

```

Cell 8 — Train EfficientNetB0 Stage 1

```

model_eff, history_eff, best_val_acc_eff = train_model_stage1(
    model=model_eff,
    train_loader=train_loader,
    val_loader=val_loader,
    criterion=criterion,
    optimizer=optimizer,
    device=device,
    epochs=8
)

print("\nBest Val Accuracy EfficientNetB0:", best_val_acc_eff)

```

Epoch 1/8

Train Loss: 0.5007 | Train Acc: 0.7704

Val Loss: 0.3840 | Val Acc: 0.8369

Saved best EfficientNetB0 Stage 1

Epoch 2/8

Train Loss: 0.4190 | Train Acc: 0.8143

Val Loss: 0.3717 | Val Acc: 0.8452

Saved best EfficientNetB0 Stage 1

Epoch 3/8

Train Loss: 0.3974 | Train Acc: 0.8198

Val Loss: 0.3474 | Val Acc: 0.8562

Saved best EfficientNetB0 Stage 1

Epoch 4/8

Train Loss: 0.3899 | Train Acc: 0.8275

Val Loss: 0.3427 | Val Acc: 0.8627

Saved best EfficientNetB0 Stage 1

Epoch 5/8

Train Loss: 0.3792 | Train Acc: 0.8291

Val Loss: 0.3275 | Val Acc: 0.8737

Saved best EfficientNetB0 Stage 1

Epoch 6/8

Train Loss: 0.3823 | Train Acc: 0.8291

Val Loss: 0.3193 | Val Acc: 0.8728

Epoch 7/8

Train Loss: 0.3812 | Train Acc: 0.8334

Val Loss: 0.3338 | Val Acc: 0.8654

Epoch 8/8

Train Loss: 0.3774 | Train Acc: 0.8362

Val Loss: 0.3261 | Val Acc: 0.8618

Best Val Accuracy EfficientNetB0: 0.8737327188940092

Cell 9 — Evaluate trên test

```
test_loss_eff, test_acc_eff, y_true_eff, y_pred_eff = evaluate(
    model_eff, test_loader, criterion, device
)
```

```
print(f"EfficientNetB0 Stage 1 Test Loss: {test_loss_eff:.4f}")
print(f"EfficientNetB0 Stage 1 Test Acc : {test_acc_eff:.4f}")
```

```
EfficientNetB0 Stage 1 Test Loss: 0.3330
EfficientNetB0 Stage 1 Test Acc : 0.8654
```

Cell 10 — Classification report

```
print(classification_report(
    y_true_eff,
    y_pred_eff,
    target_names=["other", "cafe"],
    digits=4
))
```

	precision	recall	f1-score	support
other	0.8351	0.8989	0.8658	524
cafe	0.8983	0.8342	0.8651	561
accuracy			0.8654	1085
macro avg	0.8667	0.8665	0.8654	1085
weighted avg	0.8678	0.8654	0.8654	1085

Cell 11 — F1 score

```
macro_f1_eff = f1_score(y_true_eff, y_pred_eff, average="macro")
weighted_f1_eff = f1_score(y_true_eff, y_pred_eff, average="weighted")

print(f"EfficientNetB0 Stage 1 Macro F1 : {macro_f1_eff:.4f}")
print(f"EfficientNetB0 Stage 1 Weighted F1: {weighted_f1_eff:.4f}")
```

```
EfficientNetB0 Stage 1 Macro F1 : 0.8654
EfficientNetB0 Stage 1 Weighted F1: 0.8654
```

Cell 12 — Confusion matrix

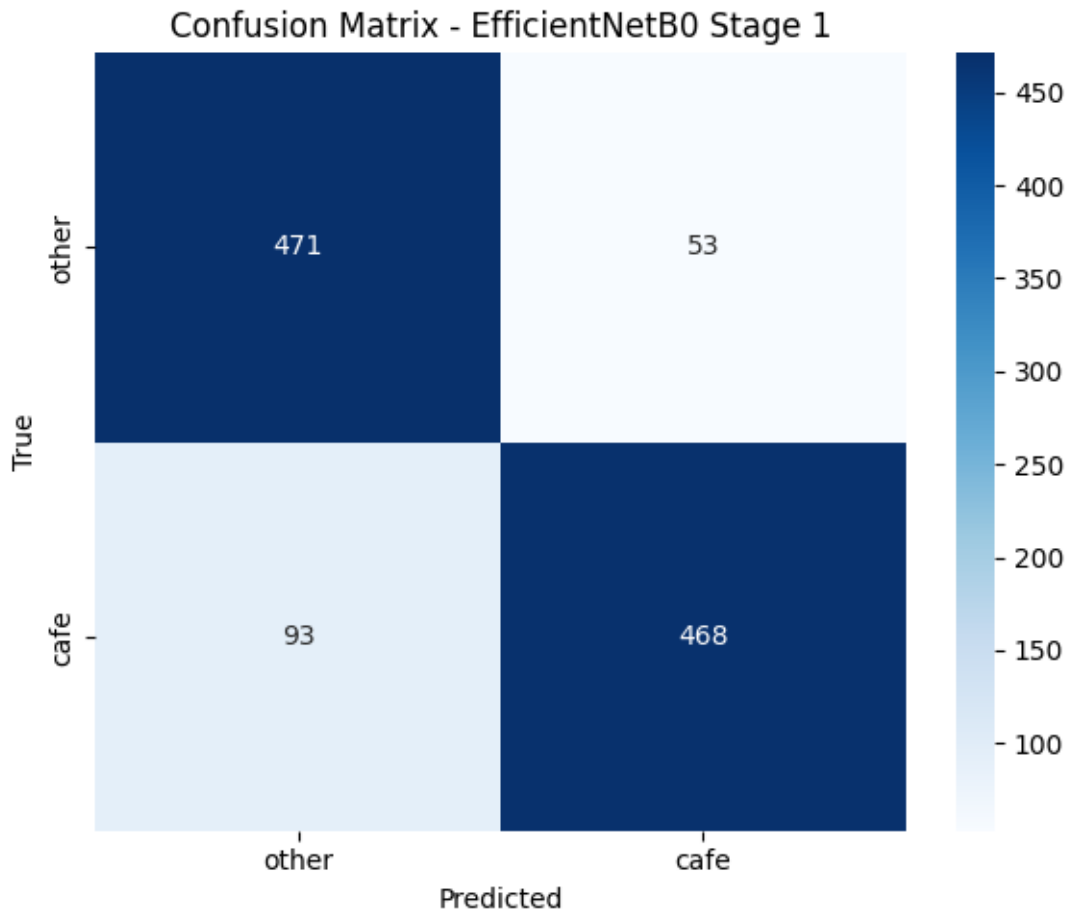
```

import matplotlib.pyplot as plt
import seaborn as sns

cm_eff = confusion_matrix(y_true_eff, y_pred_eff)

plt.figure(figsize=(6, 5))
sns.heatmap(
    cm_eff,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["other", "cafe"],
    yticklabels=["other", "cafe"]
)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix - EfficientNetB0 Stage 1")
plt.tight_layout()
plt.show()

```



Cell 13 — Vẽ biểu đồ train

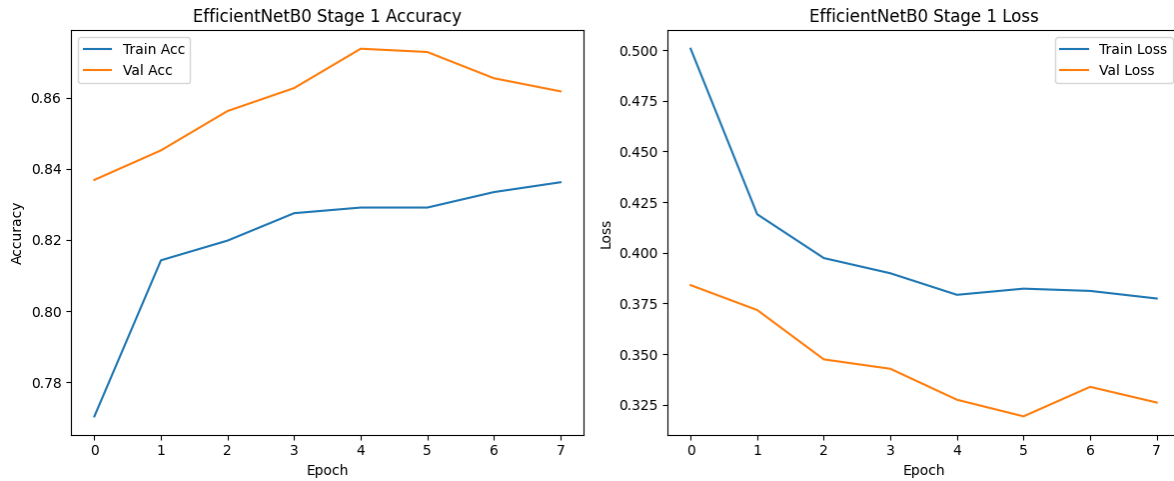
```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history_eff["train_acc"], label="Train Acc")
plt.plot(history_eff["val_acc"], label="Val Acc")
plt.title("EfficientNetB0 Stage 1 Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history_eff["train_loss"], label="Train Loss")
plt.plot(history_eff["val_loss"], label="Val Loss")
plt.title("EfficientNetB0 Stage 1 Loss")
```

```
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
```

```
plt.tight_layout()
plt.show()
```



BƯỚC 5: Train ResNet50 Stage 1 bằng PyTorch Mục tiêu bước này Ta sẽ:

dùng pretrained ResNet50 đóng băng backbone chỉ train head đánh giá trên validation và test so sánh với EfficientNetB0

Cell 1 — Build ResNet50 Stage 1

```
from torchvision.models import resnet50, ResNet50_Weights

weights = ResNet50_Weights.DEFAULT
model_res = resnet50(weights=weights)

# Đóng băng toàn bộ backbone
for param in model_res.parameters():
    param.requires_grad = False

# Thay fully connected head cho 2 lớp
in_features = model_res.fc.in_features
model_res.fc = nn.Sequential(
    nn.Dropout(p=0.3),
    nn.Linear(in_features, 2)
)
```

```
# Chi train fc
for param in model_res.fc.parameters():
    param.requires_grad = True

model_res = model_res.to(device)
print(model_res)
```

Downloading: "<https://download.pytorch.org/models/resnet50-11ad3fa6.pth>" to C:\Users\ASUS/.cache\pytorch\torchmodels\resnet50-11ad3fa6.pth

100%| | 97.8M/97.8M [00:13<00:00, 7.48MB/s]

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
  (layer1): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
  )
  (1): Bottleneck(
    (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
  )
  (2): Bottleneck(
```

```

(conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
(bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
(bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
(bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(rel): ReLU(inplace=True)
)
)
(layer2): Sequential(
  (0): Bottleneck(
    (conv1): Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (downsample): Sequential(
      (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
      (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
)
(1): Bottleneck(
  (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
  (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
)
(2): Bottleneck(
  (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
  (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
)
(3): Bottleneck(
  (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)

```



```

        (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
    )
)
(layer3): Sequential(
  (0): Bottleneck(
    (conv1): Conv2d(512, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (downsample): Sequential(
      (0): Conv2d(512, 1024, kernel_size=(1, 1), stride=(2, 2), bias=False)
      (1): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
)
(1): Bottleneck(
  (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
  (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
)
(2): Bottleneck(
  (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
  (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
)
(3): Bottleneck(
  (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

```

```

(conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
(bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
(bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(rel): ReLU(inplace=True)
)
(4): Bottleneck(
  (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
  (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (rel): ReLU(inplace=True)
)
(5): Bottleneck(
  (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
  (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (rel): ReLU(inplace=True)
)
)
(layer4): Sequential(
  (0): Bottleneck(
    (conv1): Conv2d(1024, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (rel): ReLU(inplace=True)
    (downsample): Sequential(
      (0): Conv2d(1024, 2048, kernel_size=(1, 1), stride=(2, 2), bias=False)
      (1): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
)
(1): Bottleneck(
  (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)

```

```

        (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc): Sequential(
    (0): Dropout(p=0.3, inplace=False)
    (1): Linear(in_features=2048, out_features=2, bias=True)
  )
)

```

Cell 2 — Loss và optimizer

```

criterion = nn.CrossEntropyLoss()

optimizer_res = optim.Adam(
    model_res.fc.parameters(),
    lr=1e-3
)

```

Cell 3 — Hàm train stage 1 cho ResNet50 Ta dùng lại chính hàm đã viết trước:

train_one_epoch evaluate train_model_stage1 Không cần viết lại nếu vẫn còn trong notebook.

Nếu bạn đã restart kernel thì chạy lại các hàm này:

```

import copy
from sklearn.metrics import accuracy_score

def train_one_epoch(model, dataloader, criterion, optimizer, device):
    model.train()

```

```

running_loss = 0.0
all_preds = []
all_labels = []

for images, labels in dataloader:
    images = images.to(device)
    labels = labels.to(device)

    optimizer.zero_grad()

    outputs = model(images)
    loss = criterion(outputs, labels)

    loss.backward()
    optimizer.step()

    running_loss += loss.item() * images.size(0)

    preds = torch.argmax(outputs, dim=1)
    all_preds.extend(preds.detach().cpu().numpy())
    all_labels.extend(labels.detach().cpu().numpy())

epoch_loss = running_loss / len(dataloader.dataset)
epoch_acc = accuracy_score(all_labels, all_preds)

return epoch_loss, epoch_acc

def evaluate(model, dataloader, criterion, device):
    model.eval()
    running_loss = 0.0
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for images, labels in dataloader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            loss = criterion(outputs, labels)

```

```

        running_loss += loss.item() * images.size(0)

        preds = torch.argmax(outputs, dim=1)
        all_preds.extend(preds.detach().cpu().numpy())
        all_labels.extend(labels.detach().cpu().numpy())

    epoch_loss = running_loss / len(dataloader.dataset)
    epoch_acc = accuracy_score(all_labels, all_preds)

    return epoch_loss, epoch_acc, np.array(all_labels), np.array(all_preds)

def train_model_stage1(model, train_loader, val_loader, criterion, optimizer, device, epochs):
    best_model_wts = copy.deepcopy(model.state_dict())
    best_val_acc = 0.0

    history = {
        "train_loss": [],
        "train_acc": [],
        "val_loss": [],
        "val_acc": []
    }

    for epoch in range(epochs):
        print(f"\nEpoch {epoch+1}/{epochs}")

        train_loss, train_acc = train_one_epoch(model, train_loader, criterion, optimizer, device)
        val_loss, val_acc, _, _ = evaluate(model, val_loader, criterion, device)

        history["train_loss"].append(train_loss)
        history["train_acc"].append(train_acc)
        history["val_loss"].append(val_loss)
        history["val_acc"].append(val_acc)

        print(f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f}")
        print(f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f}")

        if val_acc > best_val_acc:
            best_val_acc = val_acc
            best_model_wts = copy.deepcopy(model.state_dict())
            torch.save(model.state_dict(), save_path)
            print(" Saved best model")

```

```
model.load_state_dict(best_model_wts)
return model, history, best_val_acc
```

Cell 4 — Train ResNet50 Stage 1

```
model_res, history_res, best_val_acc_res = train_model_stage1(
    model=model_res,
    train_loader=train_loader,
    val_loader=val_loader,
    criterion=criterion,
    optimizer=optimizer_res,
    device=device,
    epochs=8,
    save_path="best_resnet50_stage1_binary.pth"
)

print("\nBest Val Accuracy ResNet50:", best_val_acc_res)
```

Epoch 1/8

Train Loss: 0.4046 | Train Acc: 0.8257

Val Loss: 0.3087 | Val Acc: 0.8866

Saved best model

Epoch 2/8

Train Loss: 0.2966 | Train Acc: 0.8783

Val Loss: 0.2525 | Val Acc: 0.9032

Saved best model

Epoch 3/8

Train Loss: 0.2785 | Train Acc: 0.8876

Val Loss: 0.2315 | Val Acc: 0.9152

Saved best model

Epoch 4/8

Train Loss: 0.2588 | Train Acc: 0.8935

Val Loss: 0.2222 | Val Acc: 0.9134

Epoch 5/8

Train Loss: 0.2463 | Train Acc: 0.8951

Val Loss: 0.2127 | Val Acc: 0.9198

Saved best model

Epoch 6/8

Train Loss: 0.2386 | Train Acc: 0.9018

Val Loss: 0.2093 | Val Acc: 0.9161

Epoch 7/8

Train Loss: 0.2407 | Train Acc: 0.9008

Val Loss: 0.2124 | Val Acc: 0.9152

Epoch 8/8

Train Loss: 0.2396 | Train Acc: 0.9042

Val Loss: 0.2020 | Val Acc: 0.9180

Best Val Accuracy ResNet50: 0.919815668202765

Cell 5 — Evaluate trên test

```
test_loss_res, test_acc_res, y_true_res, y_pred_res = evaluate(
    model_res, test_loader, criterion, device
)

print(f"ResNet50 Stage 1 Test Loss: {test_loss_res:.4f}")
print(f"ResNet50 Stage 1 Test Acc : {test_acc_res:.4f}")
```

ResNet50 Stage 1 Test Loss: 0.2280

ResNet50 Stage 1 Test Acc : 0.9097

Cell 6 — Classification report

```
print(classification_report(
    y_true_res,
    y_pred_res,
    target_names=["other", "cafe"],
    digits=4
))
```

	precision	recall	f1-score	support
other	0.9049	0.9084	0.9067	524
cafe	0.9141	0.9109	0.9125	561

accuracy			0.9097	1085
macro avg	0.9095	0.9096	0.9096	1085
weighted avg	0.9097	0.9097	0.9097	1085

Cell 7 — F1 score

```
macro_f1_res = f1_score(y_true_res, y_pred_res, average="macro")
weighted_f1_res = f1_score(y_true_res, y_pred_res, average="weighted")

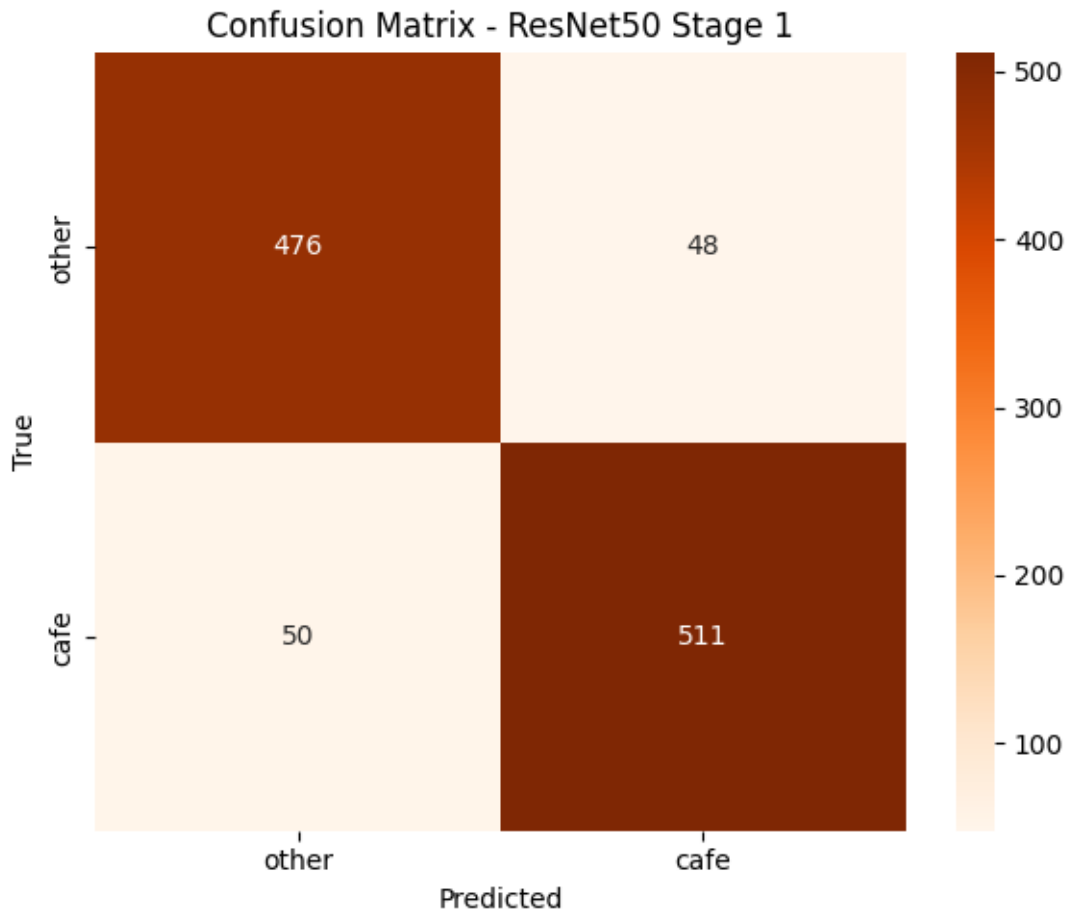
print(f"ResNet50 Stage 1 Macro F1    : {macro_f1_res:.4f}")
print(f"ResNet50 Stage 1 Weighted F1: {weighted_f1_res:.4f}")
```

```
ResNet50 Stage 1 Macro F1    : 0.9096
ResNet50 Stage 1 Weighted F1: 0.9097
```

Cell 8 — Confusion matrix

```
cm_res = confusion_matrix(y_true_res, y_pred_res)

plt.figure(figsize=(6, 5))
sns.heatmap(
    cm_res,
    annot=True,
    fmt="d",
    cmap="Oranges",
    xticklabels=["other", "cafe"],
    yticklabels=["other", "cafe"]
)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix - ResNet50 Stage 1")
plt.tight_layout()
plt.show()
```

Cell 9 — Vẽ biểu đồ train

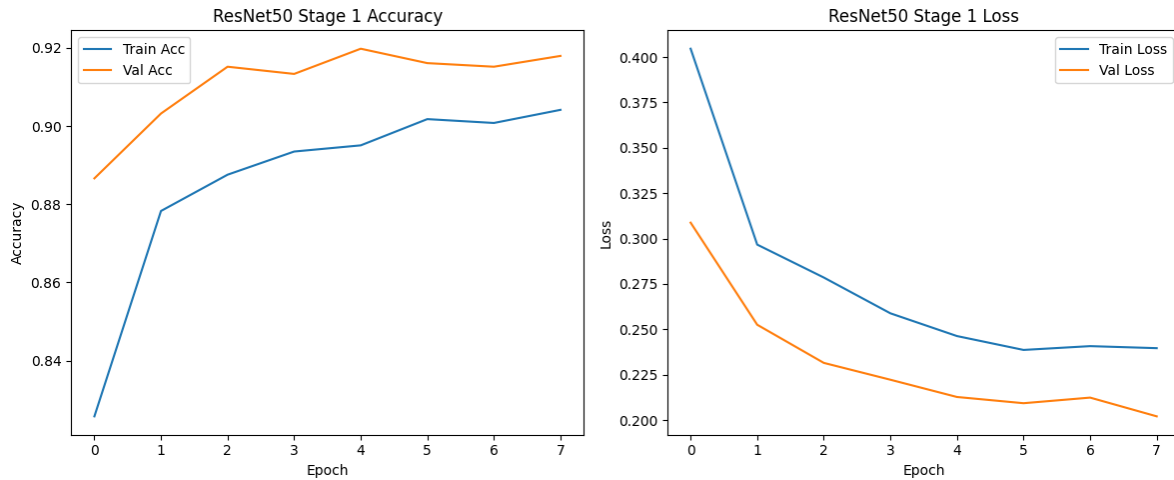
```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history_res["train_acc"], label="Train Acc")
plt.plot(history_res["val_acc"], label="Val Acc")
plt.title("ResNet50 Stage 1 Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history_res["train_loss"], label="Train Loss")
plt.plot(history_res["val_loss"], label="Val Loss")
plt.title("ResNet50 Stage 1 Loss")
```

```
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
```

```
plt.tight_layout()
plt.show()
```



BƯỚC 6: Fine-tune ResNet50 cho bài toán binary Mục tiêu Ta sẽ:

build lại ResNet50 load weights stage 1 mở layer4 và fc fine-tune đánh giá test so sánh với stage 1

Cell 1 — Build lại ResNet50

```
from torchvision.models import resnet50, ResNet50_Weights
import torch.nn as nn

weights = ResNet50_Weights.DEFAULT
model_res_ft = resnet50(weights=weights)

# thay head giống stage 1
in_features = model_res_ft.fc.in_features
model_res_ft.fc = nn.Sequential(
    nn.Dropout(p=0.3),
    nn.Linear(in_features, 2)
)

# load weights stage 1
model_res_ft.load_state_dict(torch.load("best_resnet50_stage1_binary.pth", map_location=device))
```

```
model_res_ft = model_res_ft.to(device)
print("Loaded ResNet50 Stage 1 weights.")
```

Loaded ResNet50 Stage 1 weights.

Cell 2 — Freeze toàn bộ trước

```
for param in model_res_ft.parameters():
    param.requires_grad = False
```

Cell 3 — Mở layer4 và fc để fine-tune

```
for param in model_res_ft.layer4.parameters():
    param.requires_grad = True

for param in model_res_ft.fc.parameters():
    param.requires_grad = True

# kiểm tra số params trainable
trainable_params = sum(p.numel() for p in model_res_ft.parameters() if p.requires_grad)
total_params = sum(p.numel() for p in model_res_ft.parameters())

print("Trainable params:", trainable_params)
print("Total params      :", total_params)
```

```
Trainable params: 14968834
Total params      : 23512130
```

Cell 4 — Loss và optimizer fine-tune Chỉ optimize các tham số đang mở.

```
criterion = nn.CrossEntropyLoss()

optimizer_res_ft = optim.Adam(
    filter(lambda p: p.requires_grad, model_res_ft.parameters()),
    lr=1e-5
)
```

Cell 5 — Hàm train fine-tune Ta có thể dùng lại train_model_stage1, chỉ đổi tên file lưu.

```

model_res_ft, history_res_ft, best_val_acc_res_ft = train_model_stage1(
    model=model_res_ft,
    train_loader=train_loader,
    val_loader=val_loader,
    criterion=criterion,
    optimizer=optimizer_res_ft,
    device=device,
    epochs=8,
    save_path="best_resnet50_finetuned_binary.pth"
)

print("\nBest Val Accuracy ResNet50 Fine-tuned:", best_val_acc_res_ft)

```

Epoch 1/8

Train Loss: 0.2326 | Train Acc: 0.9075

Val Loss: 0.1965 | Val Acc: 0.9198

Saved best model

Epoch 2/8

Train Loss: 0.2088 | Train Acc: 0.9162

Val Loss: 0.1931 | Val Acc: 0.9198

Epoch 3/8

Train Loss: 0.1882 | Train Acc: 0.9253

Val Loss: 0.1827 | Val Acc: 0.9309

Saved best model

Epoch 4/8

Train Loss: 0.1649 | Train Acc: 0.9366

Val Loss: 0.1755 | Val Acc: 0.9290

Epoch 5/8

Train Loss: 0.1523 | Train Acc: 0.9435

Val Loss: 0.1701 | Val Acc: 0.9327

Saved best model

Epoch 6/8

Train Loss: 0.1458 | Train Acc: 0.9439

Val Loss: 0.1646 | Val Acc: 0.9364

Saved best model

Epoch 7/8

Train Loss: 0.1368 | Train Acc: 0.9468

Val Loss: 0.1644 | Val Acc: 0.9355

Epoch 8/8

Train Loss: 0.1265 | Train Acc: 0.9494

Val Loss: 0.1664 | Val Acc: 0.9318

Best Val Accuracy ResNet50 Fine-tuned: 0.936405529953917

Cell 6 — Evaluate fine-tuned model trên test

```
test_loss_res_ft, test_acc_res_ft, y_true_res_ft, y_pred_res_ft = evaluate(
    model_res_ft, test_loader, criterion, device
)

print(f"ResNet50 Fine-tuned Test Loss: {test_loss_res_ft:.4f}")
print(f"ResNet50 Fine-tuned Test Acc : {test_acc_res_ft:.4f}")
```

ResNet50 Fine-tuned Test Loss: 0.1856

ResNet50 Fine-tuned Test Acc : 0.9318

Cell 7 — Classification report

```
print(classification_report(
    y_true_res_ft,
    y_pred_res_ft,
    target_names=["other", "cafe"],
    digits=4
))
```

	precision	recall	f1-score	support
other	0.9261	0.9332	0.9297	524
cafe	0.9372	0.9305	0.9338	561
accuracy			0.9318	1085
macro avg	0.9316	0.9318	0.9317	1085
weighted avg	0.9318	0.9318	0.9318	1085

Cell 8 — F1 score

```
macro_f1_res_ft = f1_score(y_true_res_ft, y_pred_res_ft, average="macro")
weighted_f1_res_ft = f1_score(y_true_res_ft, y_pred_res_ft, average="weighted")

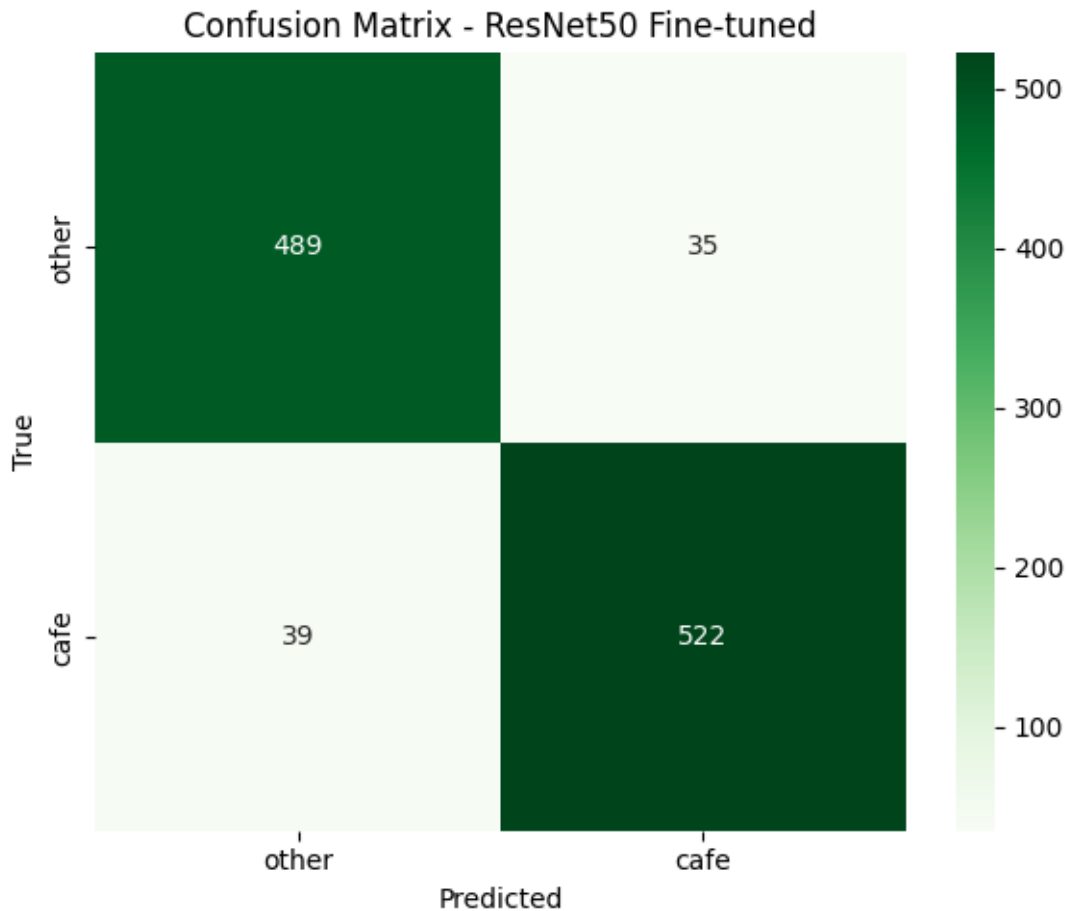
print(f"ResNet50 Fine-tuned Macro F1    : {macro_f1_res_ft:.4f}")
print(f"ResNet50 Fine-tuned Weighted F1: {weighted_f1_res_ft:.4f}")
```

```
ResNet50 Fine-tuned Macro F1    : 0.9317
ResNet50 Fine-tuned Weighted F1: 0.9318
```

Cell 9 — Confusion matrix

```
cm_res_ft = confusion_matrix(y_true_res_ft, y_pred_res_ft)

plt.figure(figsize=(6, 5))
sns.heatmap(
    cm_res_ft,
    annot=True,
    fmt="d",
    cmap="Greens",
    xticklabels=["other", "cafe"],
    yticklabels=["other", "cafe"]
)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix - ResNet50 Fine-tuned")
plt.tight_layout()
plt.show()
```



Cell 10 — Vẽ biểu đồ fine-tune

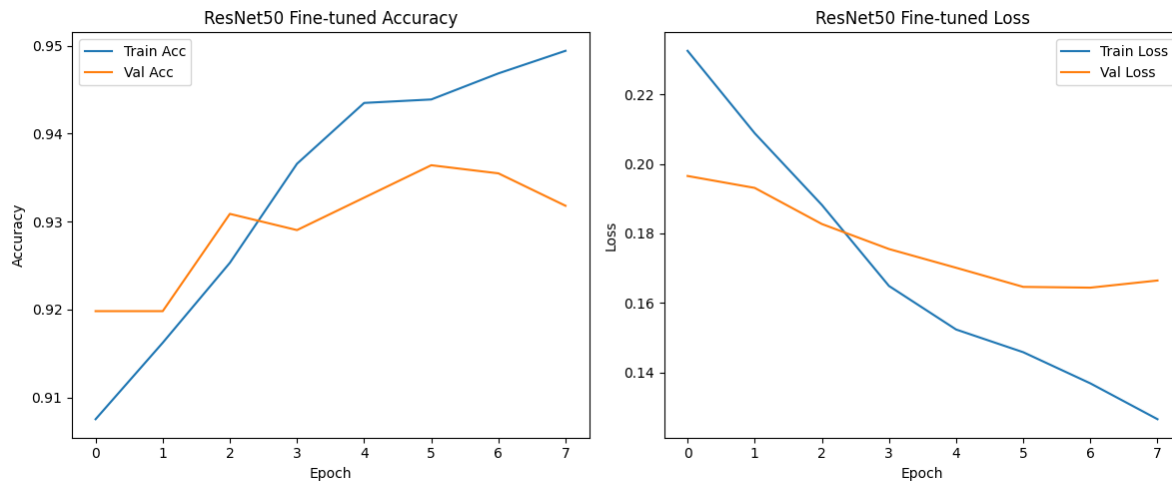
```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history_res_ft["train_acc"], label="Train Acc")
plt.plot(history_res_ft["val_acc"], label="Val Acc")
plt.title("ResNet50 Fine-tuned Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history_res_ft["train_loss"], label="Train Loss")
plt.plot(history_res_ft["val_loss"], label="Val Loss")
plt.title("ResNet50 Fine-tuned Loss")
```

```
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
```

```
plt.tight_layout()
plt.show()
```



Cell 11 — So sánh Stage 1 vs Fine-tuned

```
import pandas as pd

comparison_df = pd.DataFrame([
    {
        "Model": "EfficientNetB0 Stage 1",
        "Val Accuracy": 0.8737,
        "Test Accuracy": 0.8654,
        "Macro F1": 0.8654
    },
    {
        "Model": "ResNet50 Stage 1",
        "Val Accuracy": 0.9198,
        "Test Accuracy": 0.9097,
        "Macro F1": 0.9096
    },
    {
        "Model": "ResNet50 Fine-tuned",
        "Val Accuracy": best_val_acc_res_ft,
        "Test Accuracy": test_acc_res_ft,
```



```

        "Macro F1": macro_f1_res_ft
    }
])

comparison_df = comparison_df.sort_values(by=["Test Accuracy", "Macro F1"], ascending=False)
comparison_df

```

	Model	Val Accuracy	Test Accuracy	Macro F1
2	ResNet50 Fine-tuned	0.936406	0.931797	0.931734
1	ResNet50 Stage 1	0.919800	0.909700	0.909600
0	EfficientNetB0 Stage 1	0.873700	0.865400	0.865400

BƯỚC 7.1 — Lưu model cuối cùng

Cell 1 — Tạo thư mục lưu model

```

import os
import json
import shutil
import torch

FINAL_DIR = r"d:\Máy học 5 classes\binary_cafe_final_model"
os.makedirs(FINAL_DIR, exist_ok=True)

print("Final dir:", FINAL_DIR)

```

Final dir: d:\Máy học 5 classes\binary_cafe_final_model

Cell 2 — Lưu state_dict tốt nhất

```

src_model_path = "best_resnet50_finetuned_binary.pth"
dst_model_path = os.path.join(FINAL_DIR, "best_resnet50_finetuned_binary.pth")

shutil.copy(src_model_path, dst_model_path)

print("Đã lưu model vào:", dst_model_path)

```

Đã lưu model vào: d:\Máy học 5 classes\binary_cafe_final_model\best_resnet50_finetuned_binary

```
src_model_path = "best_resnet50_stage1_binary.pth"
dst_model_path = os.path.join(FINAL_DIR, "best_resnet50_stage1_binary.pth")

shutil.copy(src_model_path, dst_model_path)

print("Đã lưu model vào:", dst_model_path)
```

Đã lưu model vào: d:\Máy học 5 classes\binary_cafe_final_model\best_resnet50_stage1_binary.p

```
src_model_path = "best_efficientnetb0_stage1_binary.pth"
dst_model_path = os.path.join(FINAL_DIR, "best_efficientnetb0_stage1_binary.pth")

shutil.copy(src_model_path, dst_model_path)

print("Đã lưu model vào:", dst_model_path)
```

Đã lưu model vào: d:\Máy học 5 classes\binary_cafe_final_model\best_efficientnetb0_stage1_bi

Cell 3 — Lưu label mapping

```
index_to_label = {
    0: "other",
    1: "cafe"
}

label_to_index = {
    "other": 0,
    "cafe": 1
}

with open(os.path.join(FINAL_DIR, "index_to_label_resnet50_stage1_binary.json"), "w", encoding='utf-8') as f:
    json.dump(index_to_label, f, ensure_ascii=False, indent=2)

with open(os.path.join(FINAL_DIR, "label_to_index_resnet50_stage1_binary.json"), "w", encoding='utf-8') as f:
    json.dump(label_to_index, f, ensure_ascii=False, indent=2)

print("Đã lưu label mapping.")
```

Đã lưu label mapping.

BƯỚC 7.2 — Build lại model để inference

Ta cần build đúng kiến trúc ResNet50 binary rồi load weights vào.

Cell 4 — Build model inference

```
import torch.nn as nn
from torchvision.models import resnet50, ResNet50_Weights

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

def build_resnet50_binary_model(device):
    model = resnet50(weights=ResNet50_Weights.DEFAULT)

    in_features = model.fc.in_features
    model.fc = nn.Sequential(
        nn.Dropout(p=0.3),
        nn.Linear(in_features, 2)
    )

    model = model.to(device)
    return model

inference_model = build_resnet50_binary_model(device)
inference_model.load_state_dict(torch.load(dst_model_path, map_location=device))
inference_model.eval()

print("Đã load model inference.")
```

Đã load model inference.

BƯỚC 7.3 — Tạo transform cho ảnh inference

Transform này phải giống tinh thần lúc train:

resize 224x224

tensor

normalize theo ImageNet

Cell 5 — Inference transform

```

from torchvision import transforms
from PIL import Image

inference_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

```

BƯỚC 7.4 — Predict 1 ảnh

Cell 6 — Hàm predict 1 ảnh

```

import torch
import numpy as np

CLASS_NAMES = ["other", "cafe"]

def predict_one_image(image_path, model, device, class_names=CLASS_NAMES):
    image = Image.open(image_path).convert("RGB")
    x = inference_transform(image).unsqueeze(0).to(device)

    with torch.no_grad():
        outputs = model(x)
        probs = torch.softmax(outputs, dim=1).cpu().numpy()[0]

    pred_idx = int(np.argmax(probs))
    pred_label = class_names[pred_idx]
    confidence = float(probs[pred_idx])

    return {
        "predicted_label": pred_label,
        "confidence": confidence,
        "probabilities": {
            class_names[i]: float(probs[i]) for i in range(len(class_names))
        }
    }

```

Cell 7 — Hàm hiển thị kết quả 1 ảnh

```

import matplotlib.pyplot as plt

def show_prediction_one_image(image_path, model, device, class_names=CLASS_NAMES):
    result = predict_one_image(image_path, model, device, class_names)

    image = Image.open(image_path).convert("RGB")

    plt.figure(figsize=(6, 6))
    plt.imshow(image)
    plt.axis("off")
    plt.title(f"{result['predicted_label']} ({result['confidence']:.4f})")
    plt.show()

    print("Kết quả dự đoán:")
    print("Predicted label:", result["predicted_label"])
    print("Confidence      :", f"{result['confidence']:.4f}")
    print("Probabilities:")
    for label, score in result["probabilities"].items():
        print(f"  {label}: {score:.4f}")

```

Cell 8 — Test 1 ảnh

```

test_image_path = r"d:\Máy học 5 classes\test_binary\test1.jpg" # sửa lại
show_prediction_one_image(test_image_path, inference_model, device)

```

BƯỚC 7.5 — Predict một folder ảnh

Cell 9 — Hàm predict folder

```

VALID_EXTS = (".jpg", ".jpeg", ".png", ".bmp", ".webp")

def predict_folder(folder_path, model, device, class_names=CLASS_NAMES):
    files = [
        f for f in os.listdir(folder_path)
        if f.lower().endswith(VALID_EXTS)
    ]

    results = []

    for file_name in files:
        image_path = os.path.join(folder_path, file_name)
        result = predict_one_image(image_path, model, device, class_names)

```

```

        results.append({
            "file_name": file_name,
            "predicted_label": result["predicted_label"],
            "confidence": result["confidence"],
            "prob_other": result["probabilities"]["other"],
            "prob_cafe": result["probabilities"]["cafe"]
        })

    return results

```

Cell 10 — Hàm in kết quả folder

```

def print_folder_predictions(folder_path, model, device, class_names=CLASS_NAMES):
    results = predict_folder(folder_path, model, device, class_names)

    print(f"Tổng số ảnh: {len(results)}\n")

    for item in results:
        print(f"Ảnh: {item['file_name']}")
        print(f" Predicted : {item['predicted_label']}")
        print(f" Confidence: {item['confidence']:.4f}")
        print(f" prob_other: {item['prob_other']:.4f}")
        print(f" prob_cafe : {item['prob_cafe']:.4f}")
        print("-" * 50)

```

Cell 11 — Test folder ảnh

```

test_folder_path = r"d:\Máy học 5 classes\test_binary_folder" # sửa lại
print_folder_predictions(test_folder_path, inference_model, device)

```

BƯỚC 7.6 — Lưu kết quả folder thành CSV

Cell 12 — Xuất CSV

```

import pandas as pd

def predict_folder_to_csv(folder_path, model, device, output_csv_path, class_names=CLASS_NAMES):
    results = predict_folder(folder_path, model, device, class_names)
    df_result = pd.DataFrame(results)
    df_result.to_csv(output_csv_path, index=False, encoding="utf-8-sig")
    print("Đã lưu CSV tại:", output_csv_path)
    return df_result

```

Ví dụ chạy:

Python

```
output_csv = r"d:\Máy học 5 classes\binary_predictions.csv"
df_pred = predict_folder_to_csv(test_folder_path, inference_model, device, output_csv)
df_pred.head()
```

BƯỚC 7.7 — Hàm dùng cho hệ thống thực tế

Nếu sau này bạn tích hợp vào backend, bạn thường chỉ cần một hàm đơn giản như sau:

Cell 13 — Hàm dùng cho API/backend

```
def classify_cafe_or_other(image_path):
    result = predict_one_image(image_path, inference_model, device, CLASS_NAMES)
    return {
        "label": result["predicted_label"],
        "confidence": result["confidence"],
        "probabilities": result["probabilities"]
    }
```

Ví dụ:

Python

```
result = classify_cafe_or_other(test_image_path)
print(result)
```

Kết quả dạng:

Python

```
{
  "label": "cafe",
  "confidence": 0.9732,
  "probabilities": {
    "other": 0.0268,
    "cafe": 0.9732
  }
}
```